

Purbanchal University

School of Engineering and Technology, Biratnagar, Nepal

Department of Computer Science

Introduction to Web Page Technology **(For Diploma, BCA, MCA, BIT & BE)**



Prepared by: Naresh Kumar Shah
Puset (Asso. Prof.)

Introduction To Web Technology

What is Internet?

The Internet is essentially a global network of computing resources. You can think of the Internet as a physical collection of routers and circuits as a set of shared resources.

Some common definitions given in the past include –

- A network of networks based on the TCP/IP communications protocol.
- A community of people who use and develop those networks.
- A community of people who use and develop those networks.

Internet-Based Services

Some of the basic services available to Internet users are –

- **Email** – A fast, easy, and inexpensive way to communicate with other Internet users around the world.
- **Telnet** – Allows a user to log into a remote computer as though it were a local system.
- **FTP** – Allows a user to transfer virtually every kind of file that can be stored on a computer from one Internet-connected computer to another.
- **UseNet news** – A distributed bulletin board that offers a combination news and discussion service on thousands of topics.
- **World Wide Web (WWW)** – A hypertext interface to Internet information resources.

What is WWW?

- WWW stands for **World Wide Web**. A technical definition of the **World Wide Web** is (W3) – All the resources and users on the Internet that are using the Hypertext Transfer Protocol (HTTP).
- The **World Wide Web** is a system of interlinked hypertext documents accessed via the Internet. With a Web browser, one can view Web pages that may contain text, images, videos, and other multimedia and navigate between them using hyperlinks.
- The World Wide Web enabled the spread of information over the Internet through an easy-to-use and flexible format. It thus played an important role in popularizing use of the Internet, to the extent that the World Wide Web has become a synonym for Internet, with the two being conflated in popular use.
- In simple terms, The World Wide Web is a way of exchanging information between computers on the Internet, tying them together into a vast collection of interactive multimedia resources.



What is HTTP?

HTTP stands for **Hypertext Transfer Protocol**. This is the protocol being used to transfer hypertext documents that makes the World Wide Web possible.

A standard web address such as Yahoo.com is called a URL and here the prefix **http** indicates its protocol

What is URL?

URL stands for **Uniform Resource Locator**, and is used to specify addresses on the World Wide Web. A URL is the fundamental network identification for any resource connected to the web (e.g., hypertext pages, images, and sound files).

A URL will have the following format – protocol://hostname/other_information

The protocol specifies how information is transferred from a link. The protocol used for web resources is HyperText Transfer Protocol (HTTP). Other protocols compatible with most web browsers include FTP, telnet, newsgroups, and Gopher.

The protocol is followed by a colon, two slashes, and then the domain name. The domain name is the computer on which the resource is located.

Links to particular files or subdirectories may be further specified after the domain name. The directory names are separated by single forward slashes.

The first part of the address is called a *protocol identifier* and it indicates what protocol to use and the second part is called a *resource name* and it specifies the IP address or the domain name where the resource is located. The protocol identifier and the resource name are separated by a colon and two forward slashes

For example, the two URLs below point to two different files at the domain *pcwebopedia.com*. The first specifies an executable file that should be fetched using the FTP protocol; the second specifies a Web page that should be fetched using the HTTP protocol:

<ftp://www.pcwebopedia.com/stuff.exe> <http://www.pcwebopedia.com>

What is Website? (Collection of various pages written in HTML markup language)

A Web page is a document, typically written in HTML that is almost always accessible via HTTP, a protocol that transfers information from the Web server to display in the user's Web browser.

A **web site** is a collection of Web pages, images, videos or other digital assets that is hosted on one or more web servers, usually accessible via the Internet. Similarly, there are millions of websites available on the web. Each page available on the website is called a *web page* and first page of any website is called *home page* for that site.

Some functional website may be:

- a Commercial Website
- a Personal Website
- a Government Website
- a Non-profit organization website

Websites are written in, or dynamically converted to, HTML (Hyper Text Markup Language) and are accessed using a software interface classified as a user agent. Web pages can be viewed or otherwise accessed from a range of computer-based and Internet-enabled devices of various sizes, including desktop computers, laptops, PDAs and cell phones.

A website is hosted on a computer system known as a web server, also called an HTTP server, and these terms can also refer to the software that runs on these systems and that retrieves and delivers the Web pages in response to requests from the website users. Apache is the most commonly used Web server software and Microsoft's Internet Information Server (IIS) is also commonly used.

What is Web Server? (A computer that hosts a website on the Internet)

The term **Web Server** can mean one of two things: A computer program that is responsible for accepting HTTP requests from clients (user agents such as web browsers), and serving them HTTP responses along with optional data contents, which usually are web pages such as HTML documents and linked objects (images, etc.)

Every Website sits on a computer known as a Web server. This server is always connected to the internet. Every Web server that is connected to the Internet is given a unique

address made up of a series of four numbers between 0 and 256 separated by periods. For example, 68.178.157.132 or 68.122.35.127

When you register a Web address, also known as a domain name, you have to specify the IP address of the Web server that will host the site. We will see different type of Web servers.

What is Web Browser?

Web Browsers are software installed on your PC. To access the Web you need a web browsers, such as Netscape Navigator, Microsoft Internet Explorer or Mozilla Firefox.

Currently you must be using any sort of Web browser while you are navigating through my site tutorialspoint.com. On the Web, when you navigate through pages of information this is commonly known as *browsing or surfing*.

What is SMTP Server?

SMTP stands for **S**imple **M**ail **T**ransfer **P**rotocol **S**erver. This server takes care of delivering emails from one server to another server. When you send an email to an email address, it is delivered to its recipient by a SMTP Server.

What is ISP?

ISP stands for **I**nternet **S**ervice **P**rovider. They are the companies who provide you service in terms of internet connection to connect to the internet. You will buy space on a Web Server from any Internet Service Provider. This space will be used to host your Website.

What is HTML?

HTML stands for **H**yper **T**ext **M**arkup **L**anguage. This is the language in which we write web pages for any Website. Even the page you are reading right now is written in HTML. This is a subset of Standard Generalized Mark-Up Language (SGML) for electronic publishing, the specific standard used for the World Wide Web.

What is Hyperlink?

A hyperlink or simply a link is a selectable element in an electronic document that serves as an access point to other electronic resources. Typically, you click the hyperlink to access the linked resource. Familiar hyperlinks include buttons, icons, image maps, and clickable text links.

What is DNS?

DNS stands for **D**omain **N**ame **S**ystem. When someone types in your domain name, www.example.com, your browser will ask the Domain Name System to find the IP that hosts your site. When you register your domain name, your IP address should be put in a DNS along with your domain name. Without doing it your domain name will not be functioning properly.

What is W3C?

W3C stands for **W**orld **W**ide **W**eb Consortium which is an international consortium of companies involved with the Internet and the Web. The W3C was founded in 1994 by Tim Berners-Lee, the original architect of the World Wide Web. The organization's purpose is to develop open standards so that the Web evolves in a single direction rather than being splintered among competing factions. The W3C is the chief standards body for HTTP and HTML.

Tier Technology:

Tier application architecture is a modular client-server architecture that consists of a presentation tier, an application tier and a data tier. The data tier stores information, the

application tier handles logic and the presentation tier is a graphical user interface (GUI) that communicates with the other two tiers. Tier Technology developed and maintained as independent modules on separate platforms.”

A “tier” in this case can also be referred to as a “layer”. The three tiers, or layers, involved include:

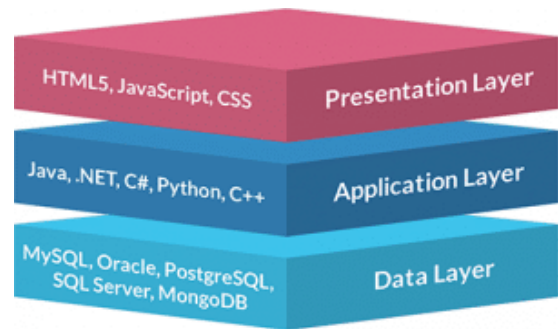
Presentation tier

This tier, which is built with **HTML5**, cascading style sheets (**CSS**) and **JavaScript**, is deployed to a computing device through a web browser or a web-based application. The presentation tier communicates with the other tiers through application program interface (API) calls.

Application tier

An Application Layer that uses an application server also referred to as the logic tier and processes the business logic for the application’s core functions. This might be written in C#, Java, C++, Python, Ruby, etc.

The underlying application tier can either be hosted on distributed servers in the cloud or on a dedicated in-house server, depending on how much processing power the application requires.



Data tier

The data tier consists of a database and a program for managing read and write access to a database. This tier may also be referred to as the storage tier and can be hosted on-premises or in the cloud. Popular database systems for managing read/write access include MySQL, PostgreSQL, Microsoft SQL Server and MongoDB.

Benefits of a 3-tier app architecture

The benefits of using a 3-layer architecture include improved horizontal scalability, performance and availability. With three tiers, each part can be developed concurrently by different team of programmers coding in different languages from the other tier developers. Because the programming for a tier can be changed or relocated without affecting the other tiers, the 3-tier model makes it easier for an enterprise or software packager to continually evolve an application as new needs and opportunities arise. Existing applications or critical parts can be permanently or temporarily retained and encapsulated within the new tier of which it becomes a component.

Home page

- The page typically encountered first on a website that usually contains links to the other pages of the site
- A home page is the default or front page of a site. It is the first page that visitors see when they load a URL. Web managers can control the home page as a way of directing the user experience.
- The homepage or home page is the name of the main page of a website where visitors can find hyperlinks to other pages on the site. By default, the homepage on all web servers is index.html, however, can also be index.htm, index.php, or whatever the developer decide.

WEB PAGE(A document which can be displayed in a web browser)

A **web page** or **webpage** is a resource of information that is suitable for the World Wide Web and can be accessed through a web browser. This information is usually in HTML or XHTML format, and may provide navigation to other web pages via hypertext links. Web pages are requested and served from web servers using Hypertext Transfer Protocol (HTTP).

Web pages may consist of files of static text stored within the web server's file system (static web pages), or the web server may construct the HTML for each web page when it is requested by a browser (dynamic web pages). Client-side scripting can make web pages more responsive to user input once in the client browser.

Elements of a web page

A web page, as an information set, can contain numerous types of information, which is able to be seen, heard or interact by the end user:

1. Perceived information:
 - Textual information
 - Non-textual information
 - Static images.
 - Animated images
 - Audio
 - Video
 - Interactive information
 - For "on page" interaction
 - Interactive text
 - Interactive illustrations
 - Buttons
 - For "between pages" interaction
 - Hyperlinks
 - Forms
2. Internal information:
 - Comments
 - Metadata
 - Diagrammation and style information.
 - Scripts.

Static Web pages:

Static Web pages are very simple. It is written in languages such as HTML and CSS only, with no Scripting. For static web pages when a server receives a request for a web page, then the server

sends the response to the client without doing any additional **Static Web Page** process. A static website is one which does **not make use of any external database** and is **written completely in HTML**. And these web pages are seen through a web browser. In static web pages, Pages will remain the same until someone changes it manually.



Pros

- Less complex to develop
- Usually faster loading times and smaller page size
- All assets can be cached for even faster page speed

Cons

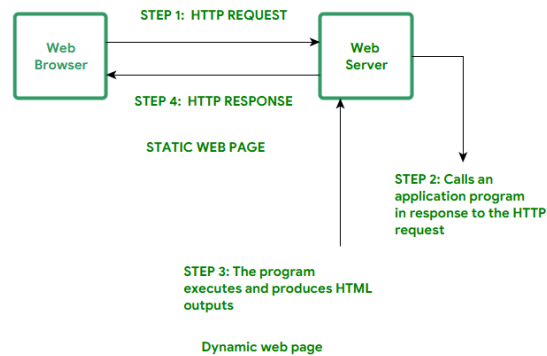
- Non-interactive
- Updates must be made directly within the HTML code
- Cannot perform the complex functionality required in many modern sites

Dynamic Web Pages:

A dynamic website is a website that not only uses HTML and CSS, but includes website scripting as well. There are two main reasons why you'd want to use website scripting on your site:

1. you want an interactive web app that people can use, not just read
2. you want to be able to share HTML code between your pages.

The website is a dynamic website, for both of the above reasons. Dynamic Web Pages are written in languages such as JavaScript, PHP, CGI, AJAX, ASP, ASP.NET, etc. In dynamic web pages, the Content of pages is different for different visitors. Dynamic Web Page can **generate content based on the user**. It takes more time to load than the static web page. Dynamic web pages are used where the information is changed frequently, for example, stock prices, weather information, etc.



Pros

- Dynamic content generation based on user
- Easily update content when used with a CMS
- Able to use it in more complex scenarios such as an online store, forum, social media platform, etc

Cons

- May be slower than a static site due to the increase in required resources
- More complex to develop
- Greater risk of security issues

Difference between Static and Dynamic Web Pages:

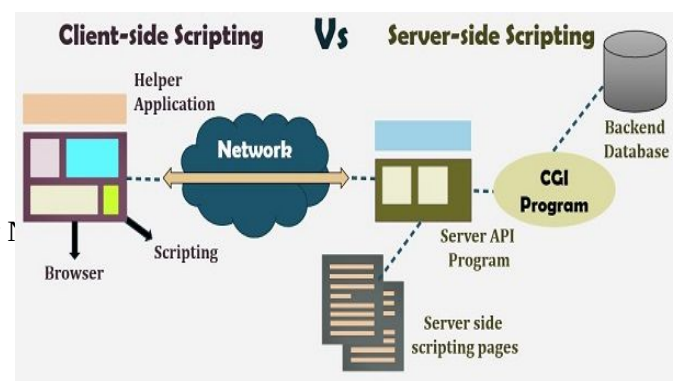
Static web pages	Dynamic web pages
In static web pages, Pages will remain same until someone changes it manually.	In dynamic web pages, Content of pages are different for different visitors.
Static Web Pages are simple in terms of complexity.	Dynamic web pages are complicated.
In static web pages, Information are change rarely.	In dynamic web page, Information are change frequently.
Static Web Page takes less time for loading than dynamic web page.	Dynamic web page takes more time for loading.
In Static Web Pages, database is not used.	In dynamic web pages, database is used.
Static web pages are written in languages such as: HTML, JavaScript, CSS, etc.	Dynamic web pages are written in languages such as: CGI, AJAX, ASP, ASP.NET, etc.

CLIENT-SIDE & SERVER-SIDE

Script:

A script is generally a series of program or instruction, which has to be executed on other program or application. As we know that the web works in a client-server environment. The client-

Prepared By I



side script executes the code to the client side which is visible to the users while a server-side script is executed in the server end which users cannot see.

Server: The Server is responsible for serving the web pages depending on the client/end user requirement. It can be either static or dynamic.

Client: A client is a party that requests pages from the server and displays them to the end user. In general a client program is a web browser.

The main difference between server-side scripting and client-side scripting is that the server side scripting involves server for its processing. On the other hand, client-side scripting requires browsers to run the scripts on the client machine but does not interact with the server while processing the client-side scripts.

Client-Side Scripting:

- The term **client-side** refers to operations that are performed by the client in a client-server relationship.
- Client-side scripting is performed to generate a code that can run on the client end (browser) without needing the server side processing.
- Basically, these types of scripts are placed inside an HTML document.
- The client-side scripting can be used to examine the user's form for the errors before submitting it and for changing the content according to the user input.
- As mentioned before, the web requires three elements for its functioning which are, client, database and server.
- The effective client-side scripting can significantly reduce the server load. It is designed to run as a scripting language utilizing a web browser as a host program. For example, when a user makes a request via browser for a webpage to the server, it just sent the HTML and CSS as plain text, and the browser interprets and renders the web content in the client end.

Client-side scripting languages:

HTML: It is the fundamental building blocks of web programming which provides the frame to the website. It describes the arrangement of the content. Examples are: HTML, CSS, Javascript; **CSS:** provides the way to design the graphic elements which help in making the appearance of the web application more attractive. **JavaScript:** It is also a client-side scripting language which essentially devised for the specific purpose, but currently there are various JavaScript frameworks used as server-side scripting technology.

Server-side

- **Server-side** refers to operations that are performed by the server in a client-server relationship in computer networking.
- Is a technique of programming for producing the code which can run software on the server side, in simple words any scripting or programming that can run on the web server is known as server-side scripting.
- The operations like customization of a website, dynamic change in the website content, response generation to the user's queries, accessing the database, and so on are performed at the server end.
- The server-side scripting constructs a communication link between a server and a client (user). Earlier the server side scripting is implemented by the **CGI (Common Gateway**

Interface) scripts. The CGI was devised to execute the scripts from programming languages such as C++ or Perl on the websites.

- The server-side involves three parts: server, database, API's and back-end web software developed by the server-side scripting language.
- When a browser sends a request to the server for a webpage consisting of server-side scripting, the web server processes the script prior to serving the page to the browser. Here the processing of a script could include extracting information from a database, making simple calculations, or choosing the appropriate content that is to be displayed in the client end. The script is being processed and the output is sent to the browser. The web server abstracts the scripts from the end user until serving the content, which makes the data and source code more secure.

Server-side scripting languages: After the advent of CGI, multiple programming languages were evolved such as PHP, Python, Ruby, ColdFusion, C#, Java, C++ and so on for server-side scripting.

Key Differences Between Server-side Scripting and Client-side Scripting

Server-side scripting is used at the backend, where the source code is not viewable or hidden at the client side (browser). On the other hand, client-side scripting is used at the front end which users can see from the browser.

When a server-side script is processed it communicates to the server. As against, client-side scripting does not need any server interaction.

Server-side scripting is useful in customizing the web pages and implements the dynamic changes in the websites. Conversely, the client-side script can effectively minimize the load to the server.

Server-side scripting is more secure than client-side scripting as the server side scripts are usually hidden from the client end, while a client-side script is visible to the users.

Client-side scripting and server-side scripting work in a coordinated manner with each other. However, both the scripting techniques are very different, where the client-side scripting emphasize on making the interface of the web application or website more appealing and functional. Conversely, server-side scripting emphasizes on the data accessing methods, error handling and fast processing etcetera.

We can explain this entire mechanism of client server using the following:

- The user opens his web browser (client)
- The user starts browsing. (for example: <http://c-sharpcorner.com>)
- The client forwards this request to the server, for accessing their web page.
- The server then acknowledges the request and replies back to the client program.
(An access link to that web page)
- The client then receives the page source and renders it.
(Into a viewable/under a stable website)
- Now the user types into search bar
- The client then submits data to the server
- The server processes the data and replies back with a related search result
- The client again renders it back for the user's view

We would also define a script as a set of instructions. For Web pages they are instructions either to the Web server(server-side scripting) or to the Web Browser (client-side scripting). Let's have a look how they differ from each other :

Server-side scripting	Client-side scripting
Works in the back end which could not be visible at the client end. Relatively secure.	Works at the front end and script are visible among the users. Insecure
Server side scripting is used to create dynamic pages based a number of conditions when the users browser makes a request to the server.	Client side scripting is used when the users browser already has all the code and the page is altered on the basis of the users input.
The Web Server executes the server side scripting that produces the page to be sent to the browser.	The Web Browser executes the client side scripting that resides at the user's computer.
Server executes server-side scripts to send out a page but it does not execute client-side scripts	The browser receives the page sent by the server and executes the client-side scripts.
Server side scripting is used to connect to the databases that reside on the web server.	Client side scripting cannot be used to connect to the databases on the web server.
Server side scripting can access the file system residing at the web server.	Client side scripting can't access the file system that resides at the web server.
The settings that belong to Web server can be accessed using Server side scripting.	The files and settings that are local at the user's computer can be accessed using Client side scripting.
Server side scripting can't be blocked by the user.	Client side scripting is possible to be blocked by the user
Response from a server-side script is slower as compared to a client-side script because the scripts are processed on the remote computer.	Response from a client-side script is faster as compared to a server-side script because the scripts are processed on the local computer.
Server-side Uses • It processes the user input • Displays the requested pages • Structure web applications • Interaction with servers/storages • Interaction with databases • Querying the database • Encoding of data into HTML • Operations over databases like delete, update.	Client-side Uses • Makes interactive web pages • Make stuffs work dynamically • Interact with temporary storage • Works as an interface between user and server • Sends requests to the server • Retrieval of data from Server • Interact with local storage • Provides remote access for client server program
Examples of Server side scripting languages : PHP, JSP, ASP, ASP.Net, Ruby, Perl n many more.	Examples of Client side scripting languages : Javascript, VB script, etc.

Web Protocols

The Internet relies on a number of *protocols* in order to function properly. A protocol is a set of rules that help in governing the way a particular technology will function for communication. In other words, it can be said that the protocols are digital languages implemented in the form of networking algorithms. There are different networks and network protocols, user's use while surfing.

It is simply a standard for enabling the connection, communication, and data transfer between two places on a network. Here are some of the key protocols that are used for transferring data across the Internet.

1. HTTP (Hypertext Transfer Protocol) It is the backbone of World Wide Web (WWW) It is the standard protocol for transferring web pages (and their content) across the Internet. HTTP is designed for transferring a hypertext among two or more systems. HTML tags are used for creating links. These links may be in any form like text or images. HTTP is designed on Client-server principles which allow a client system for establishing a connection with the server machine for making a request. The server acknowledges the request initiated by the client and responds accordingly. The default port of HTTP is 80.

2. **Hyper Text Transfer Protocol Secure (HTTPS):** HTTPS is abbreviated as Hyper Text Transfer Protocol Secure is a standard protocol to secure the communication among two computers one using the browser and other fetching data from web server. HTTP is used for transferring data between the client browser (request) and the web server (response) in the hypertext format, same in case of HTTPS except that the transferring of data is done in an encrypted format. So it can be said that https thwart hackers from interpretation or modification of data throughout the transfer of packets. The default port of HTTPS is 443.
3. **File Transfer Protocol (FTP):** It is used to transfer files across the Internet. FTP is commonly used by web developers to publish updates to a website (i.e. to upload a new version of the website). It establishes two TCP connections: **Control Connection** to authenticate the user, and **Data Connection** to transfer the files. The default port of FTP is 20/21.

Where HTTP is used for displaying the file in your browser, FTP is used simply to transfer the file from one computer to a specified location on another computer. You can use FTP to transfer the files from your computer to a remote computer (such as a web server), or to transfer from the remote computer to your local computer.
4. **Transmission Control Protocol (TCP):** TCP is a popular communication protocol which is used for communicating over a network. It divides any message into series of packets that are sent from source to destination and there it gets reassembled at the destination.
5. **Internet Protocol (IP):** IP is designed explicitly as addressing protocol. It is mostly used with TCP. The IP addresses in packets help in routing them through different nodes in a network until it reaches the destination system. TCP/IP is the most popular protocol connecting the networks.
6. **User Datagram Protocol (UDP):** UDP is a substitute communication protocol to Transmission Control Protocol implemented primarily for creating loss-tolerating and low-latency linking between different applications.
7. **Post office Protocol (POP):** The Post Office Protocol version 3(POP3) is designed for receiving incoming E-mails. The default port of POP3 is 110
8. **Simple mail transport Protocol (SMTP):** SMTP is designed to send and distribute outgoing E-Mail. The default port of SMTP is 25.
9. **Telnet:** Telnet is a set of rules designed for connecting one system with another. The connecting process here is termed as remote login. The system which requests for connection is the local computer, and the system which accepts the connection is the remote computer. The default port of Telnet is 23.
10. **Gopher:** Gopher is a collection of rules implemented for searching, retrieving as well as displaying documents from isolated sites. Gopher also works on the client/server principle.

Introduction To Free and Open Source Software

What is FOSS?

Free and open source software (FOSS), is that type of software, which lets its users to do any kind of task, read the source of software, according to our work and need, that software can be

changed and modified and lets us distribute that software's changed and unchanged features with full permission. Some of the features are

- There are two terms to separate “free” and “open source” in FOSS.
- Free as in freedom of speech not as in free beer.
- Open mainly points to Open Source Code of the program, available for others.
- So free and open source software is in some ways free but the source code is available for analysis

What is the use of FOSS?

- Without any distraction, this software can be used for any work.
- Software's internal work is for methodology study and scour.
- For exchange of software and knowledge.
- To publicize the modified work.

Benefits of FOSS

- FOSS software can be made available in minimum and low cost investment.
- High level safety, classy, reliable and stable software is provided.
- FOSS software is openly limited, so it's not important that only one user must be dependent.
- Software can be developed with source in case of local failure.
- While using Free and open source software, we can have respect of national and international law's copy and no need of software piracy.
- International level, local resources can be developed.

Why to use FOSS ?

- Using FOSS, you can also gain minimum or low investment, reliable, stable and highly defined safe software.
- In worldly knowledge, tradition, including information and communication sector, independence protest can be done for your software's security.
- For this software's license, simply: you don't have to pay any kind of amount and in own office, home or any other place, it can be installed for free.
- If you are tired of virus in your computer, if you use FOSS software then it won't have any effect on your computer. Even, there is no need of anti-virus.
- In software importing, you will be less dependent.
- The users can scour the software, the way they want to and use it.
- Software can be used independently.

Development, Use and Promotion of FOSS in Nepal

In the 1990's start, since there was debate about independent software in the community, “Free and Open Source software FOSS” had synonyms. “Free software” named word was defined by Richard Stalmyand in 1983. The definition of free software can be found since January 1989. Nepal's internet service and other business were using this type of technology. In 2003, Noble Prize Library started working in “Nepal Linux,” which introduced its 1st edition in 2005. In 2007, 2008 and 2009's World Software Independent Day program competition, among 120 countries' more than 500 groups, Nepal became successful in getting in excellent 3. However, FOSS has given priority by the Nepal Government in its Information Technology policy rules.

“Free software” is a matter of liberty, not price.

- Run (freedom 0)

- Study how program works (freedom 1)
- Redistribute copies (freedom 2)
- Modify / improvise (freedom 3)
- So you may get the software at no cost but as per the license you can **run, copy, distribute, study, change and improve the software.**

Advantages of FOSS

• The availability of the source code and the right to modify	• The right to use the software in any way
• Core software is free	• Evolving software
• Encourages hands on	• Not tied to a single vendor
• Big community to support	• Easy Localization
• Better Security	• Reliability & Stability

Disadvantages of FOSS

- No maintenance and support (unless purchased separately)
- No warranties regarding media, viruses, and performance
- Staff must be open source savvy
- License terms are NOT standard/ Problems connected to intellectual property
- Hard to implement

FOSS & Developing Nation Like Ours

• Cost Factor(\$200 for 1 copy of MS windows XP)	• Vendor Independence/ open standards
• Legal issues with software	• Localization
• Security, reliability and stability	• Employment
• Collaborative problem solving	

Who's Doing it

FOSS Nepal Community, www.fossnepal.org.np / fossnepal.org.np

Madan Pursakar Pustakalaya (MPP)	IOE – LTSP initiative
BellaSAP	OLCP Nepal etc

An Introduction to Open Source Software

Essentially, open source software is free software for your business or personal use. Open source developers freely share their knowledge and make the source code available to the public. The software is distributed with a license which allows other developers can modify it and/or add to it. Some examples of open source software are: WordPress, Ubuntu, and Mozilla, creators of the Firefox browser.

Generally, the major difference between open source software and commercially available programs is you don't pay for it. That said some developers make free products for all users while other developers create commercially available products. Some developers create a free and paid version of their products. The free program is a stripped down version of the complete version which gives you full functionality.

Many commercial programs offer a 30-day free trial, but that might not enough time to get up to speed, which could force you to make a buying decision before being ready to do so. In contrast, many open source programs are free, so you can take all the time you need to learn how they work.

It's also important to realize that when you buy commercial software you don't actually own it but, in fact, are granted a license. If you have any doubts, take a look at the EULA (End User License Agreement which comes with the software). Some features of Open Source Software

- Source code is open (available for user)
- You can modify source code depending upon license: not totally free
- Source code is not free in the sense of users being able to modify it and redistribute it.
- There are 10 points to qualify as open source software, some are:
 - ✓ Free redistribution
 - ✓ Availability of source code
 - ✓ Possibility of derived works etc

Advantages

- Open source software allows you to make choices, rather than being locked into the design and development of a commercial application.
- Open source software is under constant development which addresses vulnerabilities, bug fixes, enhancements, and more.
- You can modify the software as necessary for your own purposes.
- Some open source programs give the user the option of automatic updates, which keep the software current. i.e. WordPress.
- A number of open source programs have a core application which can be enhanced by the use of plug-ins and themes. i.e. WordPress and Joomla.
- Open source software offers a tremendous amount of flexibility.
- Open source software is potentially more secure than commercial programs because the code is constantly being scrutinized by many programmers, not just a select few.
- Many open source programs can be installed on your computer, unlike a proprietary system which you can use, but where you have no control. If the software owner doesn't like what you are doing they can wipe out your hard work overnight.

Disadvantages

- If you don't know how to write code you're at the mercy of those who do unless you have the budget to pay for modifications or learn how to code yourself.
- If the author of a product no longer supports the software you might be out of luck unless the development is picked up by other programmers. That said, you're not at the mercy of a software company which goes out of business (and refuses to allow licensing to a 3rd party). You can take over development yourself if you wish.
- Open source is sometimes referred to as 'open wallet' in the sense that it may cost you more to have open source code modified than it would cost you to buy a commercial program.
- Unless there's a structure in place to ensure the quality of the code it might wind up with many changes, bug fixes, and patches, all of which can make the code more complex and/or degrade the quality, which in turn leads to more maintenance.
- The software might not be well-documented, which could make it difficult to learn.
- Vulnerabilities in the software can be exploited by hackers. i.e. A common way of attacking WordPress installations is through a SQL injection of malicious code into the database. Depending on the severity of the attack it might be possible to repair. If not, you will have to rebuild the site. Another option is to make sure you have backups. This will save you a lot of time if you need to restore the site.

Free Software Vs Open Source Software

	Free Software	Open Source Software
1	Free Software is social movement	Open Source focuses on practical application
2	For the free software movement, non-free software is social problem and free software is the solution.	For the open source movement, non free software is a suboptimal solution
3	Free software: GNU, firebox, LibreOffice, VLC	Open Source: Apache, Android, Hadloop, Varnish, and Linux.
4	Free Software is not allowed to work with proprietary software	Open Source Software is allowed to work with proprietary software
5	Modified code derived from free software, should also be released as free software.	Coder of open source software can specify whether a user is allowed to redistribute the modified code or not
6	Free software is open source software	Open source software may not necessarily be free software.

Open source Software Vs Proprietary Software

Open source Software	Proprietary Software
It refers to the software that is developed and tested through open collaboration.	It refers to the software that is solely owned by the individual or the organization that developed it.
Anyone with the academic knowledge can access, inspect, modify, and redistribute the source code.	Only the owner or publisher who holds the legal property rights of the source code can access it.
The project is managed by an open source community of developers and programmers.	The project is managed by a closed group of individuals or team that developed it.
They are not aimed at unskilled users outside of the programming community.	They are focused on a limited market of both skilled and unskilled end users.
It provides better flexibility which means more freedom which encourages innovation.	There is a very limited scope of innovation with the restrictions and all.
Examples: Android, Firefox, LibreOffice, Ubuntu, FreeBSD, Drupal, GNOME, etc.	Examples: Windows, macOS, iTunes, Google Earth, Adobe Flash Player, etc.

Proprietary software licenses

The hallmark of proprietary software licenses is that the software publisher grants the use of one or more copies of software under the end-user license agreement (EULA), but ownership of those copies remains with the software publisher (hence use of the term "proprietary"). This feature of proprietary software licenses means that certain rights regarding the software are reserved by the software publisher. Therefore, it is typical of EULAs to include terms which define the uses of the software, such as the number of installations allowed or the terms of distribution.

The most significant effect of this form of licensing is that, if ownership of the software remains with the software publisher, then the end-user *must* accept the software license. In other words, without acceptance of the license, the end-user may not use the software at all. One example of such a proprietary software license is the license for Microsoft Windows. As is usually the case with proprietary software licenses, this license contains an extensive list of activities which are restricted, such as: reverse engineering, simultaneous use of the software by multiple users, and publication of benchmarks or performance tests.

The most common licensing models are per single user (named user, client, node) or per user in the appropriate volume discount level, while some manufacturers accumulate existing licenses. These open volume license programs are typically called open license program (OLP), transactional license program (TLP), volume license program (VLP) etc. and are contrary to the contractual license program (CLP), where the customer commits to purchase a certain number of licenses over a fixed period (mostly two years). Licensing per concurrent/floating user also occurs, where all users in a network have access to the program, but only a specific number at the same time. Another license model is licensing per dongle, which allows the owner of the dongle to use the program on any computer. Licensing per server, CPU or points, regardless the number of users, is common practice, as well as site or company licenses. Sometimes one can choose between perpetual (permanent) and annual license. For perpetual licenses, one year of maintenance is often required, but maintenance (subscription) renewals are discounted. For annual licenses, there is no renewal; a new license must be purchased after expiration. Licensing can be host/client (or guest), mailbox, IP address, domain etc., depending on how the program is used. Additional users are *inter alia* licensed per extension pack (e.g. up to 99 users), which includes the base pack (e.g. 5 users). Some programs are modular, so one will have to buy a base product before they can use other modules.

Software licensing often also includes maintenance. This, usually with a term of one year, is either included or optional, but must often be bought with the software. The maintenance agreement (contract) typically contains a clause that allows the licensee to receive minor updates (V.1.1 => 1.2), and sometimes major updates (V.1.2 => 2.0). This option is usually called update insurance or upgrade assurance. For a major update, the customer has to buy an upgrade, if it is not included in the maintenance agreement. For a maintenance renewal, some manufacturers charge a reinstatement (reinstallment) fee retroactively per month, in the event that the current maintenance has expired.

Maintenance sometimes includes technical support. When it does, the level of technical support, which are commonly named gold, silver and bronze, can vary depending on the communication method (i.e. e-mail versus telephone support), availability (e.g. 5x8, 5 days a week, 8 hours a day) and reaction time (e.g. three hours). Support is also licensed per incident as an incident pack (e.g. five support incidents per year).

Many manufacturers offer special conditions for schools and government agencies (EDU/GOV license). Migration from another product (crossgrade), even from a different manufacturer (competitive upgrade) is offered

Proprietary software refers to any software that has a copyright and has limits to use and distribution. These limits are imposed by the developer, publisher, or vendor. It is the property of the owner and can be used with specified conditions

It may also be referred to as closed-source or commercial software. This form of software is typically commercial software that consumers can purchase, lease, or license from the developer. It does not allow a user to have access to the source code.

Consumers can purchase proprietary software for a fee but may not distribute or copy it in any way.

The majority of software is proprietary and is developed by ISVs, or independent software vendors. There are restrictions imposed by the developer or vendor that are elaborated in the end-user license agreement, or the EULA. You will also find the restrictions listed in the Terms of Service for the software.

The user of the software has to accept the agreement before the software can be installed or used. The vendor or developer can opt to take legal action against users who violate the terms of use.

Types of Proprietary Software

Source code is a way of writing software from a human standpoint using programming language before converting to machine code. Almost all of the software created by Microsoft is proprietary, which includes Windows products and Microsoft Office. There are some products in the Microsoft family that are free, including Internet Explorer.

Other creators of proprietary software include:

• Adobe	• Borland	• IBM
• Macromedia	• Oracle	• Sun Microsystems

Free and open-source software licenses

There are several organizations in the FOSS domain who give out guidelines and definitions regarding software licenses. Free Software Foundation maintains non-exhaustive lists of software licenses following their The Free Software Definition and licenses which the FSF considers non-free for various reasons. The FSF distinguishes additionally between free software licenses that are compatible or incompatible with the FSF license of choice, the copyleft GNU General Public License. The Open Source Initiative defines a list of certified open-source licenses following their The Open Source Definition. Also the Debian project has a list of licenses which follow their Debian Free Software Guidelines.

Free and open-source licenses are commonly classified into two categories: Those with the aim to have minimal requirements about how the software can be redistributed (permissive licenses), and the protective share-alike (copyleft Licenses).

An example of a copyleft free software license is the often used GNU General Public License (GPL), also the first copyleft license. This license is aimed at giving and protecting all users unlimited freedom to use, study, and privately modify the software, and if the user adheres to the terms and conditions of the GPL, freedom to redistribute the software or any modifications to it. For instance, any modifications made and redistributed by the end-user must include the source code for these, and the license of any derivative work must not put any additional restrictions beyond what the GPL allows.

Examples of permissive free software licenses are the BSD license and the MIT license, which give unlimited permission to use, study, and privately modify the software, and includes only minimal requirements on redistribution. This gives a user the permission to take the code and use it as part of closed-source software or software released under a proprietary software license.

Open source licenses are licenses that comply with the Open Source Definition — in brief, they allow software to be freely used, modified, and shared. To be approved by the Open Source Initiative (also known as the OSI), a license must go through the Open Source Initiative's license review process. Popular Licenses:

The following OSI-approved licenses are popular, widely used, or have strong communities:

Apache License 2.0	BSD 3-Clause "New" or "Revised" license
BSD 2-Clause "Simplified" or "FreeBSD" license	GNU General Public License (GPL)
GNU Library or "Lesser" General Public License (LGPL)	MIT license

Mozilla Public License 2.0	Common Development and Distribution License
Eclipse Public License version 2.0	

File creation:

Files that travel across the internet from a web server to web clients, which made the request, are known as web pages. The person who can develop web pages is called web developer.

In order for a web client to access information by network or internet it uses TCP/ IP protocol is required. Moreover an IP address of the web server containing that information is needed. The web server can be accessed using the sub-protocol http as follows: **http:// IP Address.**

Communication between a web server and a web-client

In order to client communication with a web server, it must be known that “IP address of the web server as well as the sub protocol used.”

The default port number is used in the http addressing scheme is http:// IP address for e.g., if client has to connect itself to a web-server having IP address 255.255.0.1 then the current address will be **http:// 255.255.0.1**

Communication between a web server and a web-client

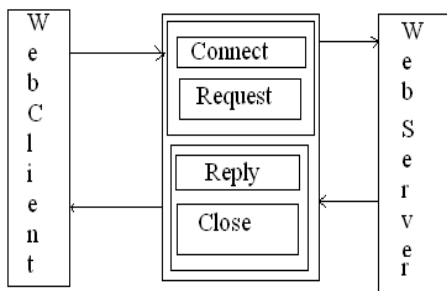


Fig: Four Step HTTP transaction

While information has to be accessed from a web-server; consists of four stages as illustrated in the following diagram:

- 1) **Connect:** For any kind of communication to occur between client and web server, the client first must established connection itself to that web server.
- 2) **Request/ Reply:** Once a client is connected to web-server it can sent its request for information. After receiving request for information, the web server located the needed information in its hard disk.

The requested information is then sent to the client, and the web server performs reset operation. Home page or default web document is sent at first and if the user clicks on any hyperlink, the connection is re-established.

- 3) **Close:** Once a user is done with accessing information it can stop the browser to turn off connection. Thus client is disconnected.

Introduction to HTML:

In the past, books, documents, or other kind of information were used to be kept in the static paper format. Once some changes were required, all those books and documents were to be updated. This way was very costly and very time consuming affairs. Moreover, a kind of information used to be static.

In reality, information be these be in the forms of books or any other kinds of paper documents, need to be revised again and again. In order to reduce the time and the cost involved in revising the information as and when required, one can think of keeping the information centrally located at the hard disk of the web server, in the form of one or several files. Such types of files can also travel across largest network of networks and it can be developed using HTML.

HTML was the first Internet-based language developed strictly for the web. Anything displayed in a browser is organized via HTML—it's central to the web development process and has evolved along with it.

HTML stands for **Hyper Text Markup Language**. The *backbone of the WWW is the HTML*. This is the language i.e. used to create web pages, and it is that glue that holds all of the pieces of the web together.

Hypertext Markup Language is a system of codes that identify parts and characteristics of documents. HTML documents are plain text (also known as ASCII) files that contain no images, no sounds, no videos, and no animations; however they can include “pointers” or link to these file types. Files that can be created using any text editor. You can also use word-processing software if you remember to save your document as “text only with line breaks”

Despite its power and flexibility, HTML is simple to understand and write. HTML is not a complicated, and it does not require special compilers or tools. We can write HTML just using the windows notepad.

HTML is static structure, organization, and content—think of those early websites that were simple text with some basic formatting. The way it works is through “tags,” which tell a browser how to display specific pieces of text. At its most simple, an HTML tag tells a browser to make the selected text bold, and the same goes for aligning text, creating headlines, adding hyperlinks, and more.

Once that basic HTML markup document is created, all other dynamic aspects of a site can be embedded into that file—e.g., a JavaScript program to add interactivity, server-side scripts that connect the site to the database, and CSS files that add stylistic elements. These files can all be linked out of the HTML file, making it much like the backbone of the site.

What is HTML?

HTML is the standard markup language for creating Web pages.

- HTML stands for Hyper Text Markup Language
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements are represented by tags
- HTML tags label pieces of content such as "heading", "paragraph", "table", and so on
- Browsers do not display the HTML tags, but use them to render the content of the page

HTML and the Web page

HTML is not the only way to represent the information on the web, but its glue that holds everything together. In addition, to being a mark up language for displaying text, images and multimedia, HTML provides instructions to web browsers in order to control how documents are viewed and how they relate to each other. For all its simplicity, HTML is very powerful language.

Without HTML, the WWW does not exist. HTML allows individual elements on the web to be brought together and presented as a collection of text, images and multimedia and other files can be packaged together using HTML.

Features of HTML

- Provide tag to make the document look attractive.
- Use graphics and display text in different font size and colour
- Create hyperlink, to other document or portion of same document
- Display data in tabular form
- create multiple windows in a web page, etc.

Benefits of HTML

There are many more benefits but major benefits are:

- ❖ Html is simple but powerful formatting language to use. The simplicity of html allows anything to credited web page
- ❖ Html document are device implemented. It works on any platform (operating system)
- ❖ The web page can be linked together using link hence controlled navigation is possible most important of all is the page can be updated any things and the appropriate link can be used without bothering about weather the linked will get effected any not.

Limitations of HTML:

HTML doesn't have any programming capabilities and can't provide any thing more the formatted text picture and sound which demands for other programming scripting language to be used along with it such as javaScript, vbscript and other programming language.

XHTML (eXtensible Hypertext Markup Language)

XHTML is essentially identical to HTML4 (the fourth iteration of HTML), but with elements of XML that extend HTML's capabilities. XHTML came along before HTML5, right after HTML4, solving for some cross-browser compatibility issues and offering a “best of both worlds” scenario. It's a bit stricter than HTML, providing more precise standards and specifications for how a site's data is broken down and transmitted. This does make it a bit unforgiving, however, and it can be tougher to debug.

XHTML is written in the same format as an XML application, another descriptive markup language that functions like a complement to HTML, specifically handling how data is organized. (In markup, XML describes elements of data, while HTML displays that data.) Much of what XHTML was designed to do has been covered with the launch of HTML5, however, making it nearly obsolete. Some features are:

- XHTML stands for EXtensible HyperText Markup Language
- XHTML is almost identical to HTML
- XHTML is stricter than HTML
- XHTML is HTML defined as an XML application
- XHTML is supported by all major browsers

HTML5: The Latest and Greatest Version of HTML

HTML's latest evolution, HTML5, is a versatile game-changer. HTML5 adds new features to the HTML repertoire, making it able to do so much more on its own. It's a hybrid of three types of code—HTML, CSS, and JavaScript—and by bundling these three components, HTML5 lets developers work faster and more efficiently. The resulting site architecture is agile, mobile-friendly, and compatible with more browsers.

So, how else is HTML5 different from HTML? First, it touts one major advantage over its predecessor: cross-platform application development. It also solves problems that XHTML formerly addressed, and boasts better compatibility across more browsers.

HTML5 has incorporated many new APIs and features like drawing, video playback, and drag-and-drop—effects that developers could only implement before with the help of third-party plug-ins. The most interesting new HTML5 elements are:

- New **semantic elements** like <header>, <footer>, <article>, and <section>.

- New **attributes of form elements** like number, date, time, calendar, and range.
- New **graphic elements**: `<svg>` and `<canvas>`.
- New **multimedia elements**: `<audio>` and `<video>`.

The most interesting new APIs in HTML5 are:

• HTML Geolocation	• HTML Drag and Drop	• HTML Local Storage
• HTML Application Cache	• HTML Web Workers	• HTML SSE

Components of HTML

TAG: HTML consist a set of predefined tag that are typed along with text. These tags tell the browser whether the text is to be paragraph, a heading, an image, and address etc. An HTML tag is a signal to browser that it should something other than just throw text upon the screen. All the HTML tag begins with an open angle brackets (`<`) and end with a closed angle brackets (`>`).

It provides a way to control text formatting, create lines to other documents and even incorporate images and sound. In short, tags are the keys to make HTML page work.

Tags are not case sensitive. For e.g. the body element tag can be typed as `<BODY>` or `<body>` or `<BoDy>`.

HTML `<meta>` Tag

- Metadata is data (information) about data.
- The `<meta>` tag provides metadata about the HTML document. Metadata will not be displayed on the page, but will be machine parsable.
- Meta elements are typically used to specify page description, keywords, author of the document, last modified, and other metadata.
- The metadata can be used by browsers (how to display content or reload page), search engines (keywords), or other web services.
- HTML5 introduced a method to let web designers take control over the viewport (the user's visible area of a web page), through the `<meta>` tag (See "Setting The Viewport" example below).

Note: `<meta>` tags always go inside the `<head>` element.

Note: Metadata is always passed as name/value pairs.

Note: The content attribute **MUST** be defined if the name or the http-equiv attribute is defined. If none of these are defined, the content attribute **CANNOT** be defined.

Setting The Viewport

- HTML5 introduced a method to let web designers take control over the viewport, through the `<meta>` tag.
- The viewport is the user's visible area of a web page. It varies with the device, and will be smaller on a mobile phone than on a computer screen.
- You should include the following `<meta>` viewport element in all your web pages:
`<meta name="viewport" content="width=device-width, initial-scale=1.0">`
 - A `<meta>` viewport element gives the browser instructions on how to control the page's dimensions and scaling.
 - The `width=device-width` part sets the width of the page to follow the screen-width of the device (which will vary depending on the device).

- The initial-scale=1.0 part sets the initial zoom level when the page is first loaded by the browser.
- Here is an example of a web page *without* the viewport meta tag, and the same web page *with* the viewport meta tag:

Tip: If you are browsing this page with a phone or a tablet, you can click on the two links can see the difference. Also can read more about the viewport in our [Responsive Web Design](#)

TYPES OF TAG: 1) Singular tag 2) Paired tag

#Singular Tag: It does not require companion tag. For e.g.
, <P> etc.

#Paired Tag: It requires a companion tag to format the textual information. A tag is enclosed with in an opening bracket (<) and a closing angle bracket (>). For e.g. , <I> etc. Tag is a companion tag because it requires a companion tag

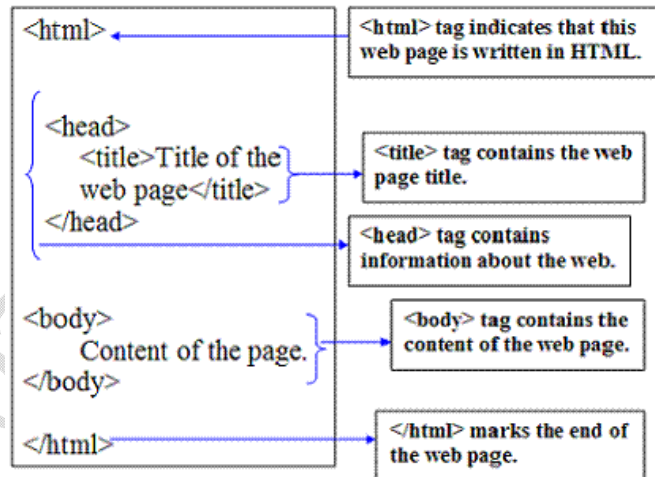
Basic structure of an HTML document

An HTML document has two* main parts:

head. The head element contains title and meta data of a web document.

body. The body element contains the information that you want to display on a web page.

In a web page, the first tag (specifically, <html>) indicates the markup language that is being used for the document. The <head> tag contains information about the web page. Lastly, the content appears in the <body> tag. The following illustration provides a summary.



NOTE:-

- Open notepad in your computer.
- Write the html document using appropriate tag
- Save it as filename.html e.g. web1.html(htm)
- The designed webpage can be displayed by clicking internet explorer and writing the address in address bar.

Some tags have additional information that can be attached after the tag followed by a space. This additional information is called attributes.

Attributes of the <body> tag are:- Most common attributes of body tags are as follows:

Attributes	Description of the attributes
Bgcolor	Changes the background from the default to the color mentioned with the attribute. E.g. <body bgcolor="red">
Background	Changes the background of the webpage specifying the name of gif file. This gif files comes up across the page to give a background.e.g.<body background="puset logo.gif">
Text	Changes the color of the text of web page<body bgcolor="red" text="yellow">
Link	Color of the hyperlink before clicking, you can give color or color code
Alink	Color of the hyperlink after clicked, you can give color or color code
top/bottom/left	Sets top margin of the page, in pixel

/right margin

Adding footers

Some information typically concerning web developer and copy right, notes appear at the bottom of the page. This information is called footer.

Information to be made footer, is kept between **<address>** and **</address>** tags. Although this tags can appear anywhere in the body section, the preferred place is just before **</body>** tag.

Character Formatting:

Tags	Examples	Description
	bold text	Makes text bold
<i>	<i>italic text</i>	Makes text italic
<u>	<u>underline text</u>	Makes text <u>underline</u>
<strike>	<strike>text</strike>	Makes text strikethrough
<sub>	H<sub>o</sub>	Makes text subscript e.g. H ₂ O
<sup>	X<sup>2</sup>	Makes text superscript e.g. X ²
		Makes text size=1 to 7, color=color name or code, face= font name
<p>	<p align="left/right/center">	Different paragraph alignment

Note: **
** tag is used for line break and it has no ending tag

Headings : There are 6 heading tag in HTML starting from **<h1>** to **<h6>**. It represent size of heading

<h1>Biratnagar**</h1>**

<h2>Biratnagar**</h2>**

<h3>Biratnagar**</h3>**

<h4>Biratnagar**</h4>**

<h5>Biratnagar**</h5>**

<h6>Biratnagar**</h6>**

Drawing line or horizontal tag

<hr> tag is used to draw a line across a web page. It produces shiny line across the page, it doesn't have end tag.

Attributes of **<hr>** tag

align= left, center, right	width=50% (N in pixel)	size=4 (N in pixel)	color=red
----------------------------	------------------------	---------------------	-----------

e.g. **<hr align="center" width=50% size=4 color="Black">**

Font Tag

A text can be made appear in different fonts size and color by enclosing them with in ****-----**** tags.

Attributes

face= name_of_font

Size=8 or any number

color= name_of_color or color code e.g. red,black

eg **puset **

Center:

<body>

<center>

.....

</center>

</body>

<center>.....</center> tags aligns everything enclosed within them to the center of the webpage.

Preformatting (pre) Tags

It gives the exact output according to the format that we have given inside the tag.

e.g.

<html>

<head><title>Using Simple Program tags </title></head>

<body>

<center> Welcome to PUNSET</center>

School of engineering & technology

The head office of punset is located at<p> Biratnagar.....

The major course taught here are:<hr>

<pre>

BCA

BIT

BE

MCA

</pre>

<blockquote>Center is located at biratnagar </blockquote>

</body></html>

HTML <section> Tag

The <section> element is a structural HTML element used to group together related elements. Each <section> typically includes one or more heading elements and additional elements presenting related content.

Section tag defines the section of documents such as chapters, headers, footers or any other sections. The section tag divides the content into section and subsections. The section tag is used when requirements of two headers or footers or any other section of documents needed. Section tag groups the generic block of related contents. The main advantage of the section tag is, it is a semantic element, which describes its meaning to both browser and developer.

Display

block

Usage

semantic | structural

Syntax: <section> Section Contents </section>

Section tag is used to distribute the content i.e., it distributes the sections and subsections.

Example:

<section>

<h1>Introduction</h1>

<p>People have been catching fish for food since before recorded history...</p>

</section>

<section>

<h1>Equipment</h1>

<p>The first thing you'll need is a fishing rod or pole that you find comfortable and is strong enough for the kind of fish you're expecting to land...</p>

</section>

Text Level Semantics

Text is an important part of any document and is the fundamental building block for most web sites. The following list gives you a short overview of the most commonly used text-level semantics and how to utilize them. Example

Element	Description
<a>	Turn text into hypertext using the <u>a element</u>
	Emphasize text using the <i>em element</i> .
	Imply any extra importance using the strong element
<code>	Define inline code snippets using the <code>code element</code> .
<mark>	Highlight text with no semantic meaning using the <code>mark element</code> .
<abbr>	Define an abbreviation using the <code>abbr element</code> with a title.
<small>	De-emphasize text for small print using the <code>small element</code> .
<sup>	Mark up typographical conventions with specific meanings, US ²
<sub>	Mark up typographical conventions with specific meanings, US ₂ .

The **HTML <embed>**:

The `<embed>` tag is new in HTML5, and will validate in an HTML5 page. **The element embeds external content at the specified point in the document. This content is provided by an external application or other source of interactive content such as a browser plug-in.**

```
<embed type="video/webm"
src="/media/examples/flower.mp4"
width="250"
height="200">
```

Attributes:

Attribute	Value	Description
<u>height</u>	Pixels	Specifies the height of the embedded content
<u>src</u>	URL	Specifies the address of the external file to embed
<u>type</u>	media_type	Specifies the media type of the embedded content
<u>width</u>	Pixels	Specifies the width of the embedded content

Interactive Element: HTML offers a selection of elements which help to create interactive user interface objects.

Element	Description
<details>	The HTML Details Element (<details>) creates a disclosure widget in which information is visible only when the widget is toggled into an "open" state.
<dialog>	The HTML <dialog> element represents a dialog box or other interactive

	component, such as an inspector or window.
<menu>	The HTML <menu> element represents a group of commands that a user can perform or activate. This includes both list menus, which might appear across the top of a screen, as well as context menus, such as those that might appear underneath a button after it has been clicked.
<menuitem>	The HTML <menuitem> element represents a command that a user is able to invoke through a popup menu. This includes context menus, as well as menus that might be attached to a menu button.
<summary>	The HTML Disclosure Summary element (<summary>) element specifies a summary, caption, or legend for a <details> element's disclosure box.

Some more and summary of tags

<center>....</center>	<big>.....</big>	<strike></strike>	<blockquote>
.....	_{.....}	<u><I>	 or
.....	^{.....}	<p> <hr><pre>	<i><cite>

Marquee: It helps to animate text or picture in different directions with different behaviors

<marquee>

attributes :

behaviour=scroll/slide/alternate,
direction=up/down/left/right,
scrollamount=value(10)
width=N (in pixels or percentage)
loop=N times or "infinite"
bgcolor=color or colorcode

Any one

HTML Grouping Tags

Grouping is where you put a wrapper around multiple elements.

HTML Grouping Tags

Tag	Description
<div>	Defines a section in a document (block-level)
	Defines a section in a document (inline)

Other examples are:

Grouping content (element)

P	address	hr	pre	blockquote
Ol	ul	li	dl	dt
dd	figure	figcaption	main	div

Lists

List is used to organize the information in a clear and easily understood format. There are three main type of list.

→ unordered list or bulleted list (using)

→ Ordered list or numbered list <using)

→ Definition list (using <DL>)

Ordered list

Any list prepared by numbering format or holding a sequence is an ordered list.

→ Ordered list is beings with tag. The list of item is created after by using tag.

E.g.

<ol type=1(i,I,a) start=1(2/3/..)>

.....

.....

Unordered list

Unordered list presents info. In bulleted format with, tag, tag is used for using the terms.

E.g

<ul type=disc(fillround/square/circle/disc)>

.....

.....

Definition list

Glossary list or definition list, in which each item in the list has a term and definition for that the term is somehow highlighted.

<dl>.....</dl> tag is used for giving definition list tag is accomplished by other two tags: namely <dt> and <dd>. <dt> tag is used as definition term and <dd> tag for definition description.

E.g.

<dl>

<dt>Keyboard

<dd>Input device.

<dt>Printer

<dd>Output device

</dl>

Comment

Comment can be used HTML page between “<!--” and(here Commented text) “-->”. Text in comment is ignored when the HTML file is passed. Comment don’t show up on screen.

<!-- - This is comment -->

Special Character.Special characters are the symbol or character entities used on a web page. Such characters are

Character	Code	Description
One space	 	One space
<	<	Less than
>	>	Greater than
£	£	Pound symbol
¥	¥	Japanese yen symbol
©	©	Copyright symbol
®	®	Registered trade mark
™	™	Trade mark

Write HTML code for followig display

```

1. PU
  a) PUSSET
    • Bachelor
      1) Engineering
        o Computer
        o Elx & Commu.
        o Civil
        o Electrical
      2) IT
        a) BCA
        b) BIT
    • Master
      a) PUSOM
      b) PUCMAS
      c) GPCAR
      d) CPAD
2. TU
3. KU
  
```

±	±	Plus minus symbol
μ	µ	Greek micro sign
°	°	Degree sign
A	A	A letter
B	B	B letter

Eg:<HTML> <head><title>using symbol</title></head>

<body>Ampersand: **&**<p>

copyright: **©**<p>

left-angle bracket: **<**<p>

right angle bracket: **>**<p>

double quotes: **"**

</body> </HTML>

Insert Image

HTML allows us to insert static and animated image to be inserted into a HTML documents. So that it can be viewed in web page. An image to be inserted in HTML document is described within ** tag in terms of its source and attributes** and has no ending tag.

Mainly, GIF and JPEG files are used in the web. GIF is most of the times used as it supports animation and animator and adobe photoshop, special effects can be inserted on the image to suit the requirements of a web page.

To use any image, it necessary that it should be placed in the same root folder where the web page, in which it is inserted is placed. Otherwise, the path must be specified in the tag, in case it is placed outside the root folder.

Eg:

<HTML><head><title>using image tag</title></head></body>

<body>

<center></center>

</body></html>

Attributes of Image

Attributes	Description
Src="URL/picture file name with path"	URL or path of image source file
Height=N	Height of image in pixel
Width=N	Width of image in pixel
Border=N	Image border in pixel
Vsapce=N	Vertical space around the image in pixel
Hspace=N	Horizontal space around the image in pixel
Align=top/bottom/middle/left/righ	Image alignment
Title	Comments on the picture area while mouse over
Alt	Display alternate text while error in path of the picture file
<center>	To place image or text at center

E.g

<html>

<Head><title>Working with an image </title></head>

<body bgcolor="red" text="white">

<h1> webpage design and web programming</h1>


```
<li>
<li>.....
</ul></body></html>
```

Alt → To display some information, if the browser can not display the image.

E.g.

Insert Object: Audio, video, and animated files can be inserted in the web page as an object.

Tere are two methods to insert objects. First method is using <EMBED> tag and Second method is using <OBJECT> tag. The object tag is standard for all type object even image.

Tags	Syntex of the tag	Attributes /description of tag
<EMBED>	<EMBED src="audio or video file">	1) src: defines the path of the source file 2) autostart="true/false" 3) hidden="true/false", loop="true/false"
<NOEMBED>	<noembed> you can type error for the browser that does not support embed<noembed>	This is sub tag and it is optional
<OBJECT>	<object data="filename" type=audio/midi>	Data defines the path of the source file type defines the type of the objects.
<audio>	<audio src="filename.mp3" controls>	Display audio windows and sound play
<video>	<video src="filename.mp4" controls>	Display video windows and sound play
<bgsound>	<bgsound src="audioname.mp3"loop="1">	Sound will play but not displaywindows

Linking (Hyperlink)

The single greatest feature of the WWW is its diverse collection of documents, which number in the millions. All of those documents are bought together through the use of hypertext links. Hypertext links are crucial part of HTML, which, after all, is short for HTML.

Hypertext connects two different documents. User can link to one of his own documents or to any other document on the World Wide Web (also called inter page linking). We can even link to a different section in the same document (called intra page linking). The images on which hyperlinks are created get highlighted and underlined. When the mouse cursor is bought over a hypertext or hyper image, it assumes shape of hand. The default color for hyperlink is blue.

We can also link to a different section in the same document.

Hyperlink is created by using anchor tag <A> and .

Syntax: Type text to know something about link

Where <a> means anchor tag and href means hyper reference

An image can also be used to create hyperlink.

Syntax: some text or can use image to follow link after mouse click

Example: routine or image

Intrapage linking or Bookmarks(Internal Document References):

About jumping to a particular location is a newly loaded Web document. For this, we have to use name attribute in the beginning anchor tag.

 top position

// To Many lines of Page

Go to top Position

You can create multiple links to the web pages from a single picture with the help of image map <MAP>. It has following attributes and sub tags of image map.

Tags	Syntax of tag	Attributes of tag
<map>	<map name=“name of map”>	Name=name of tag
<area>	<area shape=circle/rect/polygon coords=“coordinanate of the shape”>	Shape=circle/rect/polygone Cords=coordinate of the shape. It is a sub tag of map<MAP>

<HTML><HEAD><TITLE>Image Map</TITLE></HEAD>

<BODY bgcolor=“silver”>

<h1>mouse over on the picture</h1>

<map name=“map1”>

<area shape=“polygon” coords=“8,15,21,50,37,26,21,12,7” href=“fn.html”>

<area shape=“rect” coords=“10,20,20,40” href=“fn1.jpg”>

<area shape=“circle” coords=“40,80,40” href=“fn2.doc”>

</body></html>

Table

The table is very useful tool to arrenge information in a web page. It is a two-dimensional array and is matrix consisting of rows and columns. They are used to make the image and text appear at exactly the location on a page that user want to appear. The <table> is used for defining a table. A table has main three sub tags: table row<TR>, table header<TH>, and table data<TD>, moreover you can also include caption in a table by using tag <CAPTION>

A table Table are specified in HTML rows by rows, and each row definition contains definitions for each of the cells in that row. Table cell can contain normal table data or a table heading.

Attribute	Description of the attribute
border =N(in pixel)	Size of table border
bordercolor=“colorcode” or “colorname”	Border color of a table
cellspacing=N (in pixel)	Define distance between cell’s border in a table
cellpadding=N (in pixel)	Define distance between cell’s content and cells border
width=N	Defines table width and row width in pixel or N%
height=N	Height of table or a row in pixel
colspan=N(integer number)	Number of cells to be merged column wise in a row
rowspan=N(integer name)	Number of cells to be merged row wise in a column
bgcolor=“colorname” or “colorcode”	Table or row background color
align=“left/right/center”	Defines alignment in a table or a table

- <table>..... </table>, tags are used to display, table. Inside <table> tag, a number of other tags are used which make up the contents of the table.
- Every row of table is put inside <TR>..... </TR>
- A heading level is put inside <TH> </TH>
- A data is put inside <TD> </TD>
- Caption: Caption is put at the top or bottom of the table which gives some information about the table.

Syntax: <Caption valign = Bottom/up> </Caption>

Example showing table with one rows and three columns.	Example showing table with three rows and one columns.
--	--

<pre><Table border=1 width=50 bgcolor="#ffff00" align="center"> <caption>table showing HTML code</caption> <TR> <TD width=30%> S.N </TD> <TD bgcolor="red"> Name </td> <TD> Address </td> </TR></table></pre>	<pre><Table border=1 width=50 bgcolor="#ffff00" align="center"> <caption>table showing HTML code</caption> <TR> <TD width=30%> S.N </TD></TR> <TR bgcolor="red"><TD > Name </td></TR> <TR><TD> Address </td></TR> </table></pre>
<i>Example showing table with two rows and three columns.</i> <pre><Table border> <TR> <TH> S.N </TH> <TH> Name </th> <TH> Address </th> </TR> <TR> <TD> 1 </td> <TD> Data 1 </td> <TD> Data 2 </td> </TR> </Table></pre>	<i>Example showing table with three rows and three columns.</i> <pre><html> <head> <title>Creating a table in HTML </title> </head> <Body> <H3> creating a simple table </H3> <table border=2 align= center> <TR> <TH> S.N <TH> Name <TH> Address </TR> <TR bgcolor="red"> <Td align = center > 1 <Td> name<Td> address </TR> <TR bgcolor = "Red"> <td> 2 <td> <td> Your friend address Nepal India< Li> china </TR></Table> </body> </html></pre>

Spanning rows and columns

HTML CODE

		classes		Total number
		Class	Class	
Gender	Boys	12	15	25
	Girls	10	20	30
Total no. of students		20	35	55

```
<html>
<head> <title> rows & cols span
</title></head>
<body> <center>
<table border = 1 cellspacing = 2%
cellpadding = 2% width = 50%>
<caption valign = bottom /up> student record
</caption>
<TR>
<TH rowspan= 2 colspan =2>
<TH colspan =2> classes
<TH rowspan =2> total number
</TR>
```

```
<TR>
<TH> classes 1
<TH> classes 2
</TR>
<TR>
<TH rowspan= 2> Gender
<td> Boys <td> 10 <td> 15 <td> 25
</TR>
<TR>
<td> Girls <td> 10 <td> 20 <td> 30
</TR>
<TR>
<TH colspan = 2> Total number of student
<td> 20 <td> 35 <td> 55
</TR>
</table> </body> </html>
```

A test table with merged cells

	Average		Red eyes
	height	weight	
Males	1.9	0.003	40%
Females	1.7	0.002	43%

Nested Table

Prepa

Nested Table	Nested Table	Nested Table																		
<table><tr><td>OS</td><td>PACKA</td><td>PROGRA</td></tr><tr><td>DOS</td><td>WMN</td><td>LINUX</td></tr><tr><td>OFFICE</td><td>ACCES</td><td>VB</td></tr></table>	OS	PACKA	PROGRA	DOS	WMN	LINUX	OFFICE	ACCES	VB	<table><tr><td>WEB PRG</td><td>CP</td></tr><tr><td>MP</td><td>CS</td></tr><tr><td>ORACLE</td><td>JAVA</td></tr></table>	WEB PRG	CP	MP	CS	ORACLE	JAVA	<table><tr><td>C++</td></tr><tr><td>VB.NET</td></tr><tr><td>MULTIMEDIA</td></tr></table>	C++	VB.NET	MULTIMEDIA
OS	PACKA	PROGRA																		
DOS	WMN	LINUX																		
OFFICE	ACCES	VB																		
WEB PRG	CP																			
MP	CS																			
ORACLE	JAVA																			
C++																				
VB.NET																				
MULTIMEDIA																				

Tables used within table is called nested table. Nested table is a great tool for displaying and arranging information. To nest a table the cell of the outer table contains the entire inside the table. The <TABLE></TABLE> tag of the inside table is placed in the <TD></TD> tags of the outside table.

Computer Courses

```
<HTML>
  <head>
<title> Nested table </title>
</head>
  <Body>
    <table border = 1 cellpadding =10% cellspacing =10% width =10%>
      <caption> <H2> computer
courses </H2> </caption>
      <tr>
        <Th> Semester I
        <Th> Semester II
        <Th> Semester III
      </tr>
      <TR>
        1. <TD>
          <table border =2 width = 100%>
            <caption> Nested table </caption>
            <TR>
              <TH> OS
              <TH> PACKAGE
              <TH> PROGRAM
            </TR>
            <TR>
              <TD> DOS
              <TD> WIN
              <TD> LINUX
            </TR>
            <TR>
              <TD> OFFICE
              <TD> ACCESS
              <TD> VB
            </TR>
          </TABLE></TD>
        2 <td>
          <Table border = 2 width =100%>
            <caption> Nested table </caption>
            <TR>
              <Th> Web page
              <Th> CP
            </TR>
            <TR>
              <TD> MP
              <TD> CS
            </TR>
            <TR>
              <TD> ORACLE
              <TD> JAVA
            </TR>
          </Table>
        </td>
        3 <TD>
```

```

<table border = 2 width = 100%>
  <caption> Nested table </caption>
  <TR>
    <Th>    C++    </Th>
  </TR>
  <TR>
    <TD> VB.NET
  </TR>
  <TR>
    <TD> MULTIMEDIA
  </TR>
</table></Td>
</TR>
</Table> </body> </html>

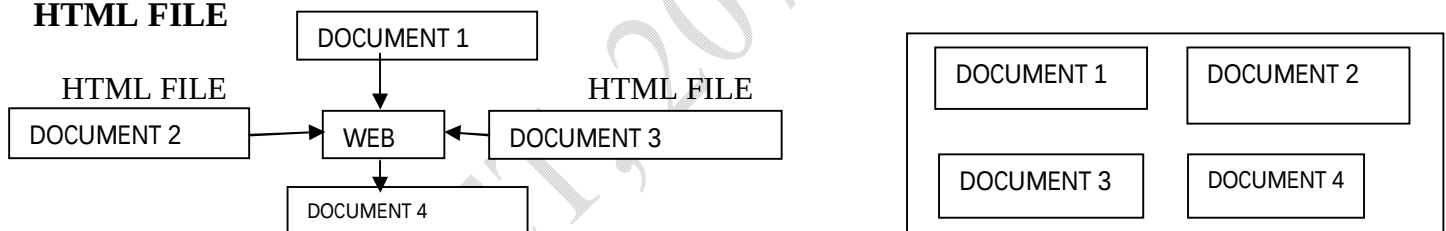
```

FRAMES

Till now, each web page when opened takes over the entire screen. The browser screen could not be split into separate section showing different but related information.

Frames provides the facilities to divide a window into many sections. In each section any file can be added to web page, hence multiple pages can be displayed in one windows. It is possible to load two or more than two html documents on the browser screen so that these documents can be viewed concurrently. Those regions where different html documents are loaded are called frames.

HTML FILE



It is possible to divide an entire browser screen into several unique and so independent sections called frame. When we use `<frameset>` tag, we don't need to use `<body>` tag. `<body>` contains only one page but `<frame>` contains multiple pages, which divides screen into many parts.

Frames are created using `<frameset> ..... </frameset>` tag. The `<frameset>` tag requires one of the following two attributes depending on whether the screen has to be divided into rows and columns(cols). The `<frame>` tag helps to insert the html file with the help of `src` attribute

Rows: It split the screen into horizontal frame. For e.g. to split the screen into equal horizontal height frames, we can write either of the following syntax:

`<frameset rows = "rows width,.....">`

e.g: `<frameset rows = "50%, 50%">` or `<frameset rows = "*, *">` or `<frameset rows = "20%,50%,*">`

Rows are can be used for rows in pixels.

"%" Express as a percentage of the screen resolution.

"*" Indicates the equal and remaining spaces.

Columns: It split the screen into vertical frame. For e.g. to split the screen into equal vertical height frames, we can write either of the following syntax:

E.g : `<frameset cols = "50%, 50%">` or `< frameset cols= " *, *">` or `<frameset cols = "20%,50%,*">`

Apart from rows and cols, FRAMEBOARDER attributes is used for giving borders to the frame, specified in pixels.

The <FRAME> tag

Once the basic frameset is laidout, one need to associate an HTML document with each frame. To do this, we use the <frame> tag, which takes the following form:

<FRAME SRC = “ Document. URL>

for each frame defined in the following <frameset> tag, we must include a corresponding <frame> tag as shown below. Program below divides a window horizontally(row) into 3 section

```
<frameset rows = “*,*,*”>
<frame SRC = “document1.html”>
<frame SRC = “ document2.doc”>
<frame SRC = “ docment3.jpg”>
</frameset>
```

We can also use frame name within frameset like.

```
<frameset rows = “*,*”>
<frameset name = “abc”>
<frameset name = “xyz”>
</frameset>
```

Below eg. Shows nested frameset which divides a windows horizontally (row wise) as well as vertically (column wise)

```
<frameset rows=”30%,*”>
<frame src=eg1.html>
<frameset cols=”50%,*”>
<frame name=”na1” src=”eg2.jpg”>
<frame src=”eg3.doc”>
</frameset>
```

Summary of tag

<u>Tags</u>	<u>Attributes</u>	<u>Detail</u>
<frameset>	Rows Cols <frameborder>	- Creates the specified no. of frames horizontally in pixel or percentage. - Creates the specified no. of frames vertically in pixels and percentage. - Border the frame.
Frame	Name SRC Marginal height Frame width Noresize Scrolling	- Creates structure of frame with name - Indicates the document of URL - Adjust the margin of the frame from the top and bottom of the frame specified in pixels. - Adjust the margin of the frame from the left and right of the frame specified in pixels. - Disallows the client to change the size of frame. - Displays scrolling of specified with “Yes” option. Disable the use of scroll bar if “No” option is used.

Forms

Forms appear like a manual form, but it is displayed electronically. They provide a mechanism to allow user to enter several information in an electronic manner. Once several information are filled in they are validated (i.e. they are checked individually against some predefine criteria or condition.

A form is a interface in web pages for entering data. A form has many elements depends on many input data types. These elements have different attributes. The most common attributes are as follows: The <FORM> tag defines the form option. The <INPUT> tag defines the form field type.

Attribute	Description of attribute
Action	Takes URL that defines where to send the form data when the submit button is clicked
Method	Defines the HTTP method: get or post
Type	Type of form element or form field
Name	Unique name for the field value
Value	Default value of the field
Checked	Defines boolean values: true/false
size=N (Integer Number)	Size of a form field in terms of number of characters
maxlength =N (Integer Number)	Maximum length of character in a field
Randomly	Defines boolean values: true/false
rows=N	No. of character in rows for textarea field
cols=N	No. of character in columns for textarea field

A webpage can contain one or more than one form. A form is enclosed within

<Form > Form object go in here </Form>

The form contains the following object:-

Form Elements	Description
Text	A text or number can be entered in text box<input type="text">
Checkbox	Select multiple choice option <input type="checkbox">
Radio	Selecting a single option from multiple choices<input type="radio">
Password	Allows to enter password. Same as text box but it displays asteristics(*) while typing<input type="Password">
File	A file helps to browse a filefrom the computer and attach the file to the from <input type="file">
Button	Helps to perform any action inside form. Mainly there are two attributes: value and onclick <input type="Button">
Reset	It clears the content of all fields of the form <input type="reset">
Submit	It provides user to submit the entire form contents <input type="submit">
Text area	It allows user to type multiple lines of text in a form. (<textarea rows=3 cols=20>default text</textarea>
Hidden	Allows user to enter data in hidden form <input type="hidden">
Select	It provides user to select option from the given list of items. The sub tag <OPTION> defines the value itemof the list.(<select> <option> Option1 </option> <option> Option2</option></select>

Following example gives an idea about form element

```
<html><head><title>form element</title></head>
<body bgcolor="#802000">
<form name="f1">
Name:      <input type="text" name="name1" size="18"><br>
Address:   <input type="text" name="ad1" size="20" maxlength="30"><br>
Country :  <select size=1>
              <option>Nepal</option>
              <option>India</option>
```

```

        <option>Chinal</option>
        <option>tan</option>
    </select><br>
Gender:      <input type="radio" name="r1" checked="true" value="male">Male
              <input type="radio" name="r1" value="female">Female<br>
Qualification: <input type="checkbox" checked="true" name="c1" >SLC
              <input type="checkbox" name="c1" checked="true">Intermediate<br>
              <input type="checkbox" name="c1" >Bachelor<br>
Password:    <input type="password" size="10" maxlength=5><br>
File Browse: <input type="file" size="20" maxlength=40><br>
Command :    <input type="button" value="OK"><br>
Reset:       <input type="reset" value="clear"><br>
Submit:      <input type="submit" value="send"><br>
Image        <input type="image" src="pp.jpg" alt="send" height=25 width=50 border=2>
              <!--just like submit
</form>

```

DHTML (Dynamic Hypertext Markup Language)

Now a days people like to develop web pages which has better outlook and dynamically. DHTML combines HTML (that gives static web pages using features like formatted text, images, bullet, table, frame and so on) with cascaded style sheets (CSS) and scripting language.

Cascaded Style Sheet (CSS)

CSS helps a programmer to specify type of font, color, background and so on that can be applied to different tags for controlling the outlook of web pages and other different interactive effect. Such effects are called dyanamic effect or also known as dyanamic HTML. These are two methods for embedding such effects. First method is using CSS and the second method is using scripts.

HTML originally had tags like , <table>, <center>, and so forth that styled each element in each page. The problem with this is that when the developer wanted to change the style of some content, every page had to be edited throughout the website. CSS eliminates the need to edit all your pages by having the code for the styles stored in a separate file that can be edited once.

Every modern website uses CSS to change the look and feel of the website. CSS is a standard that is required for modern browsers to function correctly. CSS is a w3c standard and necessary to understand website coding.

What is CSS?

- CSS stands for Cascading Style Sheets
- CSS defines **how HTML elements are to be displayed**
- The CSS code can be stored in a separate file called a style sheet.
- CSS is separate code that styles elements in your website like paragraph or header HTML tags.
- This allows user to create styles for elements throughout the website, without having to code the styles into each page.
- Styles were added to HTML 4.0 **to solve a problem**
- CSS saves a lot of work

- External Style Sheets are stored in **CSS files**

Comparison between CSS and ss3

CSS: CSS was developed by W3C World Wide Web Consortium to solve the problem with individual line styling in HTML. When individual styling attributes were introduced in HTML 3.2 to help style HTML, there still was a problem. Developers had to use style attributes at every single tag, so to overcome this CSS played an important role.

CSS3: CSS3 is the latest version of CSS and comes with new features like rounded corner shadows, gradients, transition, and animations. (That's right, it has even properties like JavaScript.) Mobile Development is the main concern of CSS3. It focuses on responsive design that reduces the problem caused by Adobe-Flash incompatibility on the different devices.

Features	Css	Css3
Type	It is a standard language that is used to design the web.	It is the newest version of CSS.
It is the newest version of CSS.	CSS does not follow the responsive Designing CSS designed webpages are not that much mobile phone friendly	The web page of the website adjusts the page size and components according to the size of the devices. CSS is more mobile friendly
Modular based code	CSS does not follow module-based code; it uses its all specifications directly.	CSS3 uses all the old specifications of CSS, but all the specifications are divided into different modules.
Rounded Borders	CSS did not have this feature	CSS3 has this feature
Bordered Images, text shadows	CSS did not have these features	CSS3 comes with these features
Multiple Background	CSS has to use the third party for the Multiple Background	CSS3 comes with the built-in Multiple background feature.
Fast page load	The loading speed of CSS designed web pages was decent	With the introduction of CSS3 in web pages, it helps to load pages very fast as compared to CSS.
Animations	CSS use JavaScript or Flash code for the animations.	CSS3 comes with built-in features like 2D and 3D transformation, which provides animation and transition effects.
Colors and Images effects	CSS has many colors and images effect features.	CSS3 contains all the CSS effect and also have some new colors and images effects.

In conclusion, CSS is an effect which can be embedded inside HTML file to make easy and standard formatting and style of a web page. mostly such effect are embedded inside <HEAD> tags. CSS is case sensitive. The general syntax for css is as follows:

Element {property1:value 1; property2:value 2; property3:value 3;.....}

Where element is a tag of HTML, properties are the attributes for the the tag, some common styles and their properties are as follows:

Controlling font attributes: for changing font attribute:

Attributes	Description
Font-family	To specify family of fonts. eg:- Arial, serif,etc
Font-style	Eg:- normal, italic or oblique
Font-weight	Normal, bold, bolder, lighter, or one the nine numerical values (100,200,.....900)
Font-size	To specify size of font eg:-font size :12 pts

Color and background attributes:

Attributes	Value
Color	Sets an elements text-color. A color name or color code
Background-color	Specify the color in an element's background. A color name or color code can be specify
Background-image	Sets background image. A URL of the image has to be specify

Text attributes:

Attitributes	Description
Text-decoration	Adds decoration to an element's text: none, underline, overline, line-through,blink
Text-align	Align text within an element left, right, center, or justify
Text-indent	Indents the first line of text length. A percentage of the element's width, or a length
Text-transform	Applies the transformation to the text capitalize(puts the text into initial caps), uppercase , lowercase or none.

Border attributes:

Attitributes	Description
Border-style	Solid, double, groove, ridge, inset, outset,none,dotted,dashed,
Border-width	Thick, thin, medium
Border-top-width	Thick, thin, medium
Border-left- width	Thick, thin, medium
Border-button- width	Thick, thin, medium
Border-right- width	Thick, thin, medium
Border-color	Specify the color of border

Margin attributes

Attribute	Description
Margin-left	Specifies the left margin in length (or % of the net screen)
Margin-right	Specify the right margin in length (or % of the net screen)
Margin-top	Specifies the top margin in length (or % of the net screen)
Margin-button	Specifies the left margin in length (or % of the net screen)

There are three types of stylesheets

There are three types of stylesheets:

- **Inline** - Placed inside a tag it will affect.
- **Internal** - Placed right on the page whose interface it will affect. It include <class> and <div>

- **External** - Placed in a separate file.

Creating an inline stylesheet

It is very simple method of style web page. Use inline stylesheets when you want to apply a style to a single occurrence of an element. Inline stylesheets are declared within individual tags and affect those tags only. Inline stylesheets are declared with the *style* attribute.

Example:

```
<HTML><head><title>inline style</title></head>
<body>
<p style="color:gray">This text will be gray.</p>
<h1 style="font-size:100pt;color:blue;text-decoration:underline">PUSET</h1>
<h2 style="font-size:50pt;color:red;text-decoration:overline">BIRATNAGAR</h2>
</BODY></HTML>
```

Creating an internal stylesheet

Use an internal stylesheet when you want an HTML document to have a unique style. An internal stylesheet is defined using the `<style>` tag and goes in the head section of an HTML document.

A style sheet helps us control the style (size, color, background of text and images and so on) of web document. Different style sheets can be combined or cascaded to control appearance of a web document.

The `<style>` tag specifies the content type of a stylesheet with its **type** attribute which is also set to "text/css".

Syntax:

```
<style type="text/css"> styles go here </style>    i.e:
<style type="text/css">
Tag1{attribute1:value;attribute2:value;.....}
Tag2{attribute1:value;attribute2:value;.....}
</style>
```

Example:

```
<html>
<head><title>-----</title>
<style type="text/css">
H1{font-family:arial;font-style:oblique;font-size:20pt;color:purple;text-align:center}
P{
font-family:serif;
font-style:italic;
background-color:red;
border-width:thick;
border-color:pink;
}
</style></head>
<body>
<h1>PUSET</h1><br>
<p>BIRATNAGAR</p>
</body></html>
```

Note: <style>_____</style> tags are always includes within <head> </head> tags

Other example

```
<html>
<head>
<title>Using style sheet</title>
<style type="text/css">
H1{font-family:serif;font-weight:400}
P{font-size:14pt;font-style:italic}
</style></head><body>
<center><h1>welcome to Biratnagar</h1></center>
<hr>
<p>Introduction: It is the second largest city of Nepal.</p>
</body></html>
```

```
<html>
<head><title> using font attributes</title>
<style type = "text/css">
H1{font-size:26pt; color:red ;text-decoration:blink}
P{text-indent:0.5in ;font-style:italic;font-family:arial}
H5{font-size:15pt;text-align:center}
</style></head>
<body>
<h1> welcome to puset </h1>
<br> <hr>
<p> introduction:.....
.....</p>
<br> <hr>
<h5>see you again </h5>
</body></html>
```

Class style: A class style is more flexible method of defining style sheet. A dot (.) operator is used to define a class. Its attributes are defined within curly braces {}. We can use this class style as an external style sheet or as an embeded style sheet. It is accessed inside class using the following code:

```
<tag class="class-name">.....</tag>
```

Moreover, you can create two types of class styles: **elements specific style** and **universal style class**. An element specific style class starts with element name followed by dot operator, which is followed by the class name.

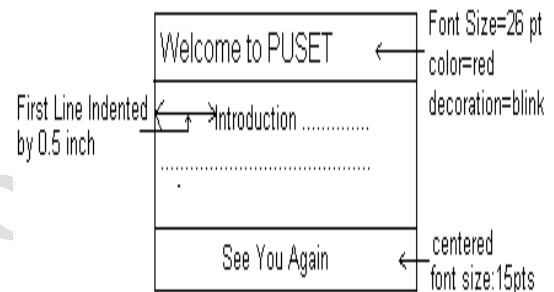
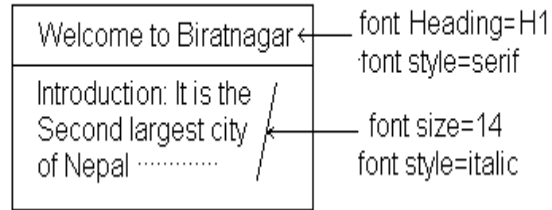
A class is defined as:

```
.class name{ }
```

Example: Element specific

```
<html><head><title>using class style</title>
<style>
.font1{font-size:50pt;color:blue;}
.font2{font-size:40pt;color:red;}
```

Write DHTML code for following feature



```
</style></head>
<body>
<font class="font1">Font with style font1</font1>
<font class="font2">Font with style font1</font2>
</body></html>
```

Example1:

```
<html><head><title>using class</title>
<style type="text/css">
P{font-family: serif; margin-left:10%; margin-right:10%}
.question{font-style:italic; color:red}
.answer{font-style:normal;color:blue}
</style>
</head> </body>
<p class="question">What are output devices?</p>
<p class="answer">output devices are monitor, printer.....</p>
<p class="question">What are input devices? </p>
<p class="answer">input devices are keyboard, Mouse,.....</p>
</body></html>
```

Using span with class

```
<head><title>spanning class</title>
<style type="text/css">
P{font-family:serif; margin-left:10%;margin-right:10%}
.question {font-style:italic; color:red}
.page{font-size:14; text-decoration:underline;color:black}
.answer{font-style:normal;color:blue}
</style></head>
<body><p class="question">what are <span class="page">output </span>devices?</p>
<p class="answer">The output devices are .....</p>
<p class="question">what are <span class="page">input </span>devices?</p>
<p class="answer">the input devices are .....</p>
</body></html>
```

A universal style class

It starts with a dot operator (.) followed by the class name. Such class can be used in any tags such as paragraph, font, headings, div etc. A division style helps you to define the position inside a page. The position properties define the actual position of the object in a browser. There are three types of positions: static, absolute, and relative. The position is measured by the two attributes: left/right : N (in pixel), and top/bottom : N (in pixel)

- **Static:** It is default position, which places the object wherever you wrote the code
- **Absolute:** It allows placing the object anywhere within a page.
- **Relative:** It doesn't overlap the object.

Example: Universal class style.

```
<html><head><title>Universal class</title>
<style>
.x1{font-size:50;color:red;font-family:arial}
```

Using CSS

```
<html><head><title>using css</title>
<style type="text/css">
H1{color:red; font-style:italic}
H2{color:blue}
P{color:yellow}
</style>
<body>
<h1>School of</h1>
<h2>Engineering &</h2>
<p>Technology</p>
</body></html>
```

```
.x2{font-size:40;color:blue;font-family:arial black}
</style>
<body>
<div class="x1">this is x1 heading</div>
<div class="x2">this is x2 heading</div>
</body></html>
```

<DIV>.....</DIV> Tag

A web page can also be divided into many segment or division. A division start with the tag <div> tag and end with the tag </div>. The attributes of<div> can be specified to position a segment. A segment or division can be set to have an absolute with reference to (0,0) or relative positive with respect to an element of the web page.

HTML Table Based Layout

Table provides the simplest way for creating layouts in HTML. Generally, this involves the process of putting the contents such as text, images, and so on into rows and columns.

HTML Div Based Layout

Using the <div> elements is the most common method of creating layouts in HTML.

The <div> (stands for division) element is used for marking out a block of content, or set of other elements inside an HTML document. It can contain further other div elements if required.

Table Vs Div Layout

<div> tag is used to group block and inline level elements and to create divisions in a webpage. Before **2004**, we were using **Table Based Layouts**. But After 2004, **Div based layout** become popular. Div can render fast as compare to table, thus improve page performance. Div can create both **Liquid Layouts** and **fixed Layouts**. Here we'll learn how to create a **fixed layout** of a Webpage using <div> tag and CSS.

TABLE	DIV
Hard to Maintain	Easy to maintain
Good only for fixed layout	For both fixed and liquid layouts
Styled by CSS & html attributes	Styled by CSS only
Not good for small screens	Good for small screens
Render slow on browser	Render fast on browser

Program

```
<html>
  <head> <title> using div tag </title>
</head>
<div id = box1 style = "position: absolute; left: 50; top: 100; width: 50 background-color: red">
<img src ="ima1.gif" height=50 width=50>
</div>
<div id = box2 style = "position: absolute; left: 110; top: 100; width: 50 background-color: blue"
height=50 width=50>
<img src ="ima2.gif">
</div>
<div id = box3 style = "position: absolute; left: 50; top: 160; width: 50 background-color:
yellow" height=50 width=50>
<img src ="ima3.gif">
</div>
```

```
<div id = box4 style = "position: absolute; left: 110; top: 160; width: 50 background-color:
green" height=50 width=50>
<img src ="ima4.gif">
</div>
</body></html>
```

Creating an external stylesheet

It is a separate HTML document file with extension .css. the css file contains css rules which are linked to the HTML file by using the following code in <head> section.

```
<link rel="stylesheet" href="filename.css">
```

Use an external stylesheet when you want to apply one style to many pages. If you make one change in an external stylesheet, the change is universal on all the pages where the stylesheet is used.

An external stylesheet is declared in an external file with a .css extension. It is called by pages whose interface it will affect. External stylesheets are called using the <link> tag which should be placed in the head section of an HTML document. This tag takes three attributes.

Attributes of the <link> tag:

- **rel** - When using an external stylesheet on a webpage, this attribute takes the value "stylesheet"
- **type** - When using an external stylesheet on a webpage, this attribute takes the value "text/css"
- **href** - Denotes the name and location of the external stylesheet to be used.

Example: First file contains the css script and second file contains HTML code. Save both file in same folder otherwise give css file name with path

Test.css(CSS File)	Test.html (HTML File)
.x1{font-size:60;color:red;font-family:arial} .x2{font-size:50;color:blue;font-family:arial black} .x2{font-size:40;color:green;font-family:arial}	<html><head><title>style</title> <link rel="stylesheet" href="test.css"></head> <body> PUSET PUSET PUSET </body></html>

NOTE: The <style> tag is NOT used in an external stylesheet, and neither are HTML comments.

Converting PSD files into HTML

The PSD file that you have created, which is made of several layers should be sliced. Slicing is the technical term used to refer to breaking up of a single large image to multiple small images. One of the advantages of using sliced version of the PSD in your HTML page is that it will aid in faster loading of the pages. If the entire PSD file is going to be a single image file, then it will take a lot of time for the page to load, which in turn could affect the quality of the user experience. To slice your image, make use of the slicing tool available in PhotoShop.

Slicing

Three types of slicing options are available in PhotoShop for breaking your web page into small pieces, namely **Normal**, **Fixed Aspect Ratio** and **Fixed Size**. If you are using the **Normal option** to slice your PSD file, the proportions of the image will be shown as you drag the slicing marker lines.

The fixed aspect ratio slicing allows you to feed the required size whereby the sliced portions will change based on the figures that you feed. For example, if you want to triple the width of

the sliced image then you just need to enter 3 for the field pertaining to the width and enter 1 in the field for the height.

In case, you want all the slices to be of specific height and width, then choose '**Fixed Size**'. Here you should enter the required sizes in pixel values.

Another easy way to slice the PSD is by adding guides to the image and using the 'Slices from Guides' option in the Slice tool. When you use this option, the file will automatically be sliced as per the guides. Remember, all previous slicing actions will be nullified when you use the 'slices from guides' option.

Once you have sliced the PSD file, make sure to save the sliced version. Go to file and save it using the option "Save for the Web." Save these images in the 'images' directory, which you have already created.

Using list

```
<html>
<head>
  <title> using list</title>
<style type="text/css">
Ol{list-style-type:lower-roman}
</style></head><body>
  Input devices are
<OL >
  <li> keyboard
  <li > mouse
  <li > microphone
</OL>
  </body > </html>
```

JSSS (Javascript supported style sheet)

```
<html>
<head><title>Using JSSS</title>
<style type="text/javascript">
Document.tags.h1.color="red";
Document.tags.h2.font-style="italic";
Document.tags.p.color="yellow";
</style></head>
<body>
<h1>School of</h1>
<h2>Engineering & </h2>
<p>Technology</p>
</body></html>
```

JSSS means JavaScript supported style sheet. Using JSSS, a web developer can specify values for different attributes using javascript codes in style sheet as shown in the above diagram. Color of the text information to which <h1> tag will be in red, moreover textual information will appear in italic.

Color of the text to which <h2> tag applies will be in blue. Similarly, color of textual information to which tag <p> applies will be yellow.

Different attributes are accessed through document tag attributes.

Also type is set equal to “text/ JavaScript”.

Layer

Layer is a transparent object laid on the top of webpage. Layer start with a tag <layer> and end with </layer> tags.

Different attributes values can be specified in a layer tag to control its appearance.

Property	Description
id /name	To specify a id name value.
Top,left	To specify the position of the layer
Height	To specify the height of the layer
Weidth	To specify the weidth of the layer
Bgcolour	To specify the background color of the layer.
Background	To specify the background.
z-index	To specify the order of stacking for a layer.

```
<html> <head><title>using layer</titlt></head>
```

```
<body>
```

```
<layer id= box1 left=400 top=600>
```

```
<img src ="ima.gif" width=50 height=50>
```

```
</layer>
```

```
</layer id=box2 left=420 top=620>
```

```
 | <pre>&lt;html&gt; &lt;head&gt; &lt;title&gt;-----&lt;/title&gt; &lt;/head&gt; &lt;body&gt; &lt;script language="Javascript"&gt; ----- ..... js codes &lt;/script&gt; &lt;/body&gt;&lt;/html&gt;</pre> |

- a) JavaScript enabled browsers can understand and execute JavaScript codes.

- b) Both Netscape communicator and Microsoft Internet Explorer support JavaScript codes .Since JavaScript was originally invented by Netscape, using Netscape communicator, the full functionality of JavaScript can be availed.
- c) The browser internet explorer supports VB scripts completely, while the browser Netscape navigator does not.
- d) Using HTML tags, although data can be entered and captured data validation and handling require the use of JavaScript or VB script codes.

### **CAPTURING INPUT:-**

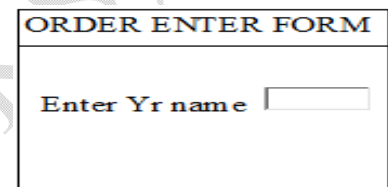
A HTML form allows us to capture data. However, that captured data often need to be validated and checked for presence of errors. HTML tags do not have these capabilities. It is compulsory then to embed JavaScript codes in the same HTML document to achieve this effect.

A HTML form can be initiated with several objects such as text area, radio button, check box and so on. For example: in order to have this display in an HTML form.

We can use the

`<form> ----- </form>` tags as follows:

```
<form name=name>Enter Yr name
<input type ="text" name="name" value="Ram">
</form>
```



Once captured data are validated and checked for errors at the client side, it is sent to the server. The server on the other side will use its own codes written in JavaScript, VB script or Java, bundle the processing information and send them all in the form of HTML document.

Thus a HTML page is created on demand. This HTML page has been created dynamically; it has all possible because of JavaScript codes.

### **ADVANTAGES OF JAVASCRIPT:-**

JavaScript is an object oriented language that allows creation of interactive web pages. JavaScript allows users entries, which are loaded into an HTML form to be expressed as required. This empowers a web site to return site information according to a user's request. JavaScript offers several advantages to a web developer. These are as follows:-

**An interpreted language:** JavaScript is an interpreted language, which avoiding the compilation process which would have taken more time. This provides an easy development process.

**Embedded into HTML document:-** JS does not require any special or separate editor for programs to be written, edited, or compiled. It can be written in any text editor like notepad, along with appropriate HTML, tags, and saved filename.html. HTML files with embedded JavaScript commands can then be read and interpreted by any browser that is JS enabled.

### **Cross Browser Supporting:**

This means that the JavaScript codes are supported by various browsers like Internet Explorer, Netscape Navigator, Mozilla etc.

### **Light weight for downloading:-**

The javascript codes run directly in the client machine. The code should be downloaded all the way from server to the client and this time duration is very minimum and the executing the codes of javascript is also very fast.

### **Validating Data:**

The data can be validated in the two different way:

- Validating is server side or in server machine.

- Validating in client side or in client machine.

In these two different types of validation of data second one is much more faster, and this is done through javascript.

### **Sophisticated User Interfaces:-**

By using javascript you can create a user interactive interfaces that can communicate with the user and the Browser.

### **In-Built Software:**

You don't need any extra tools to write javascript, any plain text or HTML editor will do, so there's no expensive development software to buy.

### **Easy debugging and testing:-**

Being an interpreted language, JavaScript's scripts are tested line by line and the errors are also listed as they are encountered i.e. an appropriate error message along with the line number is listed for every error that is encountered. It is thus easy to locate errors, make change and test it again without the over head and delay of compiling.

### **Short development time(quick development):**

JavaScript does not require time consuming compilations, only interpretation required. Usually codes are concise but powerful.

### **Easy to learn(Minimal syntax):**

By learning just a few commands and simple rules of syntax, complete applications can be built using JavaScript. Since it supports many syntax and data types as C and C++

### **Performance:**

JavaScript can be written such that the HTML files are fairly compact and quit small. This minimizes storage requirement on the web server and downloaded time for the client.

Additionally, because JavaScript programs are usually included in the same file as the html code for web page they require fewer separate network accesses.

### **Platform independent( codes run on any machine and executed on any OS)**

JavaScript is a programming language that is completely independent of the hardware on which it works. It is a language i.e. understood by any JavaScript enabled browser. Thus JavaScript applications work on any machine that has an appropriate JavaScript enabled browser installed. This machine can be anywhere on the network. A JavaScript program developed on a unix machine will work perfectly well on a windows machine.

The fact that a platform specific browser maintained at the client and does the interpretation of JavaScript, relieves the developer of the responsibility of maintaining multiple source code files for multiple platforms.

### **Functionality(procedural capabilities):**

Like in any programming language JavaScript can also create functions, condition checking, coping and branching.

JavaScript is object based programming. It supports event driven features. That is when a button

### **Event Driven(design for programming user events):**

is clicked; some program statement (associated with that button) will be executed in the background.

### **DISADVANTAGES OR WEAKNESS:**

Early version of the navigator 2.0 beta included versions of JavaScript that was rather limited. In the final release of navigator 2, which was current version where, the number of built in

method had significantly increased, but still didn't included a complete set of methods to work with documents and the client windows. Still the current versions are about as complete as we get until navigator2.1.

**1. No code hiding:** - Because the source of JavaScript presently be included a part of the HTML source code for a document, there is no way to protect code from being copied and reused by people who view the web page.

This raises concerns in the software industry about protection of intellectual property. The consensus is that JavaScript are basically freeware at this point in time.

**2. Lack of debugging and development tools:-** Most will developed programming environment include a suitable tools which make development easier and simplicity and speed up the debugging process .

But in case of JavaScript , there were no JavaScript, specific tools available for this purpose, although some people had began developing web based applications .(using JavaScript) to achieve some of this.

**Some other Weaknesses are:-**

- It is not used to create stand alone application; it is only used to add some functionality in the web page.
- Javascript can't read and write files into the client machines. It can only be used as a utility language to develop any web site.
- Javascript can't be used to send email to the visitors or user of the website. This can be done only with the server side scripting.
- Can't create database application using JS. For this you need to use server-side scripting.
- Javascript has limitation of writing in a client machine. It can only write the cookie in client machine that is also of a certain size i.e.. 4K.

**MYTHS (JAVASCRIPT MYTHS)**

Some common and persistent-myths about JavaScript are described as follows:-

**JavaScript is not java:-** One of the common misconceptions about JavaScript is that it is simplified versions of java. Java and JavaScript do resemblance and both of them do provide executable content in web browser otherwise they are entirely unrelated. The similarity of names is poorly a marketing publicity, the language is originally named Live script and was later are renamed as JavaScript.

Java and JavaScript do however make a good team. JavaScript can control browser, behaviour and content but cannot draw graphics and or perform networking but can do graphic and networking. Client side JavaScript can interact with and control java applet embedded in a web page.

1. **JavaScript is not simple:** - Through JavaScript appears at first glance to be fairly simple language. It is a full –features of programming language, as complex as any, and more complex than some. One may find writing javascript programs frustrating if he/she doesn't have a solid understanding of language.

**JAVASCRIPT Vs JAVA**

JavaScript and java are like in more than just name. However there are significant differences between these two languages. The table shows a quick overview of the difference:

<b><u>Java Script</u></b>	<b><u>Java</u></b>
→JS is small, lightweight prog. Lang.	Java is a full-blown powerful, sophisticated PL.

→ Interpreted by client at run time	→ Compiled by author run on client
→ Code interpreted in HTML document	→ Applet distinct from html document
→ object based has limited no. of built-in-Object	→ object oriented, can create their own classes.
→ Loose typing of data types	→ Strong typing of data types
→ Dynamic finding.	→ Static finding
→ Script limited to browser function.	→ Stand alone applications
→ Works with HTML document.	→ Goes beyond html (E.g. multimedia)
→ It doesn't support graphic drawing and networking.	→ It supports graphics drawing and networking.
→ Developed by Netscape communications	→ develop by sun micro system.

### **Combining JavaScript with HTML:**

Java script system is included in to an HTML file. A browser reads HTML files and interprets HTML tags. Since, all JavaScript need to be included as an integrated part of an HTML document when required, the browser need to be informed that specific sectors of HTML code in JavaScript. The browsers will that use it built in JavaScript engine to interpreter this code.

The browser is given this information using the HTML tags <script> tag, an extension of HTML supported by Netscape, enable programmer to include one or more JavaScript function in the web page.

```
<html>
<head><Title>.....</title>
 <script language= "JavaScript">
 Variable declaration;
 </script></head>

<Body>

 <script language= "JavaScript">
 Program statement;
 </script></body></html>
```

### **JavaScript security**

Because of the wide open nature of the internet, security is an important issue. Since, coding web page can cause arbitrary code to be executed on our computer, high security precautions are required to prevent malicious code, from doing any damage to our data.

JavaScript is first line of defence against malicious code is that the language simple does not support certain capabilities. For e.g. client-side JavaScript does not provide any way to write or delete files or directories on the client computers.

Similarly, client side JavaScript has no networking capabilities. A JavaScript program can load URLs and sent HTML from data to web server, but it can't established a direct connection to any other host on the network.

Practically, all known security issue in JavaScript have been resolved in new versions of web browser.

### **Building up JavaScript syntax:**

**Basic programming technique:** Like in any language, a variable is used to store value in some designed portion of program. The syntax for declaring variable is var <variable name>;

### **We can also initialize the variable as:**

```
var <variable name>=name value;
E.g. var value(a) = 4;
```

JavaScript is loosely cast or loosely typed i.e. a variable does not have any fixed data typed as it is in case of C, C++, Java, etc. therefore the same variable can be assigned different types of data at different places in a program. E.g.

<html><head><title> Assigned different values to same variable</title>

```
<script language="Javascript">
var value=0
</script> </head>
<body>
<script language="Javascript">
document.write("the value of value is"+value);
document.write("
");
value=120.34;
document. write("the value of value is "+value);
document. write("
");
value="Hello";
document. write("the value of value is "+value);
</script> </body> </html>
```

Output

The value of value is 0  
The value of value is 120.34  
The value of value is Hello

Normally variable, functions, arrays, objectives are declared and initialized in the <head> section. However these can be done in the <body> section also.

### **Embedding JavaScript:**

JavaScript code is typically embedded into an HTML document using the SCRIPT tag. You are free to embed as many scripts into a single document as you like, using multiple SCRIPT tags. A script embedded in HTML with the SCRIPT tag uses the format:

```
<script language="JavaScript">
<!--
document.write("Hello World!");
//-->
</script>
```

The LANGUAGE attribute is optional, but recommended. You may specify that a section of code only be executed by browsers which support a particular version of JavaScript; for instance:

### **Variables and Data types:**

Variables store and retrieve data, also known as "values". A variable can refer to a value which changes or is changed. Variables are referred to by name, although the name you give them must conform to certain rules. A JavaScript identifier, or name, must start with a letter or underscore ("\_"); subsequent characters can also be digits (0-9). Because JavaScript is case sensitive, letters include the characters "A" through "Z" (uppercase) and the characters "a" through "z" (lowercase). Typically, variable names are chosen to be meaningful regarding the value they hold.

### **Scope of variable**

When you assign a new variable to an initial value, you must consider the issue of scope. A variable may be scoped as either *global* or *local*. A global variable may be accessed from any JavaScript on the page. A local variable may only be accessed from within the function in which it was assigned. Commonly, you create a new global variable by simply assigning it a value.

newVariable=5;

However, if you are coding within a function and you want to create a local variable which only scopes within that function you must declare the new variable using the var statement:

In the example above, the variable *loop* will be local to *newFunction()*,

```
function newFunction()
{
var loop=1;
total=0;
...additional statements...
}
```

while *total* will be global to the entire page.

**Types of values:** A value, the data assigned to a variable, may consist of any sort of data. However, JavaScript considers data to fall into several possible types. Depending on the type of data, certain operations may or may not be able to be performed on the values. For example, you cannot arithmetically multiply two string values. Values can be of these types:

Numbers	Integer and floating-point numbers.   Integers can be positive, 0, or negative; Integers can be expressed in decimal (base 10), hexadecimal (base 16), and octal (base 8). A decimal integer literal consists of a sequence of digits without a leading 0 (zero). A leading 0 (zero) on an integer literal indicates it is in octal; a leading 0x (or 0X) indicates hexadecimal. Hexadecimal integers can include digits (0-9) and the letters a-f and A-F. Octal integers can include only the digits 0-7.   A floating-point number can contain either a decimal point, an "e" (uppercase or lowercase), which is used to represent "ten to the power of" in scientific notation, or both. The exponent part is an "e" or "E" followed by an integer, which can be signed (preceded by "+" or "-"). A floating-point literal must have at least one digit and either a decimal point or "e" (or "E").
Booleans	True or False. The possible Boolean values are true and false. These are special values, and are not usable as 1 and 0. In a comparison, any expression that evaluates to 0 is taken to be false, and any statement that evaluates to a number other than 0 is taken to be true.
Strings	"Hello World !" Strings are delineated by single or double quotation marks. (Use single quotes to type strings that contain quotation marks.)
Objects	myObj = new Object();
Null	Not the same as zero - no value at all. A null value is one that has no value and means nothing.
Undefined	A value that is undefined is a value held by a variable after it has been created, but before a value has been assigned to it.

That said, JavaScript is a loosely typed language -- you do not have to specify the data type of a variable when you declare it, and data types are converted automatically as needed during script execution. By and large, you may simply assign any type of data to any variable. The only time data typing matters is when you need to perform operations on the data. Certain operators behave differently depending on the type of data being deal with. For example, consider the + operator:

5" + "10"	yields	"510" (string concatenation)
5 + 10	yields	15 (arithmetic sum)

### Rules for naming a variable:

0 to 9, A to Z, a to z, \_(underscore) etc.”\$” not used.

First latter should be alphabet.

Normally we use small latter.

Special character not allowed.

# Write a program in JavaScript to add numbers 2 and 5.2.

```
<HTML> <head><title> adding 2 and 5.2 </title>
 <script language= "Javascript">
 var num1=2
 var num2=5.2
 var num3
 </script> </head>

<body>
<h1> Adding two numbers </h1>
<script language="Javascript">
num3=num1+num2;
document.write("the sum of 2 & 5.2 is" +num3);
</script></body> </html>
```

### **Typecasting:**

In JavaScript variables are loosely cast. The type of a JavaScript variable is implicitly defined based on the literal values that are assigned to it from time to time.

In other words, type casting is a feature which allows a data type of an expression to be converted into another data type for e.g.

document.write("the sum is" +10);

Will give the following display

The sum is 10

Here, the number 10 is first converted into string 10, and then only the concatenation operation was performed.

**String literals:** A string is a sequence of zero or more character enclosed within single or double quotes (' or " "). Double quotes characters can be contained within strings enclosed by single quote character and vice versa.

E.g.

```
'hello'
"3.25"
'name = "my form" '
```

E.g.

```
<html><head><title> a variable</title></head>
<body><script language="Javascript">
document.write('');
document.write('<bgsound src="01.mp3">');
document.write("<h1> greeting, </h1>");
document.write(prompt("enter your name:", "your name"));
document.write("welcome to PU! </h1>");
document.write('Click me ');
</script> </body> </html>
```

**Working with string:**

One of the built in features of JavaScript is the ability to concatenate strings. If we use the '+' operators with number it address them. But if we use the use the operator on string, it joins them by appending the second to the first. Examples are:

msg = "hello" + "word" //produces the string hello word.

**Optional semicolon:**

Simple statements in JavaScript are generally followed by semicolons (;) just as they are in C, C++ and java. This serves to separate statements from each other. In JavaScript, however we are allowed to omit this semicolon is the statements are each place in as separate line.

E.g. a=3

b= 5

- But when formatted as follows, the first semi colon is required a=3; b=5;
- Omitting semicolon is not a good programming practice; we should get in the habit of using them.

**Comments:**

JavaScript supports both C++ and C style comments. Any text between //and the end of line is treated as a comment & is ignored by JavaScript. Also, any text between the characters /\* and \*/ is treated as a comment.

→ In addition, JavaScript recognizes the HTML comments opening sequence '<!--'

→ Treats this is a single line. Comment, just it does the // comment. In a JavaScript program if the line being with '<!--' and the last line ends with '-->' or //>, the entire program is contained within an HTML comment and is ignored by browser that do not support JavaScript.

Yub	Raj	Yesh	Kumar
[0]	[1]	[2]	[3]

**# Escape sequence in string literals:**

The back slash character (\) combined with a character from an escape sequences. Eg. \b- backspace; \t – tab;

yesh	.....	raj
------	-------	-----

\\backslash ; \r – carriage return; \f – form field; \“- double quotes ; \ ‘ single quotes;

**Arrays**

JavaScript arrays allowed us to keep a sequence of value. Array elements are store in consecutive memory locations. Individual array element can be accessed by using an index enclosed within square brackets immediately following the name of array.

A[0]	A[1]	.....		A[n-1]
------	------	-------	--	--------

**The array element index starts with 0.**

Hence the last array element index number is one less than the length of the array (n-1)

**Syntax:-**

Name-of-array = new Array (size of array)  optional.

Eg.

```
A = new Array (10);
customer_order = new Array(10); //ten elements in array element
customer_names= new Array();
customer_name[0]= "Yesh";
customer_name[50]= "Raj";
```

Cuct-name[0]..... Cust-name[50]

Examples:

```
<HTML><head> <title> using arrays </title> </head>
<body>
<h1> processing arrays </h1>

<script language="javascript">
customer_name=new Array(4);
customer_name[0]="BCA";
customer_name[1]="BIT";
customer_name[2]="BEE";
customer_name[3]="BEC";
document.write('Value of first element is'+ '+customer_name[0]+'
');
document.write(customer_name[1]+'
');
document.write(customer_name[2]+'
');
document.write(customer_name[3]+'
');
</script> </body> </html>
```

❖ New is an operator used to create an array object

### Dense Array

In case of a dense array, the array is initialized with values while it is declared. For e.g.

customer\_name= new Array("value1", "value2", "value3", ....., "valuen");

e.g. customer\_name= new Array("jeevan", "prakash", "Yes", "raj");

Mixed values= new Array ("drona", 30,null ,1.2 "jay");

### Join Array:

Returns all elements of the array joined together as a single string. This takes one argument; a string to be used as a separator between each element of the array in the final string. If the argument is omitted, join() uses a comma – space as the separator.

### Reverse Array:

Reverses the order of the elements in the array.

### Array Methods

The Array object has the following methods:

- concat joins two arrays and returns a new array.
- join joins all elements of an array into a string.
- pop removes the last element from an array and returns that element.
- push adds one or more elements to the end of an array and returns that last element added.
- reverse transposes the elements of an array: the first array element becomes the last and the last becomes the first.
- slice extracts a section of an array and returns a new array.
- sort sorts the elements of an array.

**Example: Program showing normal array, join array, reverse array, and length of array**

```
<html><head>
<title>Showing different array</title></head>
<body>
```

```

<script language="javascript">
students=new Array(5);
students[0]="Shuvam";
students[1]="Shivam";
students[2]="Sijan";
students[3]="Shaurav";students[4]="Yubraj";
document.write(students[0]+"
");
document.write(students[1]+"
");
document.write(students[2]+"
");
document.write(students[3]+"
");
document.write(students[4]+"
");
join_crit=students.join();
document.write(join_crit);
document.write("
")
rev=students.reverse();
document.write(rev);
document.write("
length="+students.length)
</script></body></html>

```

### **Expression and operator**

**Operators** are mathematical symbol that perform some sort of operations upon data operators act upon alphanumeric pure numeric or string data. The data which on operators acts is known as operand. For e.g. consider the following statements  $A+B$ , where '+' is the operator and A, B are operands.

Operators take one or more variables or values (*operands*) and return a new value; e.g. the '+' operator can add two numbers to produce a third. You use operators in expressions to relate values, whether to perform arithmetic or compare quantities. Operators are divided into several classes depending on the relation they perform:

The combination of an operator & its operands is known as expression. Most of the programming languages used two categories of operators binary & unary. Binary operator requires the two operands eg.  $A+B$ . While Unary operator require a single operand e.g.  $A++$  or  $++A$  or  $A--$  or  $--A$  etc.

#### → **Unary operator:**

++ (Increment operator eg  $++a$  means  $a = a+1$ )

-- (Decrement operator eg  $--a$  means  $a = a-1$ )

#### → **Binary operators:**

**Arithmetic operator:** (+, -, /, \*, %)

Arithmetic operators take numerical values (either literals or variables) as their operands and return a single numerical value.

#### → **Relational or Comparison**

**operator:** A comparison operator compares its operands and returns a logical value based on whether the comparison is true or not. The operands can be numerical or string

= = (equal to)	< (less than)
!= (not equal to)	<= (less than equal to)
= = = (strictly equal to)	> (greater than)
!= = (strictly not equal to)	>= (greater than equal to)

values. When used on string values, the comparisons are based on the standard lexicographical (alphabetic) ordering.

→ **Boolean or Logical operator:**

- &&(logical AND)—evaluates true only when both conditions are true.
- || (logical OR) –Evaluates true when either of the two condition is true.
- ! (logical NOT) –Operands on a single value & converts true to false and vice- versa.

→ **Assignment operator:** The assignment operator (=) lets you assign a value to a variable. You can assign any value to a variable, including another variable (whose value will be assigned). Several shorthand assignment operators allow you to perform an operation and assign its result to a variable in one step.

=, +=, -=, \=, \*=, %=

(e.g. a+=b evaluates a=a+b )

**Operators used for string**

Strings can be compared using the comparison operators. Additionally, you can concatenate strings using the + operator.

"Dynamic" + "HTML"	yields	"DynamicHTML"
--------------------	--------	---------------

→ **Modulus:** the remainder after division; e.g. 10 % 3 yields 1.

→ **New operator:** The term 'new' operator creates an array object having the array name with its memory e.g. customers\_new\_Array[];

→ **Delete operator:** E.g. delete customers [3]: here, the delete operator destroys an instance of an object. Here, 4<sup>th</sup> array element of the array customers is destroyed.

**Operators used for string**

Strings can be compared using the comparison operators. Additionally, you can concatenate strings using the + operator.

**Statements (Control Statements):**

Conditional statements direct program flow in specified directions depending upon the outcomes of specified conditions. These tests are a major influence on the order of execution in a program. Every program is made up of several statements where each statement is an instruction to perform some operation. The various statements are described in JavaScript are:

**(1) If Statement:**

```
if (condition)
{
 statement(s);
}
```

If condition is evaluates to true then the statement is executed otherwise ignore it.

**(2) If else statement:** *if* the condition evaluates to true then the block of *statements1* is executed. Optionally, an *else* clause specifies a block of *statements2* which are executed otherwise. You may omit the else clause if there are no statements which need to be executed if the condition is false.

```
if (condition)
{
 statement(s) //block 1
}
```

```

else
{
 statement(s) //block 2
}

```

If the condition is true then the statements within block 1 get executed otherwise statements within block 2 get executed but never both.

### (3) Switch case statement:

Commonly known as a "case statement," **switch** matches an expression with a specified case, and executes the statements defined for that case. In essence, the switch statement is a sort of shorthand for combining many implied **if** statements together. It is used when a variable is to be successfully compared against different values. The following switch statement is equivalent to repeated **if / else**.

#### Syntax:

```

switch (expression)
{
 case label :
 statement;
 break;
 case label :
 statement;
 break;
 ...
 default :
 statement;
}

```

**Note:** The *break;* statement prevents any cases below the match from being executed. The *default* case is matched if none of the cases match the expression.

**(4) While statement:** In similar fashion as the **do...while** statement, the **while** statement executes its statement block as long as the condition is true. The main difference between **while** and **do...while**, aside from the fact that only **while** is supported in all JavaScript versions, is that a while loop may not execute the statements even once if the condition is initially false.

```

while (condition)
{
 statement(s);
}

```

It is used for executing a set of statements repeatedly, while a specific condition evaluates to true.

**(5) Do while statement:** Another loop, a **do...while** statement executes a block of statements repeatedly until a condition becomes false. Due to its structure, this loop necessarily executes the statement at least once.

```

do
{

```

```

 statement(s);
}
while (loop condition)

```

It is same as while, the only difference is that the looping condition is checked at the end of loop, instead of the beginning. This means that the statements within the loop get executed at least one.

**(6) For loop:** The venerable **for** loop repeatedly cycles through a block of statements until a test condition is false. Typically, the number of times a loop is repeated depends on a counter. The JavaScript **for** syntax incorporates the counter and its increments:

```

for (initialization statement; condition; update statement)
{
 statements;
}

```

```

for (i=0; i<10; i++)
{
 statements;
}

```

The initialization is executed first, and once only. Commonly, this statement is used to initialize a counter variable. Then the condition is applied and if it succeeds then the statements are executed. The iteration (increment/decrement) is applied to the counter variable and then the loop starts again. For instance, consider a loop, which executes 10 times:

<u>Eg :</u>	<u>O/P</u>
for(i=1;i<5;i++)	i=1
{	i=2
document write ("i="+i+" ");	i=3
}	i=4
	i=5

**(7) Break statement:** It causes the innermost closing loop or a switch statement to exit immediately. It also a very simple syntax break;

**(8) Continue statement:** It is a similar to a break statement. Instead of exiting a loop, however continually starts. Restarts a loop in a new iteration. It has a syntax i.e., just as the break statement continue.

**Function:** Functions are block of JavaScript code that perform a specific task and often return a value. A function may take one or more parameters. Parameters are a standard technique via which control data can be passed to a function.

### Types of functions:

#### 1. Built-in-functions:

It provides several built-in-functions that can be used to perform explicit type conversion. Some of them are eval(), parseInt() and parseFloat().

**Eval() :-** It is used to convert a string expression to a numeric value. e.g.:

```
var total = eval ("10*10+5");
```

Result in the value 105 being assigned to the variable total. If number is not contained, the argument of eval( ), it gives error message.

**parseInt:-** Return the first integer part of string. E.g.: parseInt("125 hello!") will return 125 i.e., only first integer part of the string 125. if it doesn't have an integer as the first part of string, it gives error NAN(not a number).

E.g.: num = parseInt(prompt("enter number", "num"));

**parseFloat:-** Returns the first floating point number part of the string.

e.g.: `parseFloat("125.45 hello!")` will return 123.45 only. It returns NAN, if a first part of the string happens not be the floating point number.

**2. User defined function:** Functions offer the ability to group together. The program code that performs a specific task into a single unit that can be used repeatedly whenever required.

Function can accept information in the form of argument and can return result. Appropriate syntax needs to be followed for

Declaring function,

Invoking them,

Passing them value and accepting their return value.

### Syntax:

```
function function_name(parameter 1, parameter 2,-----, parameter n)
{
 //codes or statement
}
```

e.g.:<html><head><title>using function</title>

<script language="JavaScript">

function sum(num1,num2)

```
{
var num3= num1+num2;
return num3;
}
```

</script></head>

<body> <script language="JavaScript">

```
var num4;
num4=sum(1,2);
document.write("the sum is:"+num4);
</script></body></html>
```

```
<script language="JavaScript">
var n1 =prompt("enter 1st no: ""no");
var n2 =prompt("enter 2nd no: ""no");
var n4 =calculate.sum(n1,n2);
```

### 3. Recursive function:

A function that calls itself is called recursive function.

E.g,

<html><head><title>using recursion</title>

<script language="javascript">

function factorial(num1)

```
{
if(num1>1)
{
 return num1*factorial(num1-1) }
else
{
 return num1;
}}
```

</script></head> <body>

Finding factorial<br>

O/P

Finding factorial

factorial value is120

```
<script language="javascript">
var num1=prompt ("enter no", "5");
var num2=factorial(num1);
document.write("factorial value is"+num2);
</script></body></html>
```

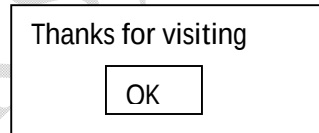
### **Dialogue boxes:**

JavaScript provides the ability to pick up user input or display small amount of text to the user by using display boxes. This dialogue box appears as separate windows and their content depend on the information provided by the user. There are generally 3 types of dialogue boxes supported by JavaScript.

#### **(1) Alert dialogue box:**

The alert method displays on alert message box. It displays a message and returns on the screen. And the execution of script is suspended until the user's clicks then OK bottom. We can use the alert methods for displaying message to the user. e.g.:

alert ("Thanks for visiting") O/P



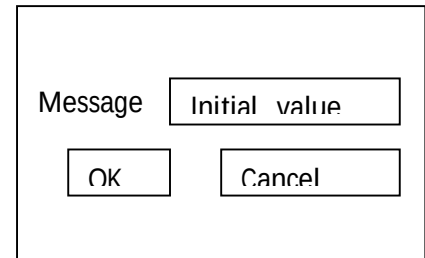
#### **(2) Prompt dialogue box:**

This function is used to allow a user to enter (as input box) a value interactively with the help of a dialog box. The prompt method accepts two arguments, the prompt for user and an optional default response, and returns the data supplied by the user.

#### **Syntax:**

```
var variable_name= prompt("message", "initial value");
```

Whatever the user has entered in the box get store in a variable name, which later be used in the program. If the user simply clicks OK bottom without entering any information. The default response is returned. If cancel bottom is clicked this function returns a null value.



#### **(3) Conform dialogue box (question):**

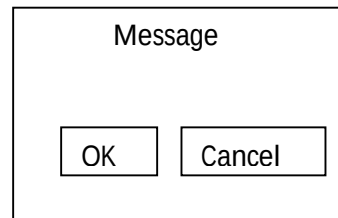
Conform display a message similar to the alert method but differ in only these boxes have two bottoms Ok and Cancel. The confirm methods returns the value true (1), if the Ok bottom is clicked and the value false (0), if the cancel bottom is clicked. This method is frequently used to notify the user that an unusual action is about to take place. Such as transmission of form's data to the server.

E.g.:

```
flag = confirm("do you submit data");
```

```
if (flag == true)
```

```
{ -----
 ----- }
```



#### **Syntax:**

```
confirm("message");
```

### **#Write a program JS (java script) using different dialogue boxes.**

```
<html><head><title> conform dialogue box </title>
```

```
<script language="javascript">
```

```
function as()
```

```
{
```

```

var flag=false
var name;
name=prompt("Please Enter Your name")
while(flag==false)
{
flag=confirm("is your name is really"+name);
if(flag==false)
{ name=prompt("Re enter your name");
}
}
alert(name+"welcome to home page");
}
</script>
</head><body onload=as()>
</body> </html>

```

### **Exception Handling** (Try, catch and throw statements)

Exception handling is a powerful and general purpose system for error handling and recovery, used by number of objects oriented language, including java, C++. It is one of the recently introduced features includes program readability and modifiability.

Script generates and exception so that the error may be handle and resolved by throw statement. The throw statement is used to thrown an exception.

The code that handles exceptions throw by our scripts (or JavaScript interpreter) is referred to an exception handler. Exception handler are said to catch exceptions.

The try statement surrounds statements for which exception handling is to be performed. It is immediately followed by catch statements that perform the exception handling. The try statement & catch statement works together to support exception.

### **SYNTAX**

```

try
{ statements where an exception may be thrown
}
catch(error variable)
{ statements that handles the exceptions }

```

### **Example**

```

<html><head>
<script language= "javascript">
function prime(n)
{
document.write("testing"+n+"")
try{if(n<1||n>20) throw "it out of range"
for(var i=2;i<n;++i)
if(n%i==0) throw "it is divisible by" +i
document.write("Its prime no
")
}
}

```

```

catch (exception)
{ document.write(exception+"
")
}
</script></head>
<body><script language="javascript">
for(i=0;i<=21;++i)
{ prime(i) }
</script></body></html>

```

### **With statements**

The “with” statement is used to temporarily modify the scope chain. We can use the “with” statement to save ourselves a lot of typing, means by using with statement eliminates re-typing the name of that object is to be referenced in a series of properties reference & methods evocation.

#### **SYNTAX**

```

with (variable name)
{
 Statements
}

```

The variable name identifies the default object to be used “with” statements enclosed in braces. E.g.

```

<html>
<head><title> With statements </title>
</head>
<body>
 <script language="javascript">
 with(document)
 {
 write("<h1> With statement </h1>");
 write("<p>");
 write("Eliminate Object");
 write("<p>");
 }
 </script>
</body>
</html>

```

In the above example we need to prefix each write method evocation. With the document object eliminated because document is identifying with statement.

### **Object**

An object is a “package” of data; a collection of properties (variable) and method & (function) all classed under a single name.

In JS, you may create your own object for storing data. More commonly, through you will use the many “built in” objects which allows to work with manipulate, and access the web page and web browser. This set of pre-existing object is known as the “Document object model” (DOM).

## DOM

Often referred to as the *DOM*, this object model is a hierarchy of all objects "built in" to JavaScript. Most of these objects are directly related to characteristics of the Web page or browser.

When webpage is download using browser the entire object contains in that web page first, stored into the memory. These objects are then assembled and the webpage is painted by the browser.

Once these objects are stored into the memory these property and methods can be accessed by using dot (.) operator. JavaScript uses DOM to map its element.

JavaScript can recognize different objects contained in a web page, which is being download successfully, mainly because it use DOM. In otherworld's, using DOM JavaScript can recognize different object that different web pages, successfully loaded, have. Once object have been recognized, their properties and methods can be accessed.

In DOM objects are related in descending manner as shown in the following diagram:

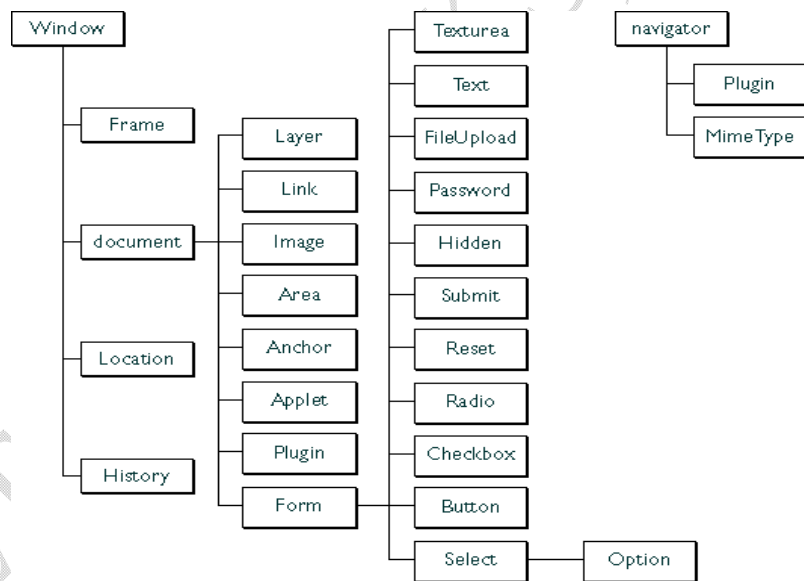


Fig:DOM Hierarchy

In fact, JS elements are mapped to DOM, which in turn is mapped to (different) objects that a web page has.

An object has:  
Properties or attributes  
Methods  
Events

### Properties:

Properties gives the value of different attributes that an object has using different methods, different properties of an object can set and displayed.

A property of an object can be accessed using dot operator as follows:

Both the object name and property name are case sensitive. So, watch your typing because property is a variable, you can create new property by simply assigning it a value.

E.g.

```

puset . batch = "066-069";
puset . faculty = "BCA";
puset . semester = "third";
document . write (puset . semester);

```

**Object\_name.property\_name**

### Method:

Object contain a function, which are known as method when part of an object. Methods can be call using the dot operator as follows:

Any arguments required for the method are passed between the parentheses. just like a normal function.

E.g.

**Window.close();**

**object\_name . method \_ name**

## Events (Signal generated when specific action occur)

Events provide the basis of interaction with the Navigator/Explorer window and currently loaded document. Events are triggered in the browser primarily by the user actions, including loading a page, entering data in a form, and clicking on form buttons e.g. clicking on command button, moving mouse cursor, clicking on check box etc. Associated with every event, we have an event handler (in fact, a function or a method containing JavaScript codes) which is executed when that event occurs.

For eg: when a user clicks on submit button, the event is called “click” and associated with this event, we have an event handler (onclick or onsubmit containing several line of codes) which gets executed in the background, and as a result, data entered in a web page are validated and then forwarded to the web server for further processing.

### Event handlers associated with hyperlink:

Event	Event handler	Description
Click	OnClick	The left button is pressed and released once. Or link
Focus	onFocus	User gives form element input focus
Blur	onBlur	User removes input focus from form element
Mouseover	onMouseOver	The mouse cursor is moved over a hyperlink.
MoseOut	onMousOut	User moves mouse pointer off of link or anchor
Select	onSelect	User selects form element's input field
Submit	onSubmit	User submits a form
Resize	onResize	User resizes the browser window
Load	onLoad	Occur when page is loaded in Navigator.
Unload	onUnload	When user leaves the page.
Change	onchange	When value of a form field is changed by user.

### Which event handlers can be used on what?

onBlur, onChange, onFocus, onSelect: → Selection list.

onBlur, onChange, onFocus, onSelect: → Text element & Text area element.

1. OnClick: → Button/Check box/Radio button/Reset/Submit etc.
2. onclick, onMouseOver: → Hypertext link
3. onLoad, onUnload: → Document/Window
4. onSubmit: → Form

### Other event handlers are:

onAbort, onDragDrop, onError, onkeydown, onkeypress, onmousedown, onmousemove, onmouseout, onmouseover, onReset, onResize etc.

eg.

```
<html>
<head> <title> Event handler </title>
<script language="javascript">
function hello()
{
 name=prompt("Enter your name:", "Name");
 alert("Greeting\t"+name+", welcome to my page!");
}
```

```
function goodbye()
{
 alert('Good bye' +name+',sorry to see you go!');
}
</script></head>
<body onload="hello();" onUnload="goodbye();">
<body></html>
```

## Forms

JS supports form and their associated properties, method, and events handler to produce sophisticated interactive programs.

Forms appear like a manual form, but it is displayed electronically. They provide a mechanism to allow user to enter several information in an electronic manner. Once several information are billed in they are validated (i.e. they are checked individually against some predefine criteria or condition, for e.g. the name of the person must begin with an alphabet, the age must be numeric integer, the name field cannot be left empty, and so on) at the client side itself with the help of browser and then only those validated information are forwarded to the web server for further processing.

### Processing Forms

JavaScript provides the form object to enable your scripts to interact with and exercise control over HTML forms. The form object is accessed as a property of the document object. Your browser creates a unique form object for every form that is contained in a document. These objects can be accessed via the document.forms[] array.

The form object is important because it provides with access to the forms contained in a documents and allows to respond to form-related events.

A webpage can contain one or more than one form. A form is enclosed with the <form > ....</form> tags a given in the following diagram.

<Form >

Form object go in here

</Form>

These tags are used in the body section. Let us consider a webpage that contains more than one form as shown in the following diagram.

Window object contain document object which in turn contains forms, image, hyperlink, etc.

→ <Form > -----</Form> tag creates a form array. In order to access the first form, we use the following

#### Syntax

window.document.name of first firm  
or, window.document.forms[0]

The second form can be accessed as following

window.document.forms[1]

Since window object refers to the current window, in above syntax we can remove window also. In order to access and assign a name, let us say Rajesh, in the first text box of the first form of the above display, we require to do the following.

document.forms[0].name\_of\_the\_text\_box.value="Rajesh";

The form contains the following object:-

Form Elements	Description
Button	A new element which provides a button other than a submit or reset button <input type="Button">
Checkbox	A checkbox <input type="checkbox">
Password	A password text field in which each keystroke appears as an asterisk(*)<input type="Password">
Radio	A radio button<input type="radio">
Reset	A reset button<input type="reset">
Select	A selection list( <select> <option> Option1 </option> <option> Option2</option></select>
Submit	A submit button<input type="submit">
Text	A text field<input type="text">
Text area	A multiple text entry field(<textarea>default text</textarea>
Hidden	A hidden field<input type="hidden">

Each of these elements can be named and referred to by name in **JAVASCRIPT**. Each also has properties and method associated with it.

**Sample:**

<HTML>

<HEAD>

<TITLE>Function and Events</TITLE>

<SCRIPT language="JavaScript">

function add()

{

var f=document.forms[0];

f.t3.value=eval(f.t1.value)+eval(f.t2.value);

}

function subtract()

{

var f=document.forms[0];

f.t3.value=eval(f.t1.value)-eval(f.t2.value);

}

function multiply()

{

var f=document.forms[0];

f.t3.value=eval(f.t1.value)\*eval(f.t2.value);

}

function divide()

{

var f=document.forms[0];

f.t3.value=eval(f.t1.value)/eval(f.t2.value);

}

</SCRIPT>

```

</HEAD>
<BODY>
<FORM>
<TABLE>
<TR>
 <TD>First No.:</TD>
 <TD><INPUT type=text name="t1"></TD>
</TR>
<TR>
 <TD>Second No.:</TD>
 <TD><INPUT type=text name="t2"></TD>
</TR>
<TR>
 <TD>Result :</TD>
 <TD><INPUT type=text name="t3" readonly></TD>
</TR>
<TR>
 <TD colspan=2>
 <INPUT type=button value="Add" onClick="add();">
 <INPUT type=button value="Subtract" onClick="subtract();">
 <INPUT type=button value="Multiply" onClick="multiply();">
 <INPUT type=button value="Divide" onClick="divide();">
 </TD>
</TR>
</TABLE>
</FORM>
</BODY>
</HTML>

```

e.g.

<html>

<head><title>Form&Events</title></head>

<body onload=alert("Welcome") onunload=alert("Bye")>

<form>

<input type="text" value="Puset" name="text1" OnMouseOver='alert("text button")'  
onFocus='alert("U R focusing")' onSelect='alert("U R selecting")'>

</form></body>

</html>

\*\*\*Input tag has many attributes such as type, name, values, etc \*\*\*

### **Textbox**

➔ It is used to enter a line of text

➔ Attributes :Type, Name, Size, etc

<u>Events</u>	<u>Events handler</u>	<u>Description</u>
Blur	On blur	Fires when the object loses focus.
Focus	On focus	Fires when the object gets focused.

Change	On change	Fires when the object is changed
Select	On select	Fires when the object is select

e.g.

```
<body>
<form>
<input type="text" value="PUSET" name="tetx1" size="10"
Onfocus='alert("U R focusing")'
Onselect= 'alert("U R selecting")'>
</form>
</body>
```

**Text Area**

→ Allows to enter multiple line of text

SYNTAX: Inside body tag, we write as follows

```
<text area rows=10 cols=20>
```

```
.....
</text area>
```

### Password

A password is a type of textbox which is used for entering confidential data. Entering data will display as a series of asterisk sign (\*).

Properties type="password" name="....." value="....." size="...."

Events ☐ Same as textbox ☐ Blur, focus, change, select

syntax:

```
<input type="password" name="a" value=".....">
```

### Button

A button other the RESET or SUBBBMIT button, for e.g. button to calculate the sum of the two no., button to find out factorial of number, and so on.

<u>Rest Button</u>	<u>Button</u>	<u>Submit Button</u>
Properties:	Properties:	Properties:
Type="reset"	Type="Button"	Type="submit"
Name="....." value="....."	Name="....." value="....."	Name="....." value="....."
Events: Click, doubleclick, mouseup, mousedown	Events: Click, doubleclick, mouseup, mousedown	Events: Click, doubleclick, mouseup, mousedown

syntax:

Reset =<input type="reset" Name="....." value="...label to be displayed.." >

Button =<input type="Button" Name="....." value="....." >

Submit =<input type="submit" Name="....." value="....." >

### Radio

A radio button allows a user to choose a one from several available option:

☐ Female ☒ Male As shown male is checked so its value will be entered

Properties : Type, Name, Value, Checked

Events : Click, Doubleclick, Mouseup, Mousedown

syntax:<Input type="Radio" **name**="Radio1" value="Puset" checked(optional)> Puset

**Note:** if **name** is same for all buttons then only single button will be selected otherwise multiple button selection can be done.

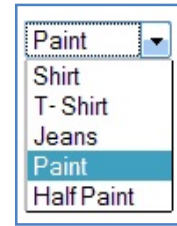
## Select

Select allows a user to display a drop down list to be displayed. Several options that a user can select from scrollable list can be displayed.

Properties : Name size, values

Events: Blur, Focus, Change

Chose Items:



### syntax:

<Select name="Menu1" size=...2.. multiple (allows for multiple selection of object)>

<option value="Shirt">Shirt

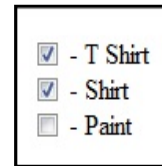
<option value="T Shirt"> T- Shirt

<option value="Jeans"> Jeans

<option value="Paint"> Paint

<option value="Half Paint"> Half Paint

</select>



## Check box

A check box allows a user to select one or more than one from several available option values of checked option values of checked option will be sent to web server.

In this example T shirt and Shirt will be sent to the web server.

Properties : Type, Name, Values, Checked

Events: Click, Double click, Mouse up, Mouse down

**SYNTAX** :<Input type="checkbox" name="check1" value="T-Shirt" checked>T Shirt<br>

Predefined core objects in JavaScript and their inbuilt function.

## **Form Validation:**

Write the code in HTML for form registration and validate the data and also link with css

```
<html><head>
```

```
<script >
```

```
function formvalidation()
```

```
{
```

```
 var name = document.forms["RegForm"]["Name"];
```

```
 var email = document.forms["RegForm"]["EMail"];
```

```
 var phone = document.forms["RegForm"]["Telephone"];
```

```
 var what = document.forms["RegForm"]["Subject"];
```

```
 var password = document.forms["RegForm"]["Password"];
```

```
 var address = document.forms["RegForm"]["Address"];
```

```
 if (name.value == "")
```

```
 {
```

```
 window.alert("Please enter your name.");
```

```
 name.focus();
```

```
 return false;
```

```
 }
```

```
if (address.value == "")
{
 window.alert("Please enter your address.");
 name.focus();
 return false;
}

if (email.value == "")
{
 window.alert("Please enter a valid e-mail address.");
 email.focus();
 return false;
}

if (email.value.indexOf("@", 0) < 0)
{
 window.alert("Please enter a valid e-mail address.");
 email.focus();
 return false;
}

if (email.value.indexOf(".", 0) < 0)
{
 window.alert("Please enter a valid e-mail address.");
 email.focus();
 return false;
}

if (phone.value == "")
{
 window.alert("Please enter your telephone number.");
 phone.focus();
 return false;
}

if (password.value == "")
{
 window.alert("Please enter your password");
 password.focus();
 return false;
}

if (what.selectedIndex < 1)
{
```

```

 alert("Please enter your course.");
 what.focus();
 return false;
 }

 return true;
} </script>

<style>
option {
 font-weight: bold ;
 color: pink;
 clear: left;
 width: 100px;
 font-family: sans-serif, bold, Arial, Helvetica;
 font-size: 12px;
}
p1 {
 box-sizing: border-box;
 width: 100%;
 border: 100px solid pink;
 float: left;
 align-content: center;
 align-items: center;
}
form {
 margin: 0 auto;
 width: 600px;
} </style> </head>
<body>
<h1> REGISTRATION FORM </h1>
<form name="RegForm" action="/submit.php" onsubmit="return formvalidation()"
method="post">
 <p>Name: <input type="text" size=65 name="Name"> </p>

 <p> Address: <input type="text" size=65 name="Address"> </p>

 <p>E-mail Address: <input type="text" size=65 name="EMail"> </p>

 <p>Password: <input type="text" size=65 name="Password"> </p>

 <p>Telephone: <input type="text" size=65 name="Telephone"> </p>

 <p>SELECT YOUR COURSE
 <select type="text" value="" name="Subject">
 <option>BTECH</option>
 <option>BBA</option>
 <option>BCA</option>
 <option>B.COM</option>
 <option>VALIDATION</option>

```

```

</select></p>

<p>Comments: <textarea cols="55" name="Comment"> </textarea></p>
<p><input type="submit" value="send" name="Submit">
 <input type="reset" value="Reset" name="Reset">
</p>
</form> </body> </html>

```

### **Date Object**

JavaScript handles dates similarly to java. The two languages have many of the date method, and both languages stores dates as the number of milliseconds since January 1, 1970, 00:00:00. e.g.

```
Variable1=new Date (); /*method1*/
```

```
Variable1= new Date (year, month, day, hour, minute, second); /*method 2*/
```

It creates a date object and returns the current date and time which will be stored in the object variable “Variable1”.

Once a date object variable is created, following methods can be used to know its properties, such as year, month, day, hour, minute and second.

getDate ()	Return date from the date object in integers (1-32)
getDay ()	Return day of the week from the date object in integers (1-7 or Sunday-Saturday)
getYear ()	Return year from the date object in integers (like 1998, 1990, 2000, etc).
getHour ()	Return hour from the date object in integers (0-24).
getMinute()	Return minute from the date object in integers (0-60).
getSecond()	Return second from the date object in integers (0-60).
setDate ()	Set date of a date object.
setDay ()	set day of a date object.
setYear ()	set year of a date object.

**Similarly, set hour (), set minute (), set second ().**

Set interval is a method used for executing a function repeatedly once certain time has elapsed.

For e.g.

`Ref= setInterval (“function1”, 1000)` will execute the function “function1” once in every 1 second. In order to stop this function repeated execution of the function, `[clearInterval(ref)]` will have to be used.

### **Example**

```

<html>
<head><title>Digital Clock</title></head>
<body>
<script language="javascript">
function funct1()
{
var1=new Date();
var h=var1.getHours();
var m=var1.getMinutes();

```

```

var s=var1.getSeconds();
document.f1.h.value=h;
document.f1.m.value=m;
document.f1.s.value=s;
}
window.setInterval("funct1()",1000);
</script><center>
<form name="f1">
<H1> Digital Clock</H1><hr>

<input type="text" name="h" size="2">
<input type="text" name="m" size="2">
<input type="text" name="s" size="2">
</form></center></body></html>

```

## Event Object

Event object is created automatically when an event is generated, it is used for providing additional information about the event.e.g. when a button is clicked, when key is pressed, when mouse cursor is moved event occurred when ever an event occur this object automatically gets created. One can access different properties of event object.

Screen X - gives X position of the mouse cursor.

Screen Y - gives Y position of the mouse cursor.

Type gives the type of event. Clicks, key press, double click, etc.

Click Here
Status Bar

## Example

### 1. Program

```

<html>
<head>
<title>Event Object</title>
<script language="javascript">
function fun(s)
{
var str=""
str="You placed the mouse cursor on the alphabet"
str+=s
alert(str)
}
function fun1()
{
alert("place the moude cursor over any alphabet")
}
</script></head>
<body>
<center>
A
B
C

```

```
</body>
</html>
```

## Image object

syntax for creating an image object:

```
imag1=new Image();
imag2= new Image(200,300);
```

Once an image Object is created a “gif” or “jpg” or ” jpeg” file can be associated or assigned to is as follows.

```
imag1.src="but1.gif"
imag2.src="logo1.gif"
```

```

<image src="but1.gof">
```

Example:

```
<html>
<head><title>.....</title></head>
<script language="javascript">
imag1=new Image();
imag2= new Image();
imag1.src= "but1.gif";
imag2.src="but2.gif";
function f1()
{document.image[0]=imag2.src;
}
Function f2()
{document.image[0]=imag1.src;
}</script>
<body>
<script language="javascript">

<imag src="but1.gif"></script>
</body></html>
```

## Window Object

Open () method is used to open a window.

Close () method is used to close an already opened window.

### SYNTAX

```
var variable_name=open (URL, name, [option]);
```

### Example

```
var win= open ("http://www.yahoo.com " "Window1");
```

Will open [www.yahoo.com](http://www.yahoo.com) in a window named window1 and it is returned to object variable\_win.win.close(); will close this windows.

Following option can be provided”

Option	Values
Tool bar	yes or no

	1 or 0
Status bar	1 or 0
Resizable	yes or no
Menu bar	yes or no
Width	in the no. of pixel
Height	in the no. of pixel

**Example**

```

<html>
<head><title>Loading a window</title></head>
<script language="javascript">
function f1()
{var win1=open ("http://www.yahoo.com " "Sample" "toolbar="yes", statusbar="yes"
menubar="yes",height='600',width='600');
}
function f2()
{win1=close();
}</script>
<body><center>
<form>
<input type="button" name="Button1" value="load a window" onclick="f1()" ">

<input type="button" name="Button2" value="Unload a window" onclick="f2()" ">

</form></center></body></html>

```

**Document Object**

Bgcolor - back ground color of the document

document.bgcolor="red"

Or, document.bgcolor="#FF0000"

fgcolor - Color of the text display on the document

linkcolor - Color of the hyperlink which is yet to be visited

alinkcolor - Color of the hyperlink which is yet to be activated

vlinkcolor - Color of the hyperlink which is already visited

**# Write a program in JS for different color display when user clicks appropriate button**

```

<html><head><title> Using the document object</title>

```

```

<script language="javascript">

```

```

function f1()

```

```

{ document.bgColor="red"; }

```

```

function f2()

```

```

{ document.bgColor="blue"; }

```

```

function f3()

```

```

{ document.bgColor="green";

```

```

}</script></head>

```

```

<body>
 <center>

```

```

<input type="button" value="red" onclick="f1();" ">

```

Red

Green

Blue

```
<input type="button" value="blue" onclick="f2();">
<input type="button" value="green" onclick="f3();"></center>
</body></html>
```

### History Object

It allows us to navigate through various WebPages already visited on a document.

history.forward()   □ takes to next pages  
 history.go()       □ takes to next Webpage type  
 history.go(2)   □ takes to 2 pages forward  
 history.go(-2) □ takes to 2 pages back

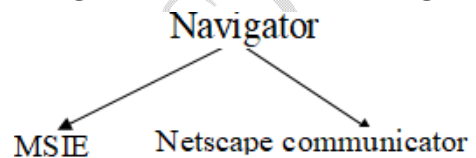
### Navigator Object:

Navigator object helps to know the name and version of the navigator or the browser being used  
 document.write(navigator.appname)

-Gives the name of the browser.

document.write(navigator.appversion)

-Gives the version of the browser.



### String Object:

```
var str1= new string ("This is a pen");
```

```
Str= new string ("Hello");
```

→ Creates a string object variable str1 and associates a string to it.

```
document.write(str.substring(0,3)); //Return hel
```

```
document.write(str.length); // return 5
```

```
document.write(str.toUpperCase()); //return HELLO
```

```
document.write(str.bold()); //return Hello
```

```
document.write(str.charAt(1)); //return h
```

```
document.write(str.indexOf("pen")); //returns 10
```

### Cookies:

Cookies were originally invented by Netscape to give "memory" to web servers and browsers. When you go to a website and the website leaves a txt file on your computer that contains information for the website to use the next time you come to the site.

The HTTP protocol, which arranges for the transfer of web pages to your browser and browser requests for pages to servers, is *state-less*, which means that once the server has sent a page to a browser requesting it, it doesn't remember a thing about it. So if you come to the same web page a second, third, hundred or millionth time, the server once again considers it the very first time you ever came here.

This can be annoying in a number of ways. The server cannot remember if you identified yourself when you want to access protected pages, it cannot remember your user preferences, it cannot remember anything. As soon as personalization was invented, this became a major problem.

Cookies were invented to solve this problem. There are other way to solve it, but cookies are easy to maintain and versatile.

The Advantages of cookies include the fact that the information can be persisted on the user's computer. You can set the cookies expire property to automatically expire the cookie after a

certain time.

### The disadvantages of Cookies

1. Users can disable cookies in their browsers
2. Size restriction by browser around 4kb to 8kb
3. Cannot store structured data in cookies
4. can't leave sensitive information in cookies

Since the cookies reside on the client's computer, any web browser can read the cookies you leave on others' computers. So it is critical you don't leave mission critical information where any smart guy with a browser can pick up your cookies and cause trouble.

### Domain and path

Each cookie also has a domain and a path. The domain tells the browser to which domain the cookie should be sent. If you don't specify it, it become the domain of the page that sets the cookie, in the case of this page [www.quirksmode.org](http://www.quirksmode.org). Please note that the purpose of the domain is to allow cookies to cross sub-domains. My cookie will not be read by [search.quirksmode.org](http://search.quirksmode.org) because its domain is [www.quirksmode.org](http://www.quirksmode.org). when I set the domain to [quirksmode.org](http://quirksmode.org), the search sub domain may also read the cookie. I cannot set the cookie domain to a domain I'm not in, I cannot make the domain [www.microsoft.com](http://www.microsoft.com). Only [quirksmode.org](http://quirksmode.org) is allowed, in this case.

The path gives you the chance to specify a directory where the cookie is active. So if you want the cookie to be only sent to pages in the directory `cgi-bin`, set the path to `/cgi-bin`. Usually the path is set to `/`, which means the cookies is valid throughout the entire domain. This script does so, so the cookies you can set on this page will be sent to any page in the [www.quirksmode.org](http://www.quirksmode.org) domain (though only this page has a script that searches for the cookies and does something with them).

### Java vs Javascript

<u>Java</u>	<u>Javascript</u>
It is a industrial programming language for developing achieving web object.	It is the essential glue that combines with HTML, Java applet, server side, and other wed component into fully integrated web application.
Java is a strongly typed, compiled programming language developed by submicosystem.	It is developed originally by Netscape, is a light weight, interpreted programming language initially called liviscript.
Java is fully committed to object oriented programming and supports all OO (object oriented) features.	It doesn't share java's commitment to object oriented programming.
Java applets are intended to embedded in a web page and their capabilities reflect this intent. Applets are assigned a limited area of the browser window.	It is not implied in the same manner as java applet. Java script are not combined to a limited area of the browser window
Java is neither integrated with nor embedded in HTML.	It is integrated with and embedded in HTML.
It is strongly typed language.	It is loosely typed language.

# \*\*\*\*To display addition of sting and number\*\*\*\*\*

```

<html><body>
<script language="javascript">
var a;var b;var c;
a=parseInt(prompt("Enter first no.(a)","5 hello 2"));
b=prompt("Enter second number(b)","3");
c=eval (a)+eval (b);
d=a+b;
e="2";
f=5;
j=(e+f);
k=eval (e)+eval(f);
l=e-f
document.write("welcome
")
document.write("a+b=",c,"
d=",d,"
j=",j,"
k=",k,
"
l=",l);
document.write("
Purbanchal"+"University");
document.write("
<hr size=4 bg color=red>");
</script></body></html>

```

# \*\*\*\*\*Program in JS to showing a different argument\*\*\*\*\*

```

<html><body>
<script language="javascript">
x=3,y=7,z=9;
document.write("Single argument
");
document.write("x=",x,"
y=",y,"
z=",z)
x=y=z=14;
document.write("
Multiple argument
");
document.write("x=",x,"
y=",y,"
z=",z)
x=(y+17)+(2*(z-2))
document.write("
Multiple argument in one parameter
");
document.write("x=",x,"
y=",y,"
z=",z)
</script>

```

# \*\*\*\*\*Finding Greatest Number\*\*\*\*\*

```

<html><body>
<script language="javascript">
var num1,num2;

```

# To display discount system as given table

Amt<100	No dis
100=< Amt<300	10%
300=< Amt<700	15%
700=< Amt<1000	20%
1000=< Amt	25%

```

<html><body>
<script language="javascript">
var price,amount,discount;
price=prompt("price","Enter price")
if(price<100)
discount=0;
if(price>=100 && price<300)
discount=price*10/100;
if(price>=300 && price<700)
discount=price*15/100;
if(price>=700 && price<1000)
discount=price*20/100;
if(price>=1000)
discount=price*25/100;
amount=price-discount
document.write("the amount to be paid="+amount)
</script></body></html>

```

# \*\*\*\*\*Using Switch\*\*\*\*\*

```

<html><body>
<script language="javascript">
a=prompt("Enter day in numeric value","num")
p=eval(a)
switch(p)
{
case 1:
document.write("Sunday")
break;
case 2:
document.write("Monday")
break;
case 3:
document.write("Tuesday")
break;

```

<pre> num1=prompt("Enter first number","num1") num2=prompt("Enter second number","num2") if(num1&gt;num2) document.write("the greatest number is"+num1) else document.write("the greatest number is"+num2) &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>	<pre> default: document.write("Out of range") }&lt;/script&gt;&lt;/body&gt;&lt;/html&gt; *****<b>Multiplication with 2</b>***** &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; for(i=2;i&lt;=20;i+=2) document.write(i+"&lt;br&gt;") &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>
Array	
<b>Processing new Array</b>  <pre> &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; values=new Array("JS","can","be","used","for", "dynamic","web","page"); for(i=0;i&lt;values.length;i++) document.write(values[i]+ "&lt;br&gt;") cu_name=new Array(4); <b>&lt;!--Array can be used separately also</b> cu_name[0]="BCA" cu_name[1]="BIT" cu_name[2]="BE" cu_name[3]="PGDCA" cu_name[4]="MCA" document.write("&lt;br&gt;" +cu_name[0]); document.write("&lt;br&gt;" +cu_name[1]); document.write("&lt;br&gt;" +cu_name[2]); document.write("&lt;br&gt;" +cu_name[3]); document.write("&lt;br&gt;" +cu_name[4]+ "&lt;br&gt;"); &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; <b>&lt;!--Count and find the sum of the all numbers in the array which are axactly divisible by 7 but not by 5--&gt;</b> &lt;html&gt;&lt;body&gt;&lt;script language="javascript"&gt; var a=new Array(5); var count=0;var sum=0; for(i=0;i&lt;5;i++) { a[i]=prompt("Enter Five element",i+" values") } for(i=0;i&lt;5;i++) {if(a[i]%7==0 &amp;&amp; a[i]%5!=0) { sum=sum+eval(a[i]); count++ document.write("&lt;br&gt;Required array element\t"+a[i]); }} document.write("&lt;br&gt;sum="+sum); document.write("&lt;br&gt;Total number of required no is\t"+count); &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>	<pre> <b>&lt;!-- program showing array and its length</b> &lt;html&gt;&lt;body&gt;&lt;script language="Javascript"&gt; x=new Array(3,6,9,12) sum=0; for(i=0;i&lt;x.length;++i) { sum+=x[i]; } document.write("Sum="+sum+"&lt;br&gt;Length="+x.length) &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; <b>&lt;!--Processing new Array</b> &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var join_values; names_age=new Array("bca",3,"be",7);//1 added bet.parameter join_values=names_age.join(); document.write("joining string with num\t"+join_values+"&lt;br&gt;"); document.write("length of array"+join_values.length) &lt;/script&gt;&lt;/body&gt;&lt;/html&gt;  <b>&lt;!--To Display only vowel from given String</b> &lt;html&gt; &lt;body&gt; &lt;script language="javascript"&gt; var a=new Array(10); var len=new Array(10); var i,j; for(i=0;i&lt;10;i++) { a[i]=prompt("Enter String","") len[i]=a[i].length; for(j=0;j&lt;=len[i];j++) { if((a[i].charAt(j)=='a')  (a[i].charAt(j)=='e')   (a[i].charAt(j)=='i')  (a[i].charAt(j)=='o')  (a[i].charAt(j)==' u')) { document.write(a[i]+"&lt;br&gt;") }} } &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>
<b>Form</b>	Prepared By Naresh kumar Shah, PUSSET (Asso. Prof) 81

**<!-- Form and its elements with numerical processing**

```

<html><head>
<script language="javascript">
function funct1()
{
window.document.myform.text3.value=
eval(window.document.myform.text1.value) +
eval(window.document.myform.text2.value)
}
function funct2()
{
window.document.myform.text3.value=
window.document.myform.text1.value-
window.document.myform.text2.value
}
function funct3()
{
alert("You have cleared a value")
}
</script></head>
<body>
<form name="myform">
Enter yr first number<input type="text"
name="text1" value="" size="10">

Enter yr second number<input type="text"
name="text2" value="" size="10">

yr result number<input type="text"
name="text3" value="" size="10">

<input type="button" name="button1"
value="addition" size="10"
onclick='funct1()'>
<input type="button" name="button2"
value="sub" size="10"
onclick='funct2()'>
<input type="reset" name="reset1"
value="reset" size="10"
onclick='funct3()'>
</form></body></html>
<--Form having different event
<html><body onLoad=alert("welcome")
onUnLoad=alert("bye")>
<form>
<input type="text" value="puset"
name="text1"
onMouseOver='alert("textbutton")'
onFocus='alert("text button activated")'
onSelect='alert("you are selecting")'>
</form></body></html>

```

**<!--Using Checkbox to display values**

```

<html><head><title>Using Radio buttons </title>
<script language="javascript">
function fun1()
{
if(document.form1.check1[0].checked)
{
alert("you are a\t"+document.form1.check1[0].value)
}
else if(document.form1.check1[1].checked)
{
alert("you are \t"+document.form1.check1[1].value)
}
}
</script></head><body><form name="form1">
Check yr gender<input type="checkbox" name="check1"
value="male" checked>Male
<input type="checkbox" name="check1"
value="Female">Female

<input type="button" name="button1" value="Display"
onClick="fun1()">
</form></body></html>
<-- Using check box to display different function
<html><head><title>Using Radio buttons </title>
<script language="javascript">
function funct1()
{
 if(document.myform1.check1.checked)
 {
 document.myform1.text2.value=
 document.myform1.text1.value*
 document.myform1.text1.value
 }
 else
 {document.myform1.elements[3].value=/*May write this
way also*/
 2*(document.myform1.elements[0].value)
 }
}
</script></head><body><form name="myform1">
Enter a Number<input type="text" name="text1">

Your Selection

Square<input type="checkbox" name="check1" value="square"
checked>

double<input type="checkbox" name="check2" value="double">
Your result<input type="text" name="text2">

<input type="button" name="button1" value="click here"
onClick="funct1()">
<input type="reset" value="reset"></form></body></html>

```

<pre> &lt;!--Using password to display different function &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; function funct1() { alert("yr name\t" "+window.document.myform1.text1.value +"\n Yr password:\t"+window.document.myform1. asd.value); } &lt;/script &gt;&lt;/head&gt; &lt;body&gt; &lt;form name="myform1"&gt; your name &lt;input type="text" name = "text1" value="" size="24"&gt;&lt;br&gt; your password&lt;input type="password" name = "asd" value="" size="20"&gt;&lt;br&gt; &lt;input type="button" name = "button1" value="Login" size="20" onclick='funct1()'&gt;&lt;br&gt; &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; </pre>	<pre> &lt;-- Using radio button to display different function &lt;html&gt;&lt;head&gt;&lt;title&gt;Using Radio buttons &lt;/title&gt; &lt;script language="javascript"&gt; function funct1() {     if(document.f1.radio[0].checked)         { document.f1.text2.value= document.f1.text1.value*document.f1.text1.value         } else     { document.f1.elements[3].value=/*May write this way also*/         2*(document.f1.elements[0].value)         } } &lt;/script&gt;&lt;/head&gt;&lt;body&gt;&lt;form name="f1"&gt; Enter a Number&lt;input type="text" name="text1"&gt;&lt;br&gt; Your Selection&lt;br&gt;Square&lt;input type="radio" name="radio" value="" onclick="funct1()"&gt;&lt;br&gt; double&lt;input type="radio" name="radio" onClick="funct1()"&gt;&lt;br&gt;Your result&lt;input type="text" name="text2"&gt;&lt;br&gt; &lt;input type="reset" value="reset"&gt; &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; </pre>
--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

<pre> // To display sum between 1-20 &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var i,sum=0; for(i=1;i&lt;=20;i+=1) sum=sum+i; document.write("sum of 10-20 is:"+sum+"&lt;br&gt;"); &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; ***To display a prime number*** &lt;html&gt;&lt;body&gt;&lt;script language="javascript"&gt; var n=1,i; while(n&lt;=100) { i=2 while(i&lt;n) { if(n%i==0) break; i++; } if(i==n) document.write(n,"&lt;br&gt;"); n++; } &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>	<pre> To display factorial table up to given by user input &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; document.write(prompt("Enter yr name","name")) document.write("\t well come to puset") document.write("&lt;br&gt;factorial Table&lt;br&gt;") n=prompt("Enter Number upto which find factorial","num") //var fact=1;i=1 for(fact=1,i=1;i&lt;=n;fact*=i) { document.write(i+"!"+"fact+"&lt;br&gt;") i++; } &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; To display factorial using recursive method &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; var num3; function fact(num1) { if(num1&gt;1) return num1*fact(num1-1) else return num1; } &lt;/script&gt;&lt;/head&gt; &lt;body&gt;&lt;h1&gt;Finding factorial&lt;/h1&gt;&lt;script language="javascript"&gt; var num1=prompt("Enter num","5") var num2=fact(num1) document.write("factorial of given no is\t"+num2) &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; </pre>
---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

<pre>// To display sum between 1-20 &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var i,sum=0; for(i=1;i&lt;=20;i+=1) sum=sum+i; document.write("sum of 10-20 is:"+sum+"&lt;br&gt;"); &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; ***To display a prime number*** &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var n=1,i; while(n&lt;=100) { i=2 while(i&lt;n) { if(n%i==0) break; i++; } if(i==n) document.write(n,"&lt;br&gt;"); n++; } &lt;/script&gt;&lt;/body&gt;&lt;/html&gt;</pre>	<pre>To display factorial table up to given by user input &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; document.write(prompt("Enter yr name","name")) document.write("\t well come to puset") document.write("&lt;br&gt;factorial Table&lt;br&gt;") n=prompt("Enter Number upto which find factorial","num") //var fact=1,i=1 for(fact=1,i=1;i&lt;=n;fact*=i) { document.write(i+"!"+"fact"&lt;br&gt;") i++; } &lt;/script&gt;&lt;/body&gt;&lt;/html&gt; To display factorial using recursive method &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; var num3; function fact(num1) { if(num1&gt;1) return num1*fact(num1-1) else return num1; } &lt;/script&gt;&lt;/head&gt; &lt;body&gt;&lt;h1&gt;Finding factorial&lt;/h1&gt; &lt;script language="javascript"&gt; var num1=prompt("Enter num","5") var num2=fact(num1) document.write("factorial of given no is\t"+num2) &lt;/script&gt;&lt;/body&gt;&lt;/html&gt;</pre>	<pre>&lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var num=prompt("Enter no","5") fact=1; for(i=num;i&gt;=1;i--) { fact*=i; } alert("fact of\t"+num+"\t is \t"+fact) &lt;/script&gt;&lt;/body&gt;</pre>
--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------

Event	
<pre>&lt;!--Dialogue boxes like: prompt(), alert(), and confirm() &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; function as() { var flag=false,name; name=prompt("Please enter yr name","Puset"); while(flag==false) { flag=confirm("is yr name is really\t"+name) if(flag==false) { name=prompt("Re Enter yr name","Purbanchal"); } } alert(name+"\twellcome to home page"); } &lt;/script&gt;&lt;/head&gt; &lt;body onload=as()&gt;&lt;/body&gt;&lt;/html&gt;</pre>	<pre>&lt;html&gt;&lt;body&gt;&lt;script language="javascript"&gt; function prime(n) { document.write("testing"+n+" ") try { if (n&lt;1  n&gt;20)throw "it out of range" for (var i=2;i&lt;n;++i) if(n%i==0)throw "it is divisible by" +i document.write("Its prime no&lt;br&gt;") } catch (exception) { document.write(exception+"&lt;br&gt;") }} &lt;/script&gt;&lt;/head&gt; &lt;body&gt;&lt;script language="javascript"&gt; for( i=0;i&lt;=21;++i) { prime(i)} &lt;/script&gt;&lt;/body&gt;&lt;/html&gt;</pre>

<pre> &lt;!--Using MouseOver for diff function &lt;html&gt;&lt;head&gt;&lt;script language= "javascript"&gt; function f1(str) { str="u have placed yr mouse curser over"+str; alert(str); } &lt;/script&gt;&lt;/head&gt;&lt;body&gt; &lt;a href="" onMouseOver='f1("A")'&gt;A&lt;/a&gt; &lt;a href="" onMouseOver='alert("B")'&gt;B&lt;/a&gt; &lt;a href="" onMouseOver='alert("C")'&gt;C&lt;/a&gt; &lt;/body&gt;&lt;/html&gt; </pre>	<pre> &lt;!--Using MouseOver,onFocus,onClick for diff. function &lt;html&gt;&lt;head&gt; &lt;script language= "javascript"&gt; num=0; function f1(str) { str="u have placed yr mouse curser over"+str; alert(str); } &lt;/script&gt;&lt;/head&gt; &lt;body&gt; &lt;a href="" onMouseOver='f1("A")'&gt;A&lt;/a&gt; &lt;a href="" onMouseOver='alert("B")'&gt;B&lt;/a&gt; &lt;a href="" onMouseOver='alert("C")'&gt;C&lt;/a&gt; &lt;input type=button value="click many times" onClick='++num,alert("You are pointing times for "+num+"times")'&gt; &lt;input type=button value="activate" onFocus='alert("Button onFocus")'&gt; &lt;/body&gt;&lt;/html&gt; </pre>
Object	
<pre> &lt;!--To open a file in new Window &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; function newWin() { window.open("prime.html"); } &lt;/script&gt;&lt;/head&gt; &lt;body&gt; Click on the link or the button to open the file in new window&lt;br&gt; &lt;a href="#" onClick="newWin();"&gt;open file in new window&lt;/a&gt; &lt;form&gt; &lt;input type=button value="open file in new window" onClick="newWin()"&gt; &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; &lt;!--automatically pause program within 5 seconds &lt;html&gt;&lt;body&gt;&lt;head&gt;&lt;script language="javascript"&gt; function go() { open("prime.html","newWindow");} &lt;/script&gt;&lt;/head&gt; &lt;body onLoad="timeout=setTimeout('go()',5000);"&gt; &lt;form method=POST&gt; &lt;input type=button value="continue...." onClick="clearTimeout(timeout);go();"&gt; &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; </pre>	<pre> &lt;!--Document Object using changing color &lt;html&gt;&lt;body&gt;&lt;head&gt; &lt;script language="javascript"&gt; var colors=new Array("red","white","green","blue"); index=0; var temp; function fun1() { if(index&gt;3) { index=0; document.bgColor=colors[index]; index+=1; } else { document.bgColor=colors[index]; index++; } } function startc() {temp=setInterval("fun1()","500"); } function stopc() { clearInterval(temp); } &lt;/script&gt;&lt;body&gt; &lt;input type="button" value="start" onClick="startc()"&gt; &lt;input type="button" value="Stop" onClick="stopc()"&gt;&lt;/body&gt;&lt;/html&gt; </pre>

<pre> &lt;!--To display Status Bar &lt;html&gt;&lt;body&gt; &lt;a href="5.html" onMouseOver="self.status='Go Home!';return true;"&gt;Home&lt;/a&gt; &lt;a href="prime.html" onMouseOver="self.status='Go next!';return true;"&gt;Next&lt;/a&gt; &lt;/body&gt;&lt;/html&gt; &lt;!--To display about Browser &lt;html&gt; &lt;body&gt; &lt;script&gt; document.write(navigator.appName+"&lt;br&gt;") document.write(navigator.appVersion) &lt;/script&gt; &lt;/body&gt;&lt;/html&gt; </pre>	<pre> &lt;!--program having documet object object to show color &lt;!--program having documet object object &lt;html&gt;&lt;head&gt; &lt;script language="javascript"&gt; function funct1() { document.bgColor="pink"} function funct2() { document.bgColor="blue"} &lt;/script&gt;&lt;/head&gt; &lt;body&gt;&lt;form&gt; &lt;input type="button" value="pink" onClick="funct1()"&gt; &lt;input type="button" value="blue" onClick="funct2()"&gt; &lt;input type="button" value="green" onClick='document.bgColor="green"'&gt; &lt;/form&gt;&lt;/body&gt;&lt;/html&gt; </pre>
<pre> &lt;!--program having different string object &lt;html&gt;&lt;body&gt;&lt;head&gt; &lt;script language="javascript"&gt; var str1=new String("This is a pen"); document.write("length of string is"+str1.length+"&lt;br&gt;"); document.write(str1.substring(2)+"&lt;br&gt;") document.write(str1.substring(6)+"&lt;br&gt;") document.write("INDEX"+str1.indexOf("pen")+"&lt;br&gt; ") document.write("upper case is"+str1.toUpperCase()+"&lt;br&gt;"); document.write("Lower case is"+str1.toLowerCase()+"&lt;br&gt;"); &lt;/script&gt; &lt;/body&gt; &lt;/html&gt; </pre>	<pre> Using setInterval changing color object &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; var colors=new Array("red","green","pink","cyan","tan"); var i=0; function changeColor() { document.bgColor=colors[i]; i++; if(i==colors.length) i=0; } window.setInterval("changeColor()",1000); &lt;/script&gt;&lt;/head&gt;&lt;body&gt; &lt;h1&gt;Test Page&lt;/h1&gt;&lt;/body&gt;&lt;/html&gt; </pre>

Date & Time	Calculator
<pre> &lt;!--To display Live Time &lt;html&gt; &lt;head&gt;&lt;title&gt;Digital Clock&lt;/title&gt;&lt;/head&gt; &lt;body&gt; &lt;script language="javascript"&gt; function funct1() { var1= new Date(); var h= var1.getHours (); var m= var1.getMinutes(); var s= var1.getSeconds (); document.f1.h.value=h; </pre>	<pre> &lt;!--Digital Calculator &lt;html&gt;&lt;body&gt; &lt;script language="javascript"&gt; function fun1(ch) { if(ch=="=") document.f1.text1.value=eval(document.f1.text1.va lue); else { if(ch=="c") document.f1.text1.value=""; </pre>

```

document.f1.m.value=m;
document.f1.s.value=s;
}
window.setInterval("funct1()",1000);
</script><center>
<form name="f1">
<H1> Digital Clock</H1><hr>

<input type="text" name="h" size="2">
<input type="text" name="m" size="2">
<input type="text" name="s" size="2">
</form></center></body></html>
<!-- Simple Date & Time
<html><head><title>Digital Clock</title></head>
<body>
<script language="javascript">
e=new Date();
document.write("Date:"+e.getMonth()+
"/"+e.getDate()+"/"+e.getYear()+"
");
document.write("Time:"+e.getHours()+
"/"+e.getMinutes()+"/"+e.getSeconds());
</script></body></html>
<!-- date and time object using with statement
<html><body>
<script language="javascript">
currentDate=new Date()
with (currentDate)
{
document.write("Date(MDY):"+getMonth()+"/"+g
etDate()+"/"+getYear()+"
")
document.write("Time:"+getHours()+"/"+
getMinutes()+"/"+getSeconds())
}</script></body></html>

```

```

else
document.f1.text1.value+=ch;
}}
</script></head>
<body bgcolor="seagreen" text="marron">
<form name="f1">
Enter Your Expression

<input type="text" name="text1" value=""
size="10">

<input type="button" value="7" onClick='fun1("7")>
<input type="button" value="8" onClick='fun1("8")>
<input type="button" value="9" onClick='fun1("9")>
<input type="button" value="/" onClick='fun1("/")>

<input type="button" value="4" onClick='fun1("4")>
<input type="button" value="5" onClick='fun1("5")>
<input type="button" value="6" onClick='fun1("6")>
<input type="button" value="*" onClick='fun1("*')>

<input type="button" value="1" onClick='fun1("1")>
<input type="button" value="2" onClick='fun1("2")>
<input type="button" value="3" onClick='fun1("3")>
<input type="button" value="-" onClick='fun1("-')>

<input type="button" value="0" onClick='fun1("0")>
<input type="button" value="." onClick='fun1(".")>
<input type="button" value="=" onClick='fun1("=")>
<input type="button" value="+" onClick='fun1("+')>

<input type="button" value="c" onClick='fun1("c")>
<input type="button" value="(" onClick='fun1("(")>
<input type="button" value=")" onClick='fun1(")")>
<input type="button" value="%" onClick='fun1("%')>
</form></body></html>

```

## Introduction to XML

XML is a Extensible Markup Language. It improves the functionality of the web by identifying the information in a more accurate, flexible, and adaptable way. It is extensible because it has not a fix formats like HTML (which is single predefined markup language). Instead, XML is actually a meta markup language for describing other language-which lets you design your own markup languages for limitless different types of document.

Like SGML (unlike HTML), XML is not a mark up language, but a language for defining markup language. Therefore, XML really nothing more than a specification. Specification here means agreed – upon rules on how to create certain kind of documents. XML provides us format for describing structure data. This facilities depends declaration condition and meaningful search result across multiplatform.

## What is XML ?

- XML stands for eXtensible Markup Language is a meta markup language
- XML was designed to describe, transport and store data.
- XML tags are not predefined. You must define your own tags

- XML is a document processing standard that is an official recommendation of the World Wide Web Consortium (W3C)
- XML is a markup language much like HTML but not replacement for HTML
- XML was designed to describe data, not to display data
- XML is designed to be self-descriptive
- XML is a W3C Recommendation

### Feature of XML

- XML can be used on existing protocols.
  - It supports a wide variety of applications.
  - Data integration from disparate sources.
  - Compatible with SGML.
  - More meaningful searches.
  - XML documents are regionable's clear to the person.
  - Development of flexible wave application.
  - It is easy to write program that provides XML documents.
  - Enable a new generation of web-based data viewing and manipulation application.
- Infect, it is actually even easier to create an XML document than HTML document. You can create own XML document without any knowledge of any markup language.

#### Example : A sample XML example.

1. <?XML version = "1.0" ?>
2. <home>
3. <head>
4. <title>My home page</title>
5. <banner source="pulogo.gif">
6. </head>
7. <body>
8. <main.title>Welcome to my home page</main.title>
9. <rule/>
10. <text>
11. <para>Sorry, This home page is still under construction </para>
12. </text>
13. </body>
14. <footer source="footer.gif">
15. </home>

This is very simple example, but it contains all the important parts that you will find in nearly all the document. Type above coding in any editor namely, notepad and save it with xml as the extension. Open browser and view the xml document. Lets now look at what the program actually is

#### Viewing XML Files

- Raw XML files can be viewed in all major browsers.
- Don't expect XML files to be displayed as HTML pages.

**Viewing XML Files:** Look at this XML file: [note.xml](#)

- The XML document will be displayed with color-coded root and child elements. A plus (+) or minus sign (-) to the left of the elements can be clicked to expand or collapse the element structure. To view the raw XML source (without the + and - signs), select "View Page Source" or "View Source" from the browser menu.

```
<?xml version="1.0" encoding="UTF-8"?>
- <note>
 <to>Tove</to>
 <from>Jani</from>
 <heading>Reminder</heading>
 <body>Don't forget me this weekend!</body>
</note>
```

**Line1:** xml header file also called XML declaration. It states which version of the xml standard

**Line 2-15:** Document which have one **root element** and all the other (**child**) elements must be completely enclosed in that element. Xml contains normally, a start tag and an end tag.

**Line 9:** Empty element that is empty and has no comment. It uses special empty tag closed delimiter `</>`.

**Line 5 & 14:** Attributes that give further information about the element the delimit.

### **With XML You Invent Your Own Tags**

The tags in the example above (like `<home>` and `<banner>`) are not defined in any XML standard. These tags are "invented" by the author of the XML document.

That is because the XML language has no predefined tags.

XML allows the author to define his/her own tags and his/her own document structure.

### **The Difference Between XML and HTML**

XML is not a replacement for HTML. XML and HTML were designed with different goals:

HTML	XML
HTML was designed to display data and to focus on how data looks	XML was designed to describe data and to focus on what the data is
HTML is about displaying information	XML is about carrying information
You must use predefined tag	You have to define your own tags
It is case insensitive	It is case sensitive
When HTML is used to display data, the data is stored inside your HTML	XML can separate data from HTML. With XML, your data is stored outside your HTML
It is content unaware	It is content aware
Does not check syntax	Does a syntax check

### **Features:**

**XML is Not a Replacement for HTML, XML is a complement to HTML:** It is important to understand that XML is not a replacement for HTML. In most web applications, XML is used to describe data, while HTML is used to format and display the data.

**XML Separates Data from HTML:** If you need to display dynamic data in your HTML document, it will take a lot of work to edit the HTML each time the data changes. With XML, data can be stored in separate XML files. This way you can concentrate on using HTML/CSS for display and layout, and be sure that changes in the underlying data will not require any changes to the HTML. With a few lines of JavaScript code, you can read an external XML file and update the data content of your web page.

**XML Simplifies Data Sharing:** In the real world, computer systems and databases contain data in incompatible formats. XML data is stored in plain text format. This provides a software- and hardware-independent way of storing data. This makes it much easier to create data that can be shared by different applications.

**XML Simplifies Data Transport:** One of the most time-consuming challenges for developers is to exchange data between incompatible systems over the Internet.

Exchanging data as XML greatly reduces this complexity, since the data can be read by different incompatible applications.

**XML Simplifies Platform Changes:** Upgrading to new systems (hardware or software platforms), is always time consuming. Large amounts of data must be converted and incompatible data is often lost. XML data is stored in text format. This makes it easier to expand or upgrade to new operating systems, new applications, or new browsers, without losing data.

**XML Makes Your Data More Available:** Different applications can access your data, not only in HTML pages, but also from XML data sources. With XML, your data can be available to all kinds of "reading machines" (Handheld computers, voice machines, news feeds, etc.), and make it more available for blind people, or people with other disabilities.

**XML Tree:** XML documents form a tree structure that starts at "the root" and branches to "the leaves".

### **Why should you use XML?**

XML is a [markup language](#), in fact, technically it stands for Extensible Markup Language. By design, it is a carrier for information that needs to be incorporated into a document. Simply put, XML is a briefcase where you store data. Consider five reasons you should use it in your designs.

#### **Simplicity**

XML is easy to understand. You create the tags and overall set up of your document. What could be simpler than that? When writing a page in XML, the element tags are your own creation. You are free to develop a system based on your needs.

#### **Organization**

XML allows you to build your platform by segmenting the design process. Data sits on one page, and formatting rules stay on another. If you have a general idea of what information you need to produce, you can write the data page first then work on the design. XML allows you to produce the site in stages and stay organized in the process.

#### **Accessibility**

With [XML](#) you compartmentalize your work. Separating data makes it accessible when changes are needed. If you write both segments in [HTML](#), you create sections that incorporate the formatting instructions with the information you need to display on the page. When the time comes to change an inventory record or update your details, you must wade through all the code to find a few lines. With XML, separating data makes changes easy and time-saving.

#### **Standardization**

XML is an international standard. This means that anyone in the world is likely to have the ability to view your document. Whether you search for visitors in Alabama or Timbuktu, chances are they can access the page. XML puts the world in your virtual backyard.

#### **Multiple Applications**

You can make one data page and use it over and over again. This means if you are cataloging inventory, you only do it once. You can create as many display pages as you want for that data. XML allows you to generate different styles and formats based on one page of information.

Ultimately, XML is a tool. It keeps your design work organized into practical compartments. The easy nature of the language doesn't require massive amounts of knowledge or an alphabet behind your name. XML saves time and keeps the design flow organized. When you think about it, why wouldn't you use XML?

**Applications:** XML provides a standard method to access information, making it easier for **applications** and devices of all kinds to use, store, transmit, and display data. Major applications are:

##### **1. Web Publishing:**

XML allows user to create interactive pages that can display information which are stored in a structured that can be styled to meet display properties using XSL or XSLT. The reader or viewer can alter or customer those pages as per their need.

##### **2. Searching:**

XML contains data in definite tags which facilitate searching. For example, consider XML pages containing information of different books such as author name, publishing year etc. Now if we search the pages with author name, then this search directly search into author tags only and retrieve directly related data.

##### **3. Independent Data Interchange**

XML can be parsed by almost all programmable language thus an XML generated by human or an application can be parsed another human or application (even in different language). XML is highly adopted and used for interchanging data between web application using APIs. However, an API may return data in different formats such as JSON, but XML is widely used and popular

#### 4. Granular Updates:

Document updates are slow as the entire document needs to be refreshed from the server. In uploading an XML document, only the part of the document is uploaded.

#### 5. Metadata Applications:

XML makes it easier to express metadata in a portable, reusable format.

#### 6. Assists in Data Assessment and Aggregation:

XML document structure is designed in such a way that the documents can be efficiently assessed and aggregated part by part. Another prolific advantage XML is its ability to feature any possible type of data. The data might range from active components such as ACTIVEX and Java applets or multimedia data such as video, image and sound.

#### 7. Custom Tags:

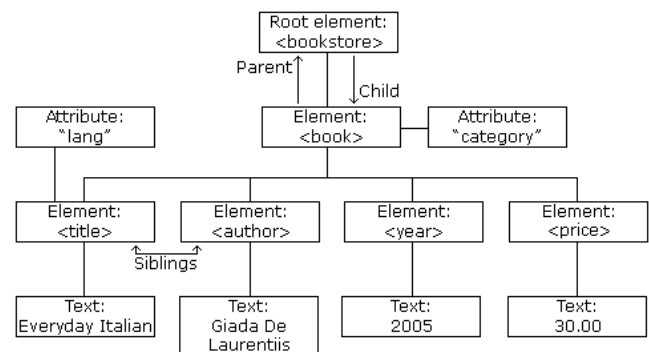
XML lets developer to use custom tags as per their need. This helps in storing data in meaningful tags that helps in understanding the context of the data.

### XML Documents Form a Tree Structure

- XML documents must contain a **root element**. This element is "the parent" of all other elements.
- The elements in an XML document form a document tree. The tree starts at the root and branches to the lowest level of the tree.
- All elements can have sub elements (child elements):
- `<root>`  
`<child>`  
`<subchild>.....</subchild>`  
`</child>`  
`</root>`
- The terms parent, child, and sibling are used to describe the relationships between elements. Parent elements have children. Children on the same level are called siblings (brothers or sisters).
- All elements can have text, content and attributes (just like in HTML).

**Example:** The image above represents one book in the XML below:

```
<bookstore>
 <book category="COOKING">
 <title lang="en">Everyday Italian</title>
 <author>Giada De Laurentiis</author>
 <year>2005</year>
 <price>30.00</price>
 </book>
 <book category="CHILDREN">
 <title lang="en">Harry Potter</title>
 <author>J K. Rowling</author>
 <year>2005</year>
 <price>29.99</price>
 </book>
 <book category="WEB">
 <title lang="en">Learning XML</title>
 <author>Erik T. Ray</author>
 <year>2003</year>
 <price>39.95</price>
 </book>
</bookstore>
```



- The root element in the example is <bookstore>. All <book> elements in the document are contained within <bookstore>.
- The <book> element has 4 children: <title>,< author>, <year>, <price>.

**XML Syntax Rules:** The syntax rules of XML are very simple and logical. The rules are easy to learn, and easy to use.

1. **All XML Elements Must Have a Closing Tag:** In HTML, some elements do not have to have a closing tag like: <p>This is a paragraph. And <br> tag but

In XML, it is illegal to omit the closing tag. All elements **must** have a closing tag:

<p>This is a paragraph.</p> and <br />

**Note:** You might have noticed from the previous example that the XML declaration did not have a closing tag. This is not an error. The declaration is not a part of the XML document itself, and it has no closing tag.

2. **XML Tags are Case Sensitive:** XML tags are case sensitive. The tag <Letter> is different from the tag <letter>.

Opening and closing tags must be written with the same case:

<Message>This is incorrect</message>

<message>This is correct</message>

**Note:** "Opening and closing tags" are often referred to as "Start and end tags". Use whatever you prefer. It is exactly the same thing.

3. **XML Elements Must be Properly Nested (tags cannot be overlap):** In HTML, you might see improperly nested elements:

<b><i>This text is bold and italic</b></i>

In XML, all elements **must** be properly nested within each other:

<b><i>This text is bold and italic</i></b>

In the example above, "Properly nested" simply means that since the <i> element is opened inside the <b> element, it must be closed inside the <b> element.

4. **XML Documents Must Have a Root Element:** XML documents must contain one and only one root element that is the **parent** of all other elements is called the **root** element. The top level element called root element and all other elements are its children.

```
<root>
 <child>
 <subchild>.....</subchild>
 </child>
</root>
```

5. **XML Attribute Values Must be Quoted:**

XML elements can have attributes in name/value pairs just like in HTML. In XML, the attribute values must always be quoted.

Study the two XML documents below. The first one is incorrect, the second is correct:

The error in the first document is that the date attribute in the note element is not quoted.

**Entity References:** Some characters have a special meaning in XML.

- If you place a character like "<" inside an XML element, it will generate an error because the parser interprets it as the start of a new element.
- This will generate an XML error:

1. <note date=12/11/2017> <to>Tove</to> <from>Jani</from> </note>	2 <note date="12/11/2017"> <to>Tove</to> <from>Jani</from> </note>
----------------------------------------------------------------------------	-----------------------------------------------------------------------------

&lt;	<	less than
&gt;	>	greater than
&amp;	&	ampersand
&apos;	'	Apostrophe
&quot;	"	quotation mark

- `<message>if salary < 1000 then</message>`
- To avoid this error, replace the "<" character with an **entity reference**:
- `<message>if salary &lt; 1000 then</message>`
- There are 5 predefined entity references in XML:  
**Note:** Only the characters "<" and "&" are strictly illegal in XML. The greater than character is legal, but it is a good habit to replace it.

**7. Comments in XML:** The syntax for writing comments in XML is similar to that of HTML.

`<!-- This is a comment -->`

**8. White-space is Preserved in XML:** HTML truncates multiple white-space characters to one single white-space:

HTML: Hello	Tove
Output: Hello	Tove

With XML, the white-space in a document is not truncated.

**9. Empty Element:** If tag has no content then xml can be used for short hand for empty element .

Following are equivalent `<title>` `</title>` to `<title/>`

**10. XML Stores New Line as LF**

- Windows applications store a new line as: carriage return and line feed (CR+LF).
- Unix and Mac OSX uses LF.
- Old Mac systems uses CR.
- XML stores a new line as LF.

**11. XML Naming Rules**

XML elements must follow these naming rules:

- Names can contain letters, numbers, and other characters
- Names cannot start with a number or punctuation character
- Names cannot start with the letters xml (or XML, or Xml, etc)
- Names cannot contain spaces
- Any name can be used, no words are reserved.

**12. Empty XML Elements**

An alternative syntax can be used for XML elements with no content:

Instead of writing a book element (with no content) like this:

`<book></book>`

It can be written like this: `<book />`, This sort of element syntax is called self-closing.

**13. Best Naming Practices**

- Make names descriptive: `<first_name>`, `<last_name>`.
- Make names short and simple, like this: `<book_title>` not like this: `<the_title_of_the_book>`.
- Names can start with letters or \_ (underscore), but not numbers or other punctuation character like -(dash). Avoid - (dash), . (dot), and : (colon). After the first character numbers are allowed and names cannot contain space. Strictly speaking, the character is allowed, but the xml specification said that it's "reserved". You should avoid using it in a document.
- Names cannot start with letters, " ", xml.
- There cannot be space after the opening "<" character, the name of the element must come immediately after it. However, there can be space before the closing ">" character, if desired.

Valid in xml are `<first_name>`, `<resume>`

Invalid in xml are:

`<xml_tag>`: start with xml

`<123>`: start with numbers

`<fun=xml>`: "=" sign is used

`<my tag>`: contain space

**Well Formed XML:** XML documents that conform to the syntax rules above are said to be "Well Formed" XML documents.

**What is an XML Element?** An XML element is everything from (including) the element's start tag to (including) the element's end tag.

An element can contain:

- other elements
- text
- attributes
- or a mix of all of the example above,

In the example above, <bookstore> and <book> have **element contents**, because they contain other elements. <book> also has an **attribute** (category="CHILDREN"). <title>, <author>, <year>, and <price> have **text content** because they contain text.

**XML Elements vs. Attributes:** Take a look at these examples:

<pre>&lt;person sex="female"&gt;   &lt;firstname&gt;Anna&lt;/firstname&gt;   &lt;lastname&gt;Smith&lt;/lastname&gt; &lt;/person&gt;</pre>	<pre>&lt;person&gt;   &lt;sex&gt;female&lt;/sex&gt;   &lt;firstname&gt;Anna&lt;/firstname&gt;   &lt;lastname&gt;Smith&lt;/lastname&gt; &lt;/person&gt;</pre>
-------------------------------------------------------------------------------------------------------------------------------------------	--------------------------------------------------------------------------------------------------------------------------------------------------------------

In the first column example sex is an attribute. In the next column, sex is an element. Both examples provide the same information.

There are no rules about when to use attributes or when to use elements. Attributes are handy in HTML. In XML my advice is to avoid them. Use elements instead.

### Uniform Resource Identifier (URI)

A **Uniform Resource Identifier** (URI) is a string of characters which identifies an Internet Resource. The most common URI is the **Uniform Resource Locator** (URL) which identifies an Internet domain address. Another, not so common type of URI is the **Universal Resource Name** (URN). In our examples we will only use URLs.

### Default Namespaces

Defining a default namespace for an element saves us from using prefixes in all the child elements. It has the following syntax:

xmlns="namespaceURI"

This XML carries HTML table information:

```
<table xmlns="http://www.w3.org/TR/html4/">
 <tr>
 <td>Apples</td>
 <td>Bananas</td>
 </tr>
</table>
```

This XML carries information about a piece of furniture:

```
<table xmlns="http://www.w3schools.com/furniture">
 <name>African Coffee Table</name>
 <width>80</width>
 <length>120</length>
</table>
```

### Namespaces in Real Use

XSLT is an XML language that can be used to transform XML documents into other formats, like HTML. In the XSLT document below, you can see that most of the tags are HTML tags.

The tags that are not HTML tags have the prefix `xsl`, identified by the namespace

`xmlns:xsl="http://www.w3.org/1999/XSL/Transform"`:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
<html>
<body>
 <h2>My CD Collection</h2>
 <table border="1">
 <tr>
 <th style="text-align:left">Title</th>
 <th style="text-align:left">Artist</th>
 </tr>
 <xsl:for-each select="catalog/cd">
 <tr>
 <td><xsl:value-of select="title"/></td>
 <td><xsl:value-of select="artist"/></td>
 </tr>
 </xsl:for-each>
 </table></body></html>
</xsl:template>
</xsl:stylesheet>
```

**Unicode:** Unicode is an industry standard for character encoding of text documents. It defines (nearly) every possible international character by a name and a number. Unicode has two variants: UTF-8 and UTF-16.

UTF = **U**niversal character set **T**ransformation **F**ormat.

UTF-8 uses 1 byte (8-bits) to represent characters in the ASCII set, and two or three bytes for the rest.

UTF-16 uses 2 bytes (16 bits) for most characters, and four bytes for the rest.

### **UTF-8 - The Web Standard**

UTF-8 is the standard character encoding on the web.

UTF-8 is the default character encoding for HTML5, CSS, JavaScript, PHP, SQL, and XML.

### **XML Encoding**

The first line in an XML document is called the **prolog**:

```
<?xml version="1.0"?>
```

The prolog is optional. Normally it contains the XML version number.

It can also contain information about the encoding used in the document. This prolog specifies UTF-8 encoding: `<?xml version="1.0" encoding="UTF-8"?>`

The XML standard states that all XML software must understand both UTF-8 and UTF-16.

UTF-8 is the default for documents without encoding information.

In addition, most XML software systems understand encodings like ISO-8859-1, Windows-1252, and ASCII.

### **XML Errors**

Most often, XML documents are created on one computer, uploaded to a server on a second computer, and displayed by a browser on a third computer. If the encoding is not correctly interpreted by all the three computers, the browser might display meaningless text, or you might get an error message.

For high quality XML documents, UTF-8 encoding is the best to use. UTF-8 covers international characters, and it is also the default, if no encoding is declared.

**Conclusion:** When you write an XML document:

- Use an XML editor that supports encoding
- Make sure you know what encoding the editor uses
- Describe the encoding in the encoding attribute
- UTF-8 is the safest encoding to use
- UTF-8 is the web standard

## Converting XML documents in other formats

### Step 1. Open an XML File

To open an XML file, you can drag and drop it in a new tab of your browser, then it will be opened by the browser.

### Step 2. Print the XML

Click the "Print" button in your browser, and select "PDFelement" as the printer to print.

### Step 3. XML to Word

The printed file will be opened in PDFelement, you can click the "Convert" > "To Word" button to convert directly.

**Convert XML to Word** (<https://pdf.wondershare.com/how-to/convert-xml-to-word.html>)

The XML file format, like several other formats, can be opened as well as edited in Microsoft Word. It's a rich text format that is employed for documents in which there is a lot of specific formatting. [PDFelement](#) is one of the most effective and affordable tools you can use to **convert XML to Word**. The conversion process involves 3 steps which are highlighted below. After the process is completed, the extension of your file will change from .xml to .doc.

## Displaying XML with CSS

With CSS (Cascading Style Sheets) you can add display information to an XML document.

### Displaying your XML Files with CSS?

- It is possible to use CSS to format an XML document.
- Below is an example of how to use a CSS style sheet to format an XML document:
- Take a look at this XML file: [The CD catalog](#)
- Then look at this style sheet: [The CSS file](#)
- Finally, view: [The CD catalog formatted with the CSS file](#)
- Below is a fraction of the XML file. The second line links the XML file to the CSS file:

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/css" href="cd_catalog.css"?>
<CATALOG>
 <CD>
 <TITLE>Empire Burlesque</TITLE>
 <ARTIST>Bob Dylan</ARTIST>
 <COUNTRY>USA</COUNTRY>
 <COMPANY>Columbia</COMPANY>
 <PRICE>10.90</PRICE>
 <YEAR>1985</YEAR>
 </CD>
 <CD>
 <TITLE>Hide your heart</TITLE>
```

```
<ARTIST>Bonnie Tyler</ARTIST>
<COUNTRY>UK</COUNTRY>
<COMPANY>CBS Records</COMPANY>
<PRICE>9.90</PRICE>
<YEAR>1988</YEAR>
</CD>
```

```
.....
</CATALOG>
```

### **File\_name.css file**

```
CATALOG {
 background-color: #ffffff;
 width: 100%;
}
CD {
 display: block;
 margin-bottom: 30pt;
 margin-left: 0;
}
TITLE {
 display: block;
 color: #ff0000;
 font-size: 20pt;
}
ARTIST {
 display: block;
 color: #0000ff;
 font-size: 20pt;
}
COUNTRY, PRICE, YEAR, COMPANY {
 display: block;
 color: #000000;
 margin-left: 20pt;
}
```

### **Well Formed XML Documents**

An XML document with correct syntax is "Well Formed".

The syntax rules were described in the previous chapters:

- XML documents must have a root element
- XML elements must have mach up begin-tag and end-tag.
- XML tags are case sensitive
- XML elements must be properly nested
- XML attribute values must be quoted

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

## Valid XML Documents

People who know SGML (Structured Mark-up Language) are used to the concept of validation. In XML, validation means exactly the same thing it does in SGML. For those who are not familiar with this idea the valid document must have document type declaration (DTD), which is a grammar or set of rules that define what text can appear in the document and how they must nest within each other. The document type declaration is also used to declare entities reusable chunks of text that can appear many types but only have to be transmitted once. A document is valid when it conforms to the rules in the document type declaration. Validity is useful because an xml-savvy (platform) editor can use the type declaration to help users to create documents that are valid; such documents are much easier to use and reuse. The major features are:

- A valid XML document is not the same as a well formed XML document.
- The first rule, for a valid XML document, is that it must be well formed.
- The second rule is that a valid XML document must have a document type.
- Rules that define legal elements and attributes for XML documents are often called document definitions, or document schemas.
- A "Valid" XML document must also conform to a specified document type.

## XSL

It stands for *Extensible Stylesheet Language* and is a very powerful language for applying styles to XML documents. XSL describes how the XML document should be displayed. An XML based language is used to create a stylesheet to transform an XML document into another document type, and to format the output. The data you want is at this place in the input document, make it look like this in output. In XML parlance, the input document is called the source tree and output document is the result tree. It consists of three parts:

1. XSLT- a language for transforming XML documents.
2. XPath- a language for navigating in XML documents.
3. XSL-FO- a language for formatting XML documents.

The formatting language allows you to apply styles similar to what CSS does. Of course, the XSLT and XSL-FO, the two languages can be used together, so that XSLT transforms the data, and XSL formatting objects then further modify the data for displaying, much like cascading style sheet. XPath is to find out how XSLT style sheet actually works.

## XSLT

The transformation language is known as XSLT (XSL Transformations). It became a W3C recommendation in November 1999; the specification can be found at <http://www.w3.org/1999/xsl/Transformation>. XSLT allows you to transform your XML document into another form. For example, you could use XSLT to dynamically output some (or all) of the contents of your XML file into an HTML document containing other content.

XSLT Template:

XSLT style sheets are built on a structure called template. A template specifies what to look for in the source tree and what to put into the result tree.

XSLT is written in XML, meaning that there are special elements and attributes used to create style sheets.

```
<xsl:template match="first">first tag found ! </xsl:template>
```

Templates are the heart and soul of XSLT. Style sheets are simply a collection of these templates which will apply to the input document to create the output document.

**XSLT element and their use:** There are different XSLT elements we can use in our style sheets, which will do the actual work of our XSL Transformation.

**Value-of <xsl:value-of>:** This element can be used to extract the value of an XML element and add it to the output stream of transformation. E.g. file saved as "file-name.css"

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
 <html> <body>
 <h1>Please see Output</h1>
 <table border="5" color="green">
 <tr bgcolor="#9acd32">
 <th>Title</th>
 <th>Artist</th>
 <th> country</th>
 <th>price</th>
 </tr>
 <xsl:for-each select="catalog/cd">
 <tr>
 <td><xsl:value-of select="title"/></td>
 <td><xsl:value-of select="artist"/></td>
 <td><xsl:value-of select="country"/></td>
 <td><xsl:value-of select="price"/></td>
 </tr>
 </xsl:for-each>
 </table> </body> </html>
 </xsl:template></xsl:stylesheet>

```

For Source:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="a.xsl"?>
<catalog>
 <cd>
 <title>Empire Burlesque</title>
 <artist>Bob Dylan</artist>
 <country>hmg</country>
 <price>10</price>
 </cd>
 <cd>
 <title>event </title>
 <artist>burmon </artist>
 <country>ggg</country>
 <price>11</price>
 </cd></catalog>

```

Title	Artist	country	price
Empire Burlesque	Bob Dylan	hmg	10
event	burmon	ggg	11

Note: The value of the select attribute is an x-path expression. An x-path expression works like navigating a file system; “/” select sub directories. The result of the transformation above look like as above:

**For-each:** **<xsl:for-each>**: This element can be used to select every xml element of a specified node-set. e.g. **<xsl:for-each select="catalog/cd">**

**sort:** **<xsl:sort>**: Element is used to sort the output. To sort the output, simply add **<xsl:sort>** element inside the **<xsl:for-each>** element in the xsl file.

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
 <html> <body>
 <h1>Please see Output</h1>
 <table border="5" color="green">
 <tr bgcolor="#9acd32">
 <th>Title</th>
 <th>Artist</th>
 <th> country</th>
 <th>price</th>
 </tr>
 <xsl:for-each select="catalog/cd">
 <xsl:sort select="country"/>
 <tr>
 <td><xsl:value-of select="title"/></td>
 <td><xsl:value-of select="artist"/></td>
 <td><xsl:value-of select="country"/></td>
 <td><xsl:value-of select="price"/></td>
 </tr>
 </xsl:for-each>
 </table> </body> </html>
 </xsl:template></xsl:stylesheet>

```

**If <xsl:if>:** Elements is used to put a conditional test against the content of the xml file, add an <xsl:if> element to the xsl document. To add a conditional test, add the <xsl: if> element inside the <xsl:for-each> element in the xsl file.

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
 <html>
 <body>
 <h1>Please see Output</h1>
 <table border="5" color="green">
 <tr bgcolor="#9acd32">
 <th>Title</th>
 <th>Artist</th>
 <th> country</th>
 <th>price</th>
 </tr>
 <xsl:for-each select="catalog/cd">
 <xsl:sort select="country"/>
 <xsl:if test="price >10">
 <tr>
 <td><xsl:value-of select="title"/></td>
 <td><xsl:value-of select="artist"/></td>
 <td><xsl:value-of select="country"/></td>
 <td><xsl:value-of select="price"/></td>
 </tr>
 </xsl:if>
 </xsl:for-each>
 </table>
 </body>
</html>

```

```

 </tr>
</xsl:if>
 </xsl:for-each>
</table> </body> </html>
</xsl:template></xsl:stylesheet>

```

The above code will only output the title, artist and country element of the series that has price that is greater than zero.

**Choose <xsl:choose>:** It is used to conditional test against the xml file. Element is used in conjunction with <xsl:when> and <xsl:otherwise> to express multiple conditional rest.

Syntax: <xsl:choose>

```

<xsl when test="expression">
.....some output.....
</xsl:when>
<xsl:otherwise>
.....some output.....
</xsl:otherwise>
</xsl:choose>

```

**Example:** <?xml version="1.0" encoding="ISO-8859-1"?>

<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">

```

 <html>
 <body>
 <h1>Please see Output</h1>
 <table border="5" color="green">
 <tr bgcolor="#9acd32">
 <th>Title</th>
 <th>Artist</th>
 <th> country</th>
 <th>price</th>
 </tr>
 <xsl:for-each select="catalog/cd">
 <tr>
 <td><xsl:value-of select="title"/></td>
 <xsl:choose>
 <xsl:when test="price >10">
 <td bgcolor="red"><xsl:value-of select="artist"/></td>
 </xsl:when>
 <xsl:otherwise>
 <td><xsl:value-of select="country"/></td>
 </xsl:otherwise>
 </xsl:choose>
 <td><xsl:value-of select="price"/></td>
 </tr>
 </xsl:for-each>
 </table> </body> </html>
</xsl:template></xsl:stylesheet>

```

The above color will add a red color background to the “artist” column when the price of the shading is higher than 10.

**Document type Definition (DTD):** There are different types of document definitions that can be used with XML:

- The original Document Type Definition (DTD)
- The newer, and XML based, XML Schema

### **Document Type Definition-DTD:**

**Introduction:** XML has been describe as a meta mark-up language, i.e. a language for describing mark-up language. Now, concept is begin to learn how to document & describe the new mark-up language that we create. Such mark-up language also known as vocabularies or XML applications are defined via a document type definition (DTD).

Individuals documents can be compared against DTD’S in a process known as validation. If the document matches the constraints listed in the DTD, the document is said valid, if the document does not match the constraints the document is said to be invalid.

**DTD** separating the XML data description from individual application allows all cooperating applications to share a common XML vocabulary is known as document type and each individual document that confirms to the document type is a document instances. This is similar to the basic principal of object oriented (OO) programming; object are grouped and described as a object class (compared to a document type). Each individual object confirming to that class description is known as an instance of that object (compared to document instance).

The XML 1.0 specification provides a standardized means of describing XML document type: the Document Type Definition (DTD). DTD define the rules that set out how a document should be structured, what element should be included, what kind of data may be included and what default values to use. Some other features are:

- A DTD defines the structure and the legal elements and attributes of an XML document.
- The purpose of DTD is to define the legal building blocks of any XML documents
- It defines the structure of a xml document. It defines the document structure with a list of legal elements and attributes.
- Defines what each XML element will contain, i.e. text, child element and so on and also defines the order and nesting of the XML elements.

### **Why Use a DTD?**

With a DTD, your XML files can carry a description of its own format. With a DTD, independent groups of people can agree on a standard for interchanging data. With a DTD, you can verify that the data you receive from the outside world is valid.

### **XML DTD**

*An XML document with correct syntax is called "Well Formed".*

*An XML document validated against a DTD is "Well Formed" and "Valid".*

**Types of DTD:** DTDs are XML documents which can either be in cooperated within the XML document containing the data (internal), or exist as a separated document(external)

**An Internal DTD Declaration:** If the DTD is declared inside the XML file, it must be wrapped inside the <!DOCTYPE> definition:

### **XML document with an internal DTD**

```
<?xml version="1.0"?>
```

```
<!DOCTYPE note [
```

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend</body>
</note>
```

//O/P: In the XML file, select "view source" to view the DTD.

The DTD above is interpreted like this:

- **!DOCTYPE note** defines that the root element of this document is note
- **!ELEMENT note** defines that the note element must contain four elements: "to,from,heading,body"
- **!ELEMENT to** defines the to element to be of type "#PCDATA"
- **!ELEMENT from** defines the from element to be of type "#PCDATA"
- **!ELEMENT heading** defines the heading element to be of type "#PCDATA"
- **!ELEMENT body** defines the body element to be of type "#PCDATA"

### An External DTD Declaration

If the DTD is declared in an external file, the **<!DOCTYPE>** definition must contain a reference to the DTD file:

XML document with a reference to an external DTD

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

*The DOCTYPE declaration, in the example above, is a reference to an external DTD file. And here is the file "note.dtd", which contains the DTD:*

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

**PCDATA:** PCDATA means parsed character data. Think of character data as the text found between the start tag and the end tag of an XML element.

**PCDATA is text that WILL be parsed by a parser. The text will be examined by the parser for entities and markup.**

Tags inside the text will be treated as markup and entities will be expanded. However, parsed character data should not contain any &, <, or > characters; these need to be represented by the & and < and > entities, respectively.

**CDATA:** CDATA means character data.

**CDATA is text that will NOT be parsed by a parser.** Tags inside the text will NOT be treated as markup and entities will not be expanded.

### Using DTD for Entity Declaration

A doctype declaration can also be used to define special characters and character strings, used in the document: **Example**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note [
 <!ENTITY nbsp " ">
 <!ENTITY writer "Writer: Donald Duck.">
 <!ENTITY copyright "Copyright: W3Schools.">
]>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
<footer>&writer; ©right;</footer>
</note>
```

An entity has three parts: an ampersand (&), an entity name, and a semicolon (;).

### XML Schema:

#### Introduction:

Schema are documents that define the valid contents of particular classes of XML documents. The W3C XML schema language has a number of useful characteristics, most notable the ability to specify the data types for text content and attribute values.

Schemas can state that a price element has type double, year attribute contents a number between 9966 and 2018. However, schemas has a no. of other useful characteristics including namespace, awareness and ability to valid complex structure built up out of many different element of many types.

**Definition:** A schema is generally defined as **“The organization or structure of database usually derived from data modeling.”**

This structure typically derived using some shorts of controlled vocabulary that names item of the data and least any constraints that may apply (database, legal, illegal value, special formatting). The relationship between data item (The object of data model) are also an important schema. Other features are:

- An XML Schema describes the structure of an XML document, just like a DTD.
- An XML document with correct syntax is called "Well Formed".
- An XML document validated against an XML Schema is both "Well Formed" and "Valid".

#### An XML Schema:

- defines elements that can appear in a document
- defines attributes that can appear in a document
- defines which elements are child elements
- defines the order of child elements
- defines the number of child elements
- defines whether an element is empty or can include text
- defines data types for elements and attributes
- defines default and fixed values for elements and attributes

### XML Schemas are More Powerful than DTD

- XML Schemas are written in XML
- XML Schemas are extensible to additions
- XML Schemas support data types
- XML Schemas support namespaces

### Why Use an XML Schema?

With XML Schema, your XML files can carry a description of its own format.

With XML Schema, independent groups of people can agree on a standard for interchanging data.

With XML Schema, you can verify data.

### XML Schemas Support Data Types

One of the greatest strength of XML Schemas is the support for data types:

- It is easier to describe document content
- It is easier to define restrictions on data
- It is easier to validate the correctness of data
- It is easier to convert data between different data types

### XML Schemas use XML Syntax

Another great strength about XML Schemas is that they are written in XML:

- You don't have to learn a new language
- You can use your XML editor to edit your Schema files
- You can use your XML parser to parse your Schema files
- You can manipulate your Schemas with the XML DOM
- You can transform your Schemas with XSLT

### XML Schema

XML Schema is an XML-based alternative to DTD:

```
<xs:element name="note">
<xs:complexType>
 <xs:sequence>
 <xs:element name="to" type="xs:string"/>
 <xs:element name="from" type="xs:string"/>
 <xs:element name="heading" type="xs:string"/>
 <xs:element name="body" type="xs:string"/>
 </xs:sequence>
</xs:complexType>
</xs:element>
```

The Schema above is interpreted like this:

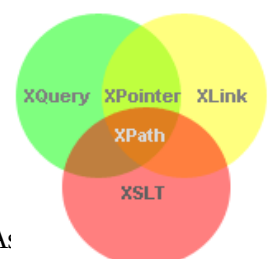
- <xs:element name="note"> defines the element called "note"
- <xs:complexType> the "note" element is a complex type
- <xs:sequence> the complex type is a sequence of elements
- <xs:element name="to" type="xs:string"> the element "to" is of type string (text)
- <xs:element name="from" type="xs:string"> the element "from" is of type string
- <xs:element name="heading" type="xs:string"> the element "heading" is of type string
- <xs:element name="body" type="xs:string"> the element "body" is of type string

### XPointer

- XPointer allows links to point to specific parts of an XML document
- XPointer uses XPath expressions to navigate in the XML document
- XPointer is a W3C Recommendation

### XPointer Example

In this example, we will use XPointer in conjunction with XLink to point to a specific part of another document.



We will start by looking at the target XML document (the document we are linking to):

```
<?xml version="1.0" encoding="UTF-8"?>
<dogbreeds>
<dog breed="Rottweiler" id="Rottweiler">
 <picture url="https://dog.com/rottweiler.gif" />
 <history>The Rottweiler's ancestors were probably Roman
 drover dogs.....</history>
 <temperament>Confident, bold, alert and imposing, the Rottweiler
 is a popular choice for its ability to protect....</temperament>
</dog>
<dog breed="FCRetriever" id="FCRetriever">
 <picture url="https://dog.com/fcretriever.gif" />
 <history>One of the earliest uses of retrieving dogs was to
 help fishermen retrieve fish from the water....</history>
 <temperament>The flat-coated retriever is a sweet, exuberant,
 lively dog that loves to play and retrieve....</temperament>
</dog>
</dogbreeds>
```

Note that the XML document above uses id attributes on each element!

So, instead of linking to the entire document (as with XLink), XPointer allows you to link to specific parts of the document. To link to a specific part of a page, add a number sign (#) and an XPointer expression after the URL in the xlink:href attribute, like this:

xlink:href="https://dog.com/dogbreeds.xml#xpointer(id('Rottweiler'))". The expression refers to the element in the target document, with the id value of "Rottweiler".

XPointer also allows a shorthand method for linking to an element with an id. You can use the value of the id directly, like this: xlink:href="https://dog.com/dogbreeds.xml#Rottweiler".

### **XLink:**

- XLink is used to create hyperlinks in XML documents.
- XLink is used to create hyperlinks within XML documents
- Any element in an XML document can behave as a link
- XLink supports simple links (like HTML) and extended links (for linking multiple resources together)
- With XLink, the links can be defined outside the linked files
- XLink is a W3C Recommendation

### **XLink Syntax**

In HTML, the <a> element defines a hyperlink. However, this is not how it works in XML. In XML documents, you can use whatever element names you want - therefore it is impossible for browsers to predict what link elements will be called in XML documents.

### **XLink Example**

The following XML document contains XLink features:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore xmlns:xlink="http://www.w3.org/1999/xlink">
<book title="Harry Potter">
 <description xlink:type="simple" xlink:href="/images/HPotter.gif" xlink:show="new">
 As his fifth year at Hogwarts School of Witchcraft and
 Wizardry approaches, 15-year-old Harry Potter is.....
 </description>
```

```

</book>
<book title="XQuery Kick Start">
 <description xlink:type="simple" xlink:href="/images/XQuery.gif" xlink:show="new">
 XQuery Kick Start delivers a concise introduction to the XQuery standard.....
 </description>
</book>
</bookstore>

```

**Example explained:**

- The XLink namespace is declared at the top of the document (xmlns:xlink="http://www.w3.org/1999/xlink")
- The xlink:type="simple" creates a simple "HTML-like" link
- The xlink:href attribute specifies the URL to link to (in this case - an image)

The xlink:show="new" specifies that the link should open in a new window

**XLink Attribute Reference**

Attribute	Value	Description
xlink:actuate	onLoad, onRequest	Defines when the linked resource is read and shown: onLoad - the resource should be loaded and shown when the document loads onRequest - the resource is not read or shown before the link is clicked
xlink:show	Embed, new, replace	Specifies where to open the link. Default is "replace"
xlink:type	Simple, extended, arc, locator, resource, title	Specifies the type of link

**XPath? ([https://www.w3schools.com/xml/xpath\\_intro.asp](https://www.w3schools.com/xml/xpath_intro.asp))**

XPath is a major element in the XSLT standard. XPath can be used to navigate through elements and attributes in an XML document.

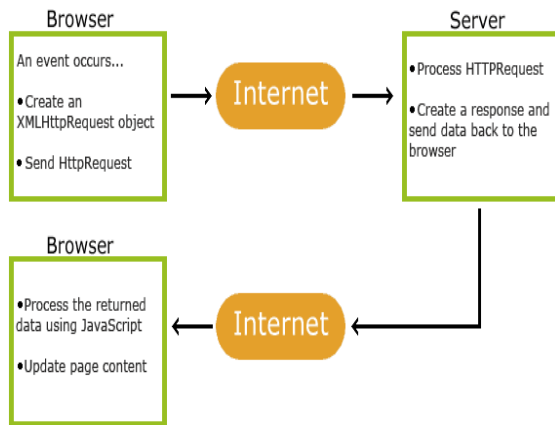
- XPath is a syntax for defining parts of an XML document
- XPath uses path expressions to navigate in XML documents
- XPath contains a library of standard functions
- XPath is a major element in XSLT
- XPath is also used in XQuery, XPointer and XLink

XPath uses path expressions to select nodes or node-sets in an XML document. These path expressions look very much like the expressions you see when you work with a traditional computer file system. XPath expressions can be used in JavaScript, Java, XML Schema, PHP, Python, C and C++, and lots of other languages.

XPath Expression	Result
/bookstore/book[1]	Selects the first book element that is the child of the bookstore element
/bookstore/book[last()]	Selects the last book element that is the child of the bookstore element
/bookstore/book[last()-1]	Selects the last but one book element that is the child of the bookstore element
/bookstore/book[position()<3]	Selects the first two book elements that are children of the bookstore element
//title[@lang]	Selects all the title elements that have an attribute named lang
//title[@lang='en']	Selects all the title elements that have a "lang" attribute with a value of "en"
/bookstore/book[price>35.00]	Selects all the book elements of the bookstore element that have a price element with a value greater than 35.00
/bookstore/book[price>35.00]/title	Selects all the title elements of the book elements of the bookstore element that have a price element with a value greater than 35.00

## AJAX = Asynchronous JavaScript and XML.

- Read data from a web server - after the page has loaded
- Update a web page without reloading the page
- Send data to a web server - in the background
- AJAX is not a new programming language, but a new way to use existing standards.
- AJAX is the art of exchanging data with a server, and updating parts of a web page - without reloading the whole page.
- AJAX is a technique for creating fast and dynamic web pages.
- AJAX allows web pages to be updated asynchronously by exchanging small amounts of data with the server behind the scenes. This means that it is possible to update parts of a web page, without reloading the whole page.
- It is a group of interrelated [Web development](#) techniques

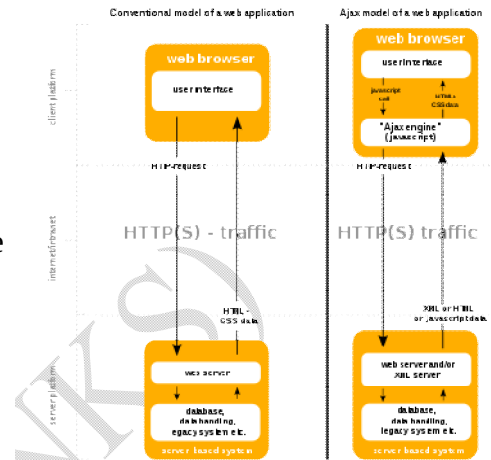


used on the [client-side](#) to create [asynchronous Web applications](#). With Ajax, web applications can send data to and retrieve from a [server](#) asynchronously (in the background) without interfering with the display and behavior of the existing page.

Ajax is not a single technology, but a group of technologies. [HTML](#) and [CSS](#) can be used in combination to mark up and style information. The [DOM](#) is accessed with JavaScript to dynamically display – and allow the user to interact with – the information presented.

JavaScript and the XMLHttpRequest object provide a method for exchanging data asynchronously between browser and server to avoid full page reloads.

Classic web pages, (which do not use AJAX) must reload the entire page if the content should change. Examples of applications using AJAX: Google Maps, Gmail, Youtube, and Facebook tabs.



## How AJAX Works

### Ajax is based on Internet Standards

AJAX is based on internet standards, and uses a combination of:

- XMLHttpRequest object (to exchange data asynchronously with a server)
- JavaScript/DOM (to display/interact with the information)
- CSS (to style the data)
- XML (often used as the format for transferring data)

### Ajax-component

### AJAX is Based on Web Standards

AJAX is based on the following web standards (component):

- HTML (or XHTML) and CSS for presentation
- The Document Object Model (DOM) for dynamic display of and interaction with data
- JSON or XML for the interchange of data, and XSLT for its manipulation
- The XMLHttpRequest object for asynchronous communication
- JavaScript to bring these technologies together

**Implementation:** There is a number of developments in the technologies used in an Ajax application, and in the definition of the term Ajax itself. XML is no longer required for data interchange and, therefore, XSLT is no longer required for the manipulation of data. [JavaScript Object Notation](#) (JSON) is often used as an alternative format for data interchange, although other formats such as preformatted HTML or plain text can also be used.<sup>[14]</sup> A variety of popular JavaScript libraries, including JQuery, include abstractions to assist in executing Ajax requests.

### JavaScript example

An example of a simple Ajax request using the [GET](#) method, written in [JavaScript](#).

#### get-ajax-data.js:

```
// This is the client-side script.
// Initialize the HTTP request.
var xhr = new XMLHttpRequest();
xhr.open('GET', 'send-ajax-data.php');
// Track the state changes of the request.
xhr.onreadystatechange = function () {
var DONE = 4; // readyState 4 means the request is done
var OK = 200; // status 200 is a successful return.
if (xhr.readyState === DONE) {
if (xhr.status === OK) {
console.log(xhr.responseText); // 'This is the output.'
} else {
console.log('Error: ' + xhr.status); // An error occurred during the request.
}
}
};
// Send the request to send-ajax-data.php
xhr.send(null);

send-ajax-data.php:
<?php
// This is the server-side script.
// Set the content type.
header('Content-Type: text/plain');
// Send the data back.
echo "This is the output.";
?>
```

**Filename extensions**

**File formats**

**Influenced by**

**.js**

**javascript**

**JavaScript and XML**

### Ajax Framework

The best technology to build dynamic web pages is [Ajax](#). JavaScript code embedded into the HTML page is used to send requests to the server. At the server side, some processing is required to handle them, find the info or store the data. To do that, we need for a specialized framework. The framework has always a JavaScript part, and sometimes a server side part in another scripting language. A lot of them exist in various programming languages, in all environments around, but we retain here only the most

The AJAX Framework is a cross browser framework that allows developers to quickly develop web pages that can call web services, web pages and other types of content through JavaScript without having to submit the current page.

#### The AJAX Framework is:

- Easy to use.
- Works well with other existing frameworks.
- Supports both "GET" and "POST".
- Works with Internet Explorer 6+, Opera 8+, Safari 3+, Firefox 2+, Google's Chrome and other Mozilla based browsers.

### **Why a framework?**

Actually, this framework is the Ajax engine described by J. J. Garrett and intended to suppress waiting for the user when accessing the server. The framework provides classical, cross-browser functions to use the XMLHttpRequest object. But a framework may go beyond that, and allow to build "rich web applications", applications with a graphical user interface and other features of desktop software that run through a browser, while exchanging data with a remote server.

### **Features of an Ajax framework**

Any Ajax framework is able to communicate with the server, and thus, to read data or to send it data or commands. In the last case a server-side script is required.

The frameworks often add components that make use of the asynchronous communication with the server. The classical examples are buttons, tabbed panels, grids, listboxes and other such widgets.

A more innovative example, the "boxes", are more and more often implemented, and Lightbox and Slimbox are two of them. There are image galleries that place them side by side on the screen and that are making use of Ajax to display them instantaneously.

Framework may be server-driven also, and in this case, component are created on the server with a scripting language such as PHP, and sent to the browser. Ajax is used to transmit user actions to the server part, and to handle the results.

### **What is PHP?**

PHP is a server side scripting language. that is used to develop Static websites or Dynamic websites or Web applications

- PHP means - Personal Home Page, but it now acronym for "Hypertext Preprocessor"
- PHP is a widely-used, open source scripting language
- PHP scripts are executed on the server
- PHP is free to download and use

### **What is a PHP File?**

- PHP files can contain text, HTML, CSS, JavaScript, and PHP code
- PHP code is executed on the server, and the result is returned to the browser as plain HTML
- PHP files have extension ".php"
- The client computers accessing the PHP scripts require a web browser only.

### **What Can PHP Do?**

- PHP can generate dynamic page content
- PHP can create, open, read, write, delete, and close files on the server
- PHP can collect form data
- PHP can send and receive cookies
- PHP can add, delete, modify data in your database
- PHP can be used to control user-access
- PHP can encrypt data

With PHP you are not limited to output HTML. You can output images, PDF files, and even Flash movies. You can also output any text, such as XHTML and XML.

### **Why PHP?**

- PHP runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- PHP is compatible with almost all servers used today (Apache, IIS, etc.)
- PHP supports a wide range of databases
- PHP is free. Download it from the official PHP resource: [www.php.net](http://www.php.net)
- PHP is easy to learn and runs efficiently on the server side

Features Technology	ASP	PHP	JSP
---------------------	-----	-----	-----

Languages	Jscript, VBScript	PHP	Java
Interconnection with databases	Good, any ODBC-compliant database, great couple with MS Access	Good, used mainly with MySql, Oracle, Sybase etc.	Very Good, any JDBC-compliant database
Web Server	IIS	Apachy	Tomcat, JBoss....
OS plateform	Windows 9x, NT, other plateforms requires third party ASP porting products.	Unix (Linux), Windows, MacOS, OS/2	UNIX, Microsoft Windows, Mac OS, Linux
Language packaging and reusing	Weak, separation from HTML absent	Weak, although there are many framework	Strong, java is the best language on the world, packaging reusing and framework are all present
Editors free or commercials	Visual Studio, Primalscript(commercial)	Many, example Netbeans	Many, Netheans is great develop tool
Cost	Free with windows NT, cost for 3 <sup>rd</sup> party software	Free	Free
Platform	Windows only	Multiple	Multiple
External hosting	Widely available although requires fee	Widely available with zero cost	Not so available although free
Framework(s)	Standard framework	Many available	Standard framework
Security	Good	Very good	Good
Hosting popularity	Very good, coupled with ASP.NET	Extremely high, based on apachy	Poor and expensive
Portability	Fair	Fair (DB portability)	Good
Scalability	Good	Fair	Good
Component support	COM component	COM, java classes (version 4)	javaBeans, Enterprise javaBeans
Learning curve	Medium (VBScript, Jscript)	High (C, Perl)	High (java)